參考用書（依十一章主題綱要整理）：Fundamentals of Data Structures in C++, 2nd Edition
Ellis Horowitz, Sartaj Sahni, Dinesh Mehta

資料結構

完整版講義

A4 直式 · 上下兩張 · Lin Cing

目錄

一、ADT、正確性與成本

第 1 章　ADT、正確性與成本　2

二、線性結構與指標操作

第 2 章　陣列、稀疏矩陣與字串　12

第 3 章　堆疊與佇列　23

第 4 章　鏈結串列　51

三、樹、圖與連結關係

第 5 章　樹、二元樹與堆積　65

第 6 章　圖與網路演算法　86

四、排序與雜湊

第 7 章　排序　110

第 8 章　雜湊　122

五、可合併堆積與攤銷分析

第 9 章　進階優先權佇列　137

六、平衡搜尋樹與外部索引

第 10 章　高效率二元搜尋樹　167

第 11 章　多元搜尋樹與外部儲存索引　192

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料結構：契約、表示與成本

資料結構不只是儲存資料的形狀；每個操作都要回答：接受什麼、回傳什麼、成功後狀態如何改變，以及代價如何隨輸入成長。本章以可追蹤的狀態與操作說明這四件事。

演算法與程式

演算法對合法輸入給出明確步驟，並須在有限步後完成；持續運作的服務程式不一定終止。本章把終止性視為操作契約的一部分。輸入可以為空，但操作必須說明空輸入的行為。

先寫抽象值，再談存法

抽象資料型態（ADT）定義可觀察的值與操作；表示（representation）是如何在記憶體中保存它。兩個實作的儲存配置可以不同，但若對每個合法操作產生相同可觀察結果，便實現相同規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有容量 stack 的契約

固定容量 $C \geq 0$，抽象值是長度不超過 C 的有限序列 S，左端為底、右端為頂。create(C) 得到空序列；isEmpty 和 isFull 分別判斷長度為 0、C。push(x) 在未滿時回傳 success 並得到 $S \parallel [x]$；pop 在非空時回傳尾元素並刪除它。滿時 push 回傳 failure，空時 pop 回報錯誤，均不改變 S。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相同值可以重複

stack 是有序序列，不是只記元素的集合：$[2, 2]$ 經一次 pop 仍是 $[2]$。圖中以三個槽位對照實際儲存與抽象序列。

容量 3：從陣列表示得到有序序列

α(a, size) = [a[0], ..., a[size − 1]]

斜線槽位不屬於抽象值；相同元素仍佔不同位置。

表示不變量與抽象函數

對陣列 a、容量 C 和整數 s，合法狀態要求 $0 \le s \le C$，且 $a[0], \ldots, a[s-1]$ 均為有效元素。抽象函數 $\alpha(a, s) = [a[0], \ldots, a[s-1]]$。槽位 $a[s..C)$ 的殘留值不在抽象序列中；不可讀 $a[C]$。

逐步證明 push 保持契約

假設表示不變量成立且 $s < C$。先成功寫入 $a[s] = x$，再令 $s \leftarrow s + 1$。新長度仍不超過 C，舊前綴不變，故新抽象值是 $\alpha(a, s) \,\|\, [x]$。若已滿，先回報錯誤、不寫槽位，抽象值維持原狀。

pop 與失敗狀態

若 $s > 0$，取出 $a[s-1]$ 後令 $s \leftarrow s - 1$；新抽象序列恰好少掉原尾端。若 $s = 0$，不能讀 $a[-1]$；應先回報錯誤。殘留在舊槽位的數值不代表仍在 stack 裡。若 $s > C$，表示本身已非法，抽象函數不適用。

C++ 的值與例外

回傳值可用 `std::optional<T>` 表示空 pop，也可由契約指定例外；兩者必須讓呼叫者知道失敗。若複製或移動 `T`、配置記憶體可能丟出例外，不能先改 `size` 再假設操作成功。只有在所選型別與操作滿足相應不丟例外或可回滾條件時，才承諾失敗後狀態不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋的規格不同，答案也不同

本章用 0-based、非遞減陣列與 lower_bound：回傳第一個不小於 x 的位置 $i \in [0, n]$；若所有元素都小於 x，回傳 n。演算法科 FIRST-MATCH 用 1-based 索引，找不到回 NIL。lower_bound 的 n 是合法插入位置，不等同找不到就回 NIL。

lower_bound 的完整不變量

對自然非遞減順序，每輪維持 $0 \le l \le r \le n$；所有 $0 \le j < l$ 有 $A[j] < x$，所有 $r \le j < n$ 有 $A[j] \ge x$。未分類元素是 $[l, r)$；候選插入位置為閉區間 $[l, r]$。重複鍵必須返回第一個不小於的位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

lower_bound：可執行的更新規則

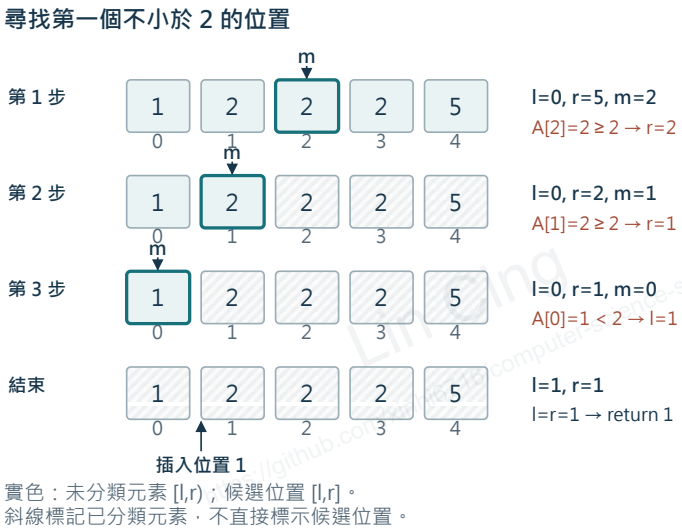
輸入 $A[0..n)$ 已由小到大排列；回傳第一個不小於 x 的位置。右端點不包含在搜尋區間內。

```
l ← 0;  r ← n
while l < r:
    m ← l + floor((r - l) / 2)
    if A[m] < x:  l ← m + 1
    else:         r ← m
return l
```

初始化時兩側區間皆空；每次更新只排除已確認的一側，並保留所有可能的插入位置。即使全部元素都小於 x，回傳 n 也不會讀取 $A[n]$。

一次搜尋的三輪追蹤

取 $A = [1, 2, 2, 2, 5]$、$x = 2$。每次比較後，不變量仍成立；最後 $l = r = 1$，因此回 1。空陣列立即回 0；$x = 6$ 回 5，不會讀取 $A[5]$。



實色：未分類元素 [l,r)；候選位置 [l,r]。
斜線標記已分類元素，不直接標示候選位置。

搜尋不變量：初始化與兩種更新

令 k 是第一個不小於 x 的位置，若不存在則 $k = n$。證明每輪開始時 $k \in [l, r]$。

- **初始化：** $l = 0, r = n$，因此 $k \in [0, n]$。兩側已分類區間均為空。
- **若 $A[m] < x$：** 排序性保證 $A[0..m]$ 全都小於 x，所以 $k \geq m + 1$。設 $l = m + 1$，仍保留 k。
- **若 $A[m] \geq x$：** 排序性保證 $A[m..n)$ 全都不小於 x，所以 $k \leq m$。設 $r = m$，仍保留 k。
- **終止：** 當 $l = r$，唯一的候選位置就是 $k = l$。即使 $k = n$ 或資料含重複鍵，推理仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼搜尋會停止？

當 $l < r$，令 $m = l + \lfloor (r - l)/2 \rfloor$，故 $l \leq m < r \leq n$，讀取 $A[m]$ 合法。若 $A[m] < x$，將 $[l, m]$ 的元素歸入左側；否則將 $[m, r)$ 的元素歸入右側，位置 m 仍可能是答案。整數 $r - l$ 每輪嚴格下降；停止時 $l = r$，兩側已分類區間推出後置條件。

比較器與可執行語意

C++ 比較器 comp 須符合 strict weak ordering，陣列依同一比較器排序。一般形式中，左區皆滿足 comp(A[j],x)，右區皆不滿足；結果是第一個使 comp(A[i],x) 為 false 的位置。先檢查 $i < n$，才能讀取 A[i]；再以 !comp(A[i],x) 與 !comp(x,A[i]) 同時成立確認等價，不能直接用 == 代替。使用 `m=l+(r-l)/2` 需確保索引型別能表示 n，且差與加法不溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較次數不是全部時間

在隨機存取、每次比較 $O(1)$ 的模型下，lower_bound 最壞使用 $\Theta(\log(n+1))$ 次比較。若每個字串最多 L 個字元，一次字典序比較最多做 $O(L)$ 次字元操作；總字元操作為 $O(L\log(n+1))$。含尋址與控制等固定成本，總時間上界是 $O((L+1)(1+\log(n+1)))$。空陣列無元素比較，仍有常數控制成本。

成本先寫清楚

固定空間、輸入儲存、輸出與額外工作空間可以分開報告。時間可按比較、元素搬移或位元操作計算。C++ `vector` 的配置、建構與搬移成本取決於元素型別與 allocator；用單位成本模型時要先聲明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

加法次數與位元成本

依序把 n 個整數加到累加器。若每次加法的**兩個運算元與結果**都能用至多 L 位表示，採逐位加法時，每次成本為 $O(L)$，總成本為 $O(nL)$；$n=0$ 時僅有初始化成本。

只知道最後結果很短並不夠。例如 $2^N+(-2^N)=0$，結果只需常數位，但兩個輸入都需 $\Theta(N)$ 位；光讀取完整輸入便可能需要 $\Omega(N)$ 時間。

若使用固定寬度機器字，且題目保證不溢位，才可在 word-RAM 模型把一次加法計為 $O(1)$，此時總時間為 $O(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴與活躍框架

對非負整數 n，$fac(0)=1$，$fac(n)=n\,fac(n-1)$。求 $fac(3)$ 依序呼叫 3、2、1、0，共 4 次呼叫、3 條呼叫邊；最深同時有 4 個活躍框架。回傳順序得到 1、1、2、6。這裡把每次呼叫算一次，並不把大整數乘法當免費。

Fibonacci 的兩種基底

若 $F(0)=0, F(1)=1$，則 $F(5)=5$；若改成 $G(0)=G(1)=1$ 且使用相同遞迴，則 $G(5)=8$。直接遞迴若把初次呼叫也算入，$C(0)=C(1)=1$、$C(n)=1+C(n-1)+C(n-2)$，故 $C(5)=15$、$C(8)=67$；不要將呼叫次數與框架最大深度混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘以 2 不等於呼叫兩次

採來源層級逐次呼叫的單位成本模型，$n=2^k\geq 1$，基底 $T(1)=\Theta(1)$。一個呼叫 `2*Test(n/2)+8` 的時間為 $T(n/2)+\Theta(1)=\Theta(\log n+1)$；兩個獨立呼叫相加則為 $2T(n/2)+\Theta(1)=\Theta(n)$。不假設編譯器做共同子式消除。

同樣乘以 2，不代表多呼叫一次

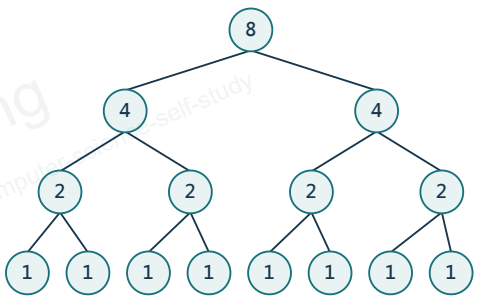
2 × Test(n/2) + 8

每層只計算一次

含基底共 4 次呼叫；時間 Θ(log n)

Test(n/2) + Test(n/2) + 8

每層分成兩次呼叫



含基底共 15 次呼叫；時間 Θ(n)

模型：程式層級計數，未消除或共用重複呼叫；n 從 8 開始，基底為 n ≤ 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

迴圈結束前還有一次測試

固定 $n \geq 1$，對 $i = 1, \ldots, n$ 設 $x = n$，只要 $x \geq 0$ 就做 $x \leftarrow x - i$。內迴圈本體精確執行 $\lfloor n/i \rfloor + 1$ 次；每個 i 還多一次失敗的條件測試。$n = 6$ 時本體次數依序為 7、4、3、2、2、2，合計 20；含最後失敗的測試共 26。

從精確計數推到成長率

總本體次數 $S(n) = \sum_{i=1}^{n} (\lfloor n/i \rfloor + 1)$。因 $n/i - 1 < \lfloor n/i \rfloor \leq n/i$，得到 $nH_n < S(n) \leq nH_n + n$。以積分界 $\log(n+1) \leq H_n \leq 1 + \log n$，因此 $S(n) = \Theta(n \log n)$。此式假設 $n \geq 2$，空範圍另處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

大 O、下界與成本函數

$f \in O(g)$ 意味存在常數 $c > 0, n_0$，使所有 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$；Ω 反向界定，Θ 要上下界都成立。最壞成本是 $W(n) = \max_{|x|=n} T(x)$；O 是界的關係，不等同「最壞情況」。

最壞、期望與攤銷

最壞成本取所有合法輸入的最大值；期望成本須指定輸入或演算法隨機性分布；攤銷成本則對一串操作求總成本，再分配到各次操作。三者可同時出現在同一演算法，不能互相取代。

重複鍵會改變 quicksort

若二路 Lomuto 分割把等值鍵都放到 pivot 同側，全相等輸入每輪為 $n - 1$ 與 0；即使 pivot 均勻隨機，成本仍是 $\Theta(n^2)$。相異鍵且隨機 pivot 的標準模型才有期望 $\Theta(n \log n)$；三路分割會把小於、等於、大於鍵分開，等值區不再遞迴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態陣列的真實搬移

初始容量 $c_0 = 3$，滿時把容量加倍；連續 append 13 個元素時，在第 4、7、13 次分別搬 3、6、12 個，總共 21 個。把任意初始容量誤寫成 $1 + 2 + 4 + 8 = 15$ 會低估本例。

為何一串 append 是線性？

設初始容量 $c_0 \geq 1$。$n = 0$ 時搬移與寫入均為 0；以下假設 $n \geq 1$。若未擴容，搬移為 0；否則搬移容量為 $c_0, 2c_0, \ldots, 2^k c_0$，最後一次舊容量 $c = 2^k c_0 < n$。等比級數總和 $2c - c_0 < 2n$。再加 n 次新元素寫入，總元素寫入小於 $3n$。此處不計初始容量配置成本；若算配置，須依 allocator 與元素建構模型另加。

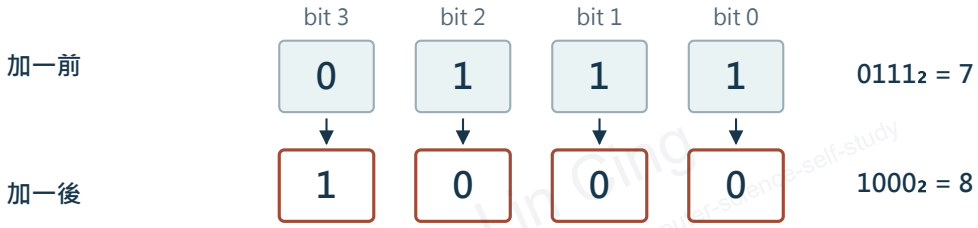
勢能法：讓便宜操作先存下信用

對只做 append 的倍增表，取 $\Phi(s, C) = \max(0, 2s - C)$，初始 $s = 0$ 故勢能為 0。未擴容時勢能增加至多 2，攤銷成本至多 $1 + 2 = 3$。滿表容量 $C > 0$ 時，擴容成本 $C + 1$、勢能由 C 變 2，攤銷成本為 3。勢能非負，故任意前綴的實際總成本不超過攤銷總和。

位元計數器的勢能例子

固定 4 bit、以模 16 計數；每翻轉一位算 1 次寫入，取勢能為 1 的個數。$0111 \to 1000$ 寫 4 位，勢能由 3 降至 1，攤銷成本為 $4 - 2 = 2$。一般翻轉 t 個尾端 1 並設定一個 0，勢能變化 $1 - t$，攤銷成本仍為 2。全 1 回捲全 0 時寫 4 位、勢能減 4。

二進位計數器加一：實際成本與攤銷成本



4 個 bit 翻轉：實際成本 c = 4；勢能 Φ = 1 的個數。

Φ前 = 3，Φ後 = 1；攤銷成本 ĉ = 4 + (1 − 3) = 2。

此例假設位數足夠，忽略計數器整體配置成本。

結束前檢查四件事

規格是否寫出空輸入、重複值與失敗行為？實作狀態是否有可驗證的不變量和抽象函數？證明是否同時處理保持與終止？成本是否明確說明操作、輸入大小和期望或攤銷的適用條件？下一章再把這些工具用在實際的線性結構。

參考資料與署名

Lin Cing，《資料結構》手寫筆記。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

前置知識與本章目標

- 前置知識為第 1 章 ADT、不變量與成本模型，以及 C++ 物件生命週期。
- 能由索引域定義 Array ADT，推導多維索引的位址式，並說明連續儲存的限制。
- 能追蹤稀疏矩陣轉置及字串匹配，辨別「字元」與編碼單位。

Array ADT 的索引契約

- 一維陣列是固定索引域 `{0,…,n-1}` 到元素值的映射。`get(i)`、`set(i,x)` 的前提是 `0≤i<n`。
- 長度改變是動態序列的額外能力，不是固定長度 Array ADT 的必要操作。
- `set(i,x)` 的後置條件為索引 `i` 的值變成 `x`，其餘索引的值不變。
- 連續記憶體是常見表示，不是 ADT 的數學定義。分段儲存也能提供索引語義，但成本可能不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C++ 表示與生命週期

- `std::array<T,N>` 的長度編譯期固定。`std::vector<T>` 的長度可在執行期改變。
- 除特化 `vector<bool>` 外，`vector` 元素連續儲存。`size()` 和 `capacity()` 分別是現有元素數與已配置容量。
- 下標 `operator[]` 不能作為一般越界檢查工具。教材範例對外部索引先檢查，再存取。
- 容量重配會使原先指向元素的指標、參照與迭代器失效。`reserve` 不會把 `size` 增加到指定容量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二維陣列的 row-major（逐列優先）位址

- 對 `R×C` 陣列，以固定寬度元素 `w` 位元組儲存，0 起始座標 `(i,j)` 的元素偏移為 `iC+j`。
- 若起點位址為 `b`，位址式是 `b+w(iC+j)`。此式只對同一連續區塊內的合法元素使用。
- 例子 `R=3,C=4,w=8`，座標 `(2,1)` 的元素偏移為 9，位元組偏移為 72。
- `vector<vector<T>>` 的各列可分別配置，不保證整個矩陣是一個連續區塊，不能直接套用上述整體位址式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐列優先的座標與位址

先數完整列，再數該列內的欄位；位址與元素偏移的單位不同。

Row-major：從座標走到連續位址

矩陣 3×4；選取 (2,1)

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	**(2,1)**	(2,2)	(2,3)

先走過完整的 2 列：2×4
再往右 1 格：+1

offset = 2×4 + 1 = 9

連續元素槽位（數字是元素偏移）

0	1	2	3	4	5	6	7	8	9	10	11
									↑		

元素寬 8 bytes：位元組偏移 9×8=72；位址 b+72。

粗框與箭頭同時標示目標；此公式假設元素位於同一連續儲存區。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多維步幅：row-major 與 column-major

對尺寸 $D_0 \times \cdots \times D_{d-1}$、合法索引 $0 \le i_t < D_t$，row-major 讓最後一個索引變化最快。第 t 維每增加 1，要跨過後面各維大小的乘積：

$$\text{offset}_{row} = \sum_{t=0}^{d-1} i_t \prod_{u=t+1}^{d-1} D_u.$$

Column-major 讓第一個索引變化最快，其步幅改為前面各維大小的乘積：

$$\text{offset}_{col} = \sum_{t=0}^{d-1} i_t \prod_{u=0}^{t-1} D_u.$$

空乘積定義為 1；元素寬 w 時再乘 w 才是位元組偏移。配置及位址運算前要檢查尺寸乘積與索引型別的範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

索引次序與快取

- 在 row-major 儲存中，內圈沿欄索引 `j` 增加，會依序讀取相鄰元素。
- 若內圈沿列索引 `i` 增加，相鄰讀取的元素距離為 `C`，可能減少每個快取列的重用。
- 兩種走訪在 RAM 模型均為 `Θ(RC)`，實際時間仍可能不同。沒有實測前不報告快取速度比。
- 若 `R×C` 會溢位索引型別，必須在配置前檢查乘法範圍。合法的邏輯尺寸不代表機器可以配置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏矩陣的 COO 與 CSR

- `nnz=k` 表示非零項數。COO 用三元組 `(row,column,value)`，即列索引、欄索引與值，儲存，另保留矩陣尺寸。
- 本教材 COO 採 row-major 排序、索引合法、同一座標不重複、不存明確零。輸入若不符合，先排序、合併重複座標，再移除零值。
- CSR 用 `row_ptr`、`col_idx`、`values`。第 `i` 列的非零項位於 `[row_ptr[i],row_ptr[i+1])`。
- CSR 空間為 `O(R+k)`。對非常多空列的矩陣，`R` 的成本不能隱藏。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏矩陣例子

矩陣為 3×4。圖中的編號對應每個非零項；第 1 列的 CSR 區間是 $[1, 3)$，包含 -2 和 7。若 $x = [1, 2, 3, 4]^T$，則 $Ax = [10, 26, 8]^T$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速轉置依欄計數

1. 對原矩陣各欄計數，得到 `count[j]`。
2. 對 `count` 作前綴和，得到轉置後各列的起始位置 `start[j]`。
3. 依原 COO 順序走訪 `(i,j,v)`，把 `(j,i,v)` 寫入 `next[j]`，再增加 `next[j]`。
4. 原始 `start` 保留作為區間界線，不能被寫入游標破壞。

時間 `O(C+k)`，輔助空間 `O(C)`，輸出空間 `O(k)`。輸出若包含完整 CSR 列指標，還要保留 `C+1` 個位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速轉置為何不覆寫

- 原例各欄計數為 `[1,2,0,1]`，前綴起點為 `[0,1,3,3]`。
- 各欄的輸出區間互不相交，長度恰等於該欄的非零項數，因此每個輸入項有唯一輸出位置。
- 依輸入列順序穩定散佈，使轉置後同一列的欄號維持遞增。
- 結果為 `(0,1,-2)`、`(1,0,5)`、`(1,2,4)`、`(3,1,7)`。空欄自然對應空區間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速轉置：每次寫入的位置

對照前綴起點與游標，逐項核對輸出位置；空欄不佔任何輸出槽。

快速轉置：計數、分區與穩定散佈

count = [1, 2, 0, 1] start = [0, 1, 3, 3, 4]
next = [0, 1, 3, 3]（寫入前）

原 COO 項	原 column	寫入位置	轉置後項	更新後 next
(0,1,5)	1	1	(1,0,5)	[0,2,3,3]
(1,0,−2)	0	0	(0,1,−2)	[1,2,3,3]
(1,3,7)	3	3	(3,1,7)	[1,2,3,4]
(2,1,4)	1	2	(1,2,4)	[1,3,3,4]

輸出依位置 0 → 3：
(0,1,−2) (1,0,5) (1,2,4) (3,1,7)

第 2 欄 count=0，因此區間 [3,3) 為空；原矩陣列順序讓各區間穩定有序。
start 保留邊界；next 指向下一個可寫位置。

COO 快速轉置的完整配置與游標

```
transpose(R,C,entries):
    require sorted unique valid nonzero entries
    count=[0]*C
    for (r,c,v) in entries: count[c]+=1
    start=[0]*(C+1)
    for c=0..C-1: start[c+1]=start[c]+count[c]
    next=copy(start[0:C]); out=array(len(entries))
    for (r,c,v) in entries:
        out[next[c]]=(c,r,v); next[c]+=1
    return (C,R,out)
```

同一3×4例子的各項散佈為：i=0的(0,1,5)寫out[1]；i=1的(1,0,−2)寫out[0]；i=2的(1,3,7)寫out[3]；i=3的(2,1,4)寫out[2]。start=[0,1,3,3,4]，結束next=[1,3,3,4]，恰好等於各列右界。每個計數區間不交疊且寫入數等於長度，保證無覆寫與無遺漏；穩定按原row次序散佈，使轉置row內col有序。

空欄2的區間[3,3)長0；R=0,C=4時仍須回傳4×0尺寸與空項目。時間Θ(C+k)，輸出Θ(k)、輔助Θ(C)；若輸入未正規化，排序與座標合併成本必須另計。加法合併也需尺寸完全相同，不只是nnz相同。

稀疏加法與乘法成本

- 兩個已排序 COO 相加，可像合併排序般同時前進。相同座標相加，結果為零就不輸出。
- 輸入非零項數為 k_1,k_2 時，合併成本 $O(k_1+k_2)$，不含輸入正規化。
- 稀疏矩陣向量乘法 $y_i=\Sigma_j A_{ij} x_j$ 以 CSR 執行為 $O(R+k)$，含初始化輸出。
- 稀疏矩陣乘法成本不能只寫 $O(k_1+k_2)$。輸出可能變稠密，且累加同座標需要額外資料結構。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏加法與矩陣向量乘的完整操作

```
add_COO(A,B):
    require same dimensions and canonical row-major entries
    i=0; j=0; out=[]
    while i<len(A) and j<len(B):
        if coord(A[i])<coord(B[j]): out.append(A[i]); i+=1
        else if coord(B[j])<coord(A[i]): out.append(B[j]); j+=1
        else:
            value=A[i].value+B[j].value
            if value!=0: out.append((A[i].row,A[i].col,value))
            i+=1; j+=1
    append remaining A[i:] and B[j:]; return out
CSR_matvec(R,C,row_ptr,col_idx,values,x):
    require len(x)==C
    y=[0]*R
    for r=0..R-1:
        for p=row_ptr[r]..row_ptr[r+1]-1:
            y[r]+=values[p]*x[col_idx[p]]
    return y
```

稀疏加法與矩陣向量乘的完整操作 （續）

令B只有(0,1,−5),(2,0,3)，與本章A相加時，首座標(0,1)抵消；A的(1,0,−2),(1,3,7)先輸出；接著B的(2,0,3)再A的(2,1,4)，結果4項而非直接kA+kB=6項。矩陣向量乘x=[1,2,3,4]，row0累加5×2=10；row1依序−2×1=−2，再+7×4得26；row2為4×2=8。

維持每列區間端點0≤row_ptr[r]≤row_ptr[r+1]≤k，row_ptr[0]=0、row_ptr[R]=k且每個col_idx∈[0,C)。這些是CSR表示不變量，不能只驗三個array長度相同。算術成本假設固定範圍值，不替溢位或任意精度求和承諾常數時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String ADT 與編碼層次

- 字串可定義為字母表 Σ 上的有限序列，基本操作有長度、連接、子字串與比較。
- `std::string` 儲存 `char` 編碼單位，可以含內嵌 `\0`。邏輯長度以 `size()` 為準，不能一律用 C 字串終止規則。
- UTF-8 中一個 Unicode 碼點可占多個位元組，一個使用者感知的字形又可能包含多個碼點。
- 本章匹配演算法預設比較固定大小的符號。若輸入採 UTF-8 位元組，回傳位置也是位元組位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

字串操作的複製與檢視語義

- 若用長度及連續編碼單位表示字串，讀取既有索引可為 O(1)，複製長度 k 的子字串需 Θ(k) 時間與輸出空間。
- 連接長度 N、M 的兩字串並產生獨立結果，需 Θ(N+M) 複製。反覆用新結果複製整個前綴，累積成本可能二次成長。
- 非擁有的字串 view 只保留起點與長度，可常數時間切片，但不延長原字串生命週期。來源被釋放或重配後，view 不能繼續使用。
- 字典序比較要逐符號找第一個差異，最壞成本包含共同前綴長度及結束判定，為 O(共同前綴長度+1)。雜湊值、指標相同或長度相同都不能一般地取代內容相等判定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

直觀匹配與 KMP 前綴函數

- 文字長度 `N`、樣式長度 `M`。逐起點比對的符號比較次數最多為 `O(NM)`；加上初始化與空輸入處理，時間上界為 `O(1+NM)`，如大量重複字元在最後一位才失配。
- 定義 `π[i]` 為樣式 `P[0..i]` 的最長真前綴與後綴共同長度。
- 若目前已匹配 `q` 個字元卻失配，改以 `q=π[q-1]` 嘗試較短的已知邊界，不回退文字讀取位置。
- π 的建立和搜尋各以回退次數的總量分析，總時間 `O(N+M)`，輔助空間 `O(M)`。

回退鏈列出已匹配前綴的較短 border（同為前綴與後綴的區段）。失配後仍可能成立的匹配長度必須是該前綴的 border，因此略過其他長度不會漏解。每次成功延伸至多使 q 增加 1，每次回退嚴格減少 q；建立 π 與搜尋各自的回退總數，都由先前延伸次數支付，不會超過線性數量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 的原創追蹤

- 樣式 `P=ababaca`，前綴函數為 `[0,0,1,2,3,0,1]`。
- 已匹配 `ababa` 後遇到 `b`，樣式下一位期待 `c`。先從 `q=5` 退到 `π[4]=3`。
- 此時 `P[3]=b` 可匹配目前文字字元，得到 `q=4`，並未重新掃描已讀的文字。
- 在文字 `abababaca` 中，完整匹配的 0 起始位置是 2。空樣式在本教材定義為位置 0 的匹配，需在介面明訂。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP：回退時沿用同一個文字字元

圖中的 q 是目前已匹配的樣式前綴長度。

i=5：原本 q=5，P[5]=c 與 T[5]=b 不同。

回退 q ← π[4] = 3；同一個 b 與 P[3] 相符，更新 q=4。

i=8 時 q=7；第一個匹配起點為 8−7+1=2。

q 是已匹配前綴長度；粗框與箭頭標記回退步驟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 建表與搜尋的完整迴圈

```
prefix(P):
    pi=[0]*len(P)
    for i=1..len(P)-1:
        q=pi[i-1]
        while q>0 and P[q]!=P[i]: q=pi[q-1]
        if P[q]==P[i]: q+=1
        pi[i]=q
    return pi
first_match(T,P):
    if len(P)==0: return 0
    pi=prefix(P); q=0
    for i=0..len(T)-1:
        while q>0 and P[q]!=T[i]: q=pi[q-1]
        if P[q]==T[i]: q+=1
        if q==len(P): return i-len(P)+1
    return not_found
```

P=ababaca 建表各步為 i=1:b,q=0→0；i=2:a,0→1；i=3:b,1→2；i=4:a,2→3；i=5:c,3→1→0且不延伸；i=6:a,0→1，得[0,0,1,2,3,0,1]。在T=abababaca，讀每字後q為1,2,3,4,5,4,5,6,7；第5號文字b使q先5→3再增至4，完整匹配終於i=8，起點2。

KMP 建表與搜尋的完整迴圈 （續）

前綴函數建構的迴圈只在i≥1且P非空時取P[q]；搜尋先處理空樣式，且在q=M時立刻回傳，故不讀P[M]。若要列舉重疊匹配，成功後令q=pi[M−1]再繼續，而非直接清零。固定符號比較下總O(N+M)；UTF-8若以byte處理，位置仍是byte offset，不能暗稱字形索引。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 vector.overview 與 vector.capacity；Knuth, Morris, Pratt, Fast Pattern Matching in Strings (1977)。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先問取出順序，再決定表示法

Stack 與 queue 都保存一串值，差別是「下一個能取出誰」。本章先建立操作契約，再追蹤索引與物件，最後用同一套狀態思考搜尋及運算式。

學習順序	要能自己回答的問題
堆疊與陣列	top 指向哪裡？擴容為何不是每次都很貴？
環狀佇列	實體位置繞回後，FIFO 次序如何保持？
資料結構互作	兩個 LIFO 如何合成 FIFO？成本如何攤銷？
搜尋	回溯時記住什麼？BFS 為何保證最少邊數？
運算式	運算子何時輸出？左右運算元如何區分？

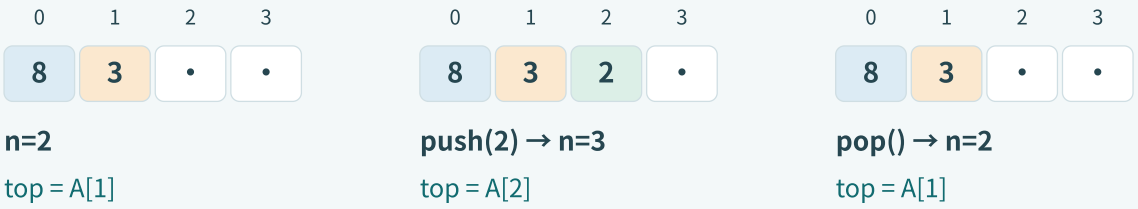
前置知識：陣列、半開區間與基本迴圈不變量。程式使用的空／滿政策與時間成本，會在各節明訂。

3.1　同一端進出：堆疊的操作契約

- 抽象狀態為序列 $[a_0,\ldots,a_{n-1}]$，頂端在末端。`push(x)` 把 `x` 接在末端。
- `top()` 回傳最後一個元素但不改狀態。`pop()` 移除末端元素，前提是非空。
- 在空堆疊呼叫 `pop` 的行為由介面定義。可回傳 optional、例外或錯誤碼，不能默默讀取無效記憶體。
- 表示為動態陣列時，`top`、`pop` 為常數時間，`push` 為攤銷常數時間。固定容量版本另有滿溢結果。

push 與 pop：先看得見狀態

容量4；索引由0起算，陰影區是活物件



序列均由**底端寫到頂端**。`top()` 只讀取 3；`pop()` 移除 3 後，新頂端才變成 8。沒有元素時，不能計算並讀取 `A[n-1]`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用元素數 n，避免空堆疊的負索引

- 不變量為 `0≤n≤capacity`，有效元素位於 `[0,n)`。頂端是 `A[n-1]`，只有 `n>0` 才能存取。
- `push(x)` 先確保有容量，再寫 `A[n]`，最後增加 `n`。若配置失敗，不應把 `n` 提前增加。
- `pop` 先檢查 `n>0`，再移除最後一個有效物件。泛型 C++ 實作還要處理析構，不只是改索引。
- 以 `n=0` 表示空狀態可避免以無號索引表示 `top=-1` 的問題。

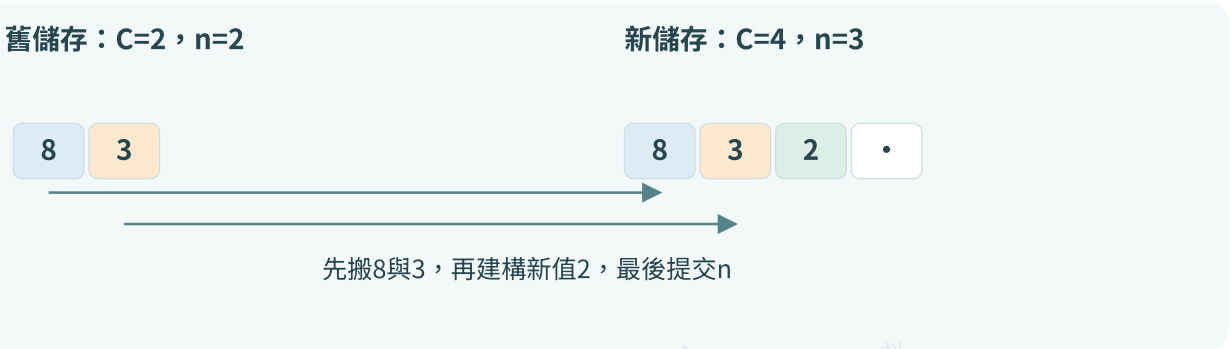
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

配置空間，不等於建構物件

設 `storage` 是容量 C 的未初始化儲存，n 是活物件數。抽象序列等於 `storage[0..n)`；不能把「配置了 C 格」誤當成「已建構 C 個 T」。

```
top(): require n>0; return const T& to storage[n-1]
push(T value):           // snapshot constructed before entering the body
    if n==C: grow_to(max(1,2*C))
    construct_at(storage+n, value)   // 成功後才提交 n
    n=n+1
pop():
    require n>0
    destroy_at(storage+n-1)
    n=n-1
```


擴容時，究竟搬了多少值？



第三次 push 是線性時間；**攤銷常數不代表每次最壞常數**。本例 pop 不縮容。

push(8)、push(3)、push(2)、pop() 的抽象序列依序為 [8]、[8,3]、[8,3,2]、[8,3]。

倍增成本：把整串操作加起來

從空容量開始，滿時依序擴為 1、2、4、8……。對 m≥1 次 push，最後容量 C 是不小於 m 的最小二次冪，因此 m≤C<2m。

$$\text{搬移次數} = 1 + 2 + 4 + \cdots + C/2 = C - 1 < 2m.$$

另有 m 次新值建構，所以總工作量為 Θ(m)，每次 push 攤銷 Θ(1)。這項結論假設每次元素建構／搬移有固定成本；若 T 是長字串或大矩陣，必須另計值操作成本。

若要混合 pop，這個**不縮容版本**仍可把每個 pop 的常數工作直接加上。若一滿就倍增、一半空就立即減半，可能反覆搬移；縮容門檻必須另外設計與證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

push(top())：先保存值，再搬家

容量 1、內容 [7] 時，參數若是 `const T&` 且直接指向舊 top，擴容釋放舊儲存後，這個參照便失效。`noexcept` 不會讓它重新有效。

本章的按值介面 push(T value)	狀態與理由
進入函式前複製參數	value=7，已是獨立物件
成功擴容並搬入原值	新容量2，舊有序列仍為 [7]
從 value 建構新元素	新物件成功後才把 n 改為2
操作完成	[7,7]；舊儲存的參照不再可用

這裡限定 T 可複製建構，且參數快照的成本也計入呼叫。不可複製的 T 不支援這個複製式 `push(top())`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

例外保證：失敗發生在哪個階段？

失敗點	本節偽碼能保證什麼？
按值參數複製失敗	尚未進入本體；n、capacity 與既有值不變
擴容搬移失敗	要還原原狀，需合適的複製或 noexcept move 策略與清理；不能默認任意 T 都可做到
擴容成功後，新元素建構失敗	n 不增加、原值仍有效；capacity 與舊參照可能已改變

不能把「size 沒增加」說成整個 push 具有強例外保證。 強保證要求呼叫失敗時可觀察狀態不變。這裡的泛型說明另假設析構不拋例外，實際配置器、容量乘二的整數上界與清理流程須由實作處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2　從另一端取出：佇列的操作契約

- 抽象序列同樣為 $[a_0,\ldots,a_{n-1}]$。`enqueue(x)` 加在末端，`dequeue()` 移除開頭。
- 若每次移除開頭都把陣列左移，單次 `dequeue` 為 $\Theta(n)$。
- 改以環狀陣列儲存，可避免為保留邏輯順序而搬動所有剩餘元素。
- FIFO 描述的是回傳次序，不保證多執行緒安全，也不等於每個工作具有相同執行時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀陣列：邏輯順序與實體槽位分開

- 令容量 `C>0`，儲存 `head` 和元素數 `size`。第 `k` 個邏輯元素在 `(head+k) mod C`。
- 空條件 `size=0`，滿條件 `size=C`。不需浪費一格區分 `head==tail` 的歧義。
- enqueue 寫入 `(head+size) mod C` 後增加 `size`。dequeue 讀 `head`，使 `head=(head+1) mod C`，再減少 `size`。
- 每次操作最壞 `O(1)`，前提是固定容量及常數時間元素移動。擴容時需依邏輯順序複製，單次為 `O(n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

索引繞回，值的先後次序不變

容量為 4，依序加入 `A,B,C`，移除兩次，再加入 `D,E,F`。

狀態	head	size	邏輯序列
加入 ABC	0	3	A,B,C
移除兩次	2	1	C
加入 D,E	2	3	C,D,E
加入 F	2	4	C,D,E,F

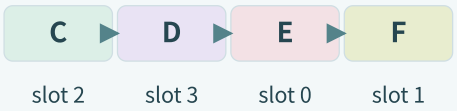
最後實體位置 0、1、2、3 依序存 `E,F,C,D`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

值 C 在實體槽2，卻是邏輯上的第一個

實體儲存：head=2，size=4，C=4

邏輯次序：從 head 開始讀



slot(k) = (head+k) mod C

同一個值C：實體槽2 ↔ 邏輯位置0；不是搬移值來整理FIFO

head=2、size=4；依序讀取槽 2→3→0→1，得到 C→D→E→F。滿佇列不能再 enqueue；本章不覆寫最舊值。

固定容量：先驗證，再改狀態

以 head、size 區分空滿，容量 C>0。以下是固定容量版本；`try_enqueue` 遇滿回報失敗，不覆寫最舊值。回傳值的移動成本與例外政策由元素型別契約決定。

```
slot(k):
    require 0<=k<C
    return head+k if k<C-head else k-(C-head)
try_enqueue(x):
    if size==C: return full
    construct_at(A+slot(size),x)
    size=size+1; return success
```

出列：回傳值、銷毀物件、推進 head

```
front(): require size>0; return reference(A[head])
dequeue():
    require size>0
    value=read_or_move(A[head])
    destroy_at(A+head)
    head=0 if head==C-1 else head+1
    size=size-1
    return value
```

先假設值操作成功且析構不拋例外。泛型回傳的 move 若可能失敗，需另訂例外契約，不能默認回傳失敗一定可還原。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用抽象映射核對入列與出列

分支式 slot 避免先算 head+k 溢位。呼叫時先確認未滿，故 k=size<C；出列最後一項後 head 可停在任一合法槽，不需強制歸零。正確性由 `(head+k) mod C` 的抽象映射驗證：入列只新增第 size 個邏輯值，出列後原第 k+1 個值成為新第 k 個值。兩者固定容量最壞 Θ(1)，並非只靠平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

擴容必須依邏輯順序重新排列

從 physical=[E,F,C,D]、head=2、size=4、C=4 擴成 C′=8。若只把四格複製至新陣列前四格卻沿用 head=2，新取模會把原應回繞的 E 指向尚未建構的槽4，立即破壞表示。

邏輯 k	舊槽 (2+k) mod 4	值	新槽
0	2	C	0
1	3	D	1
2	0	E	2
3	1	F	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搬移成功後，才提交新儲存

```
allocate new capacity C'
for k=0..size-1: construct new[k] from old[slot(k)]
commit new storage; head=0; C=C'; destroy/release old storage
```

提交後邏輯值仍 C,D,E,F；append G 寫新槽4。擴容單次 Θ(size)，倍增且僅在滿時成長，從空開始入列可按相同幾何級數得攤銷 Θ(1)。固定容量即時系統若要求最壞常數，不能把這項擴容悄悄包含在內。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

空、滿與搬移成本，分別判斷

線性左移：搬B與C

head=0, size=3 → head=0, size=2

環狀：B與C留在原槽

head=0, size=3 → head=1, size=2

C=4；空與滿可有相同head，以size=0或4區分

固定容量4時，空與滿可能有相同 head；必須用 size 或另一套明確約定區分。線性左移版出列要搬移所有剩餘值，環狀版只推進 head 並銷毀原首物件，兩者保持相同 FIFO 抽象值，但成本不同。

環狀佇列的三種空滿慣例

令陣列長度 $C \geq 2$。本章的 `head,size` 使用全部 C 格：`size=0` 為空，`size=C` 為滿。另一種 front/rear 慣例讓 front 位於首元素之前，預留一格：`front=rear` 為空，`(rear+1) mod C=front` 為滿，最多保存 $C-1$ 個值。

若仍令 front 位於首元素之前，也可加一個 tag 區分 `front=rear` 的兩種狀態：tag=false 表示空，tag=true 表示滿，因此可使用全部 C 格。只在兩索引相等時依 tag 判定空滿；非空時首元素是 `(front+1) mod C`，對應 head/size 的 `head`。不同實作必須先寫出完整更新與失效條件，不能混用空滿測試。

Tag 版本：何時更新狀態？

初始 `front=rear=0, tag=false`。假設元素的建構、讀取與銷毀都成功：

```
enqueue(x):
    if front==rear and tag: return full
    rear=(rear+1) mod C; store x at rear
    if rear==front: tag=true
dequeue():
    if front==rear and not tag: return empty
    front=(front+1) mod C; take and remove A[front]
    if front==rear: tag=false
    return value
```

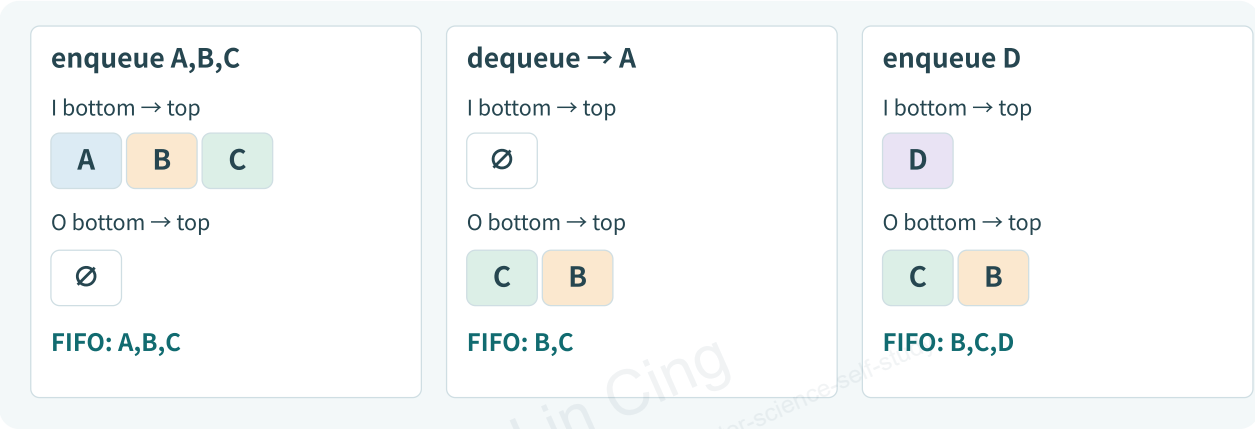
當 `front!=rear`，tag 可保留先前值，不參與空滿判定。這是固定容量的抽象步驟；泛型物件若會丟例外，必須另行安排提交順序，不能直接宣稱失敗時狀態不變。

3.3　兩個堆疊，合成一個 FIFO

令 I 保存新入列值、O 保存待出列值；兩個序列都由底寫到頂。

```
enqueue(x): I.push(x)
dequeue():
    if O.empty():
        while not I.empty(): O.push(I.pop())
    require not O.empty()
    return O.pop()
```

舊值在 O，新值在 I，不能插隊



抽象 FIFO 是 `reverse(O) + I`。**只有 O 空時才搬移**；O 還有舊值時，I 的新值不能越過它們。

一次搬很多，為何仍可攤銷常數？

先把每次 stack push/pop 的成本定為1，底層使用最壞常數的節點堆疊；不把配置器延遲或例外納入此抽象模型。取勢能 $\Phi = 2|I|$，初始為0。

操作	實際成本 c	ΔΦ	攤銷成本 c+ΔΦ
enqueue	1	+2	3
O 非空時 dequeue	1	0	1
O 空，搬 k≥1 值後 dequeue	2k+1	−2k	1

由 $\sum c = \sum \hat{c} + \Phi_0 - \Phi_{final} \le \sum \hat{c}$，m 次成功操作總成本 O(m)。空佇列的失敗檢查另為常數。

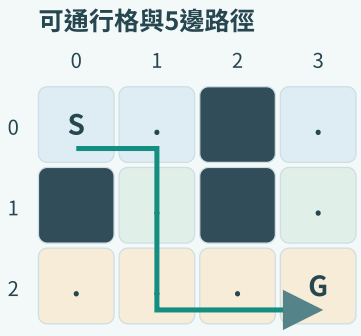
這不是「最壞每次 O(1)」：某次 dequeue 可以搬 Θ(n) 個值。若底層 stack 也會動態擴容，須再加入它的攤銷成本，不能把兩層成本都略去。

3.4　迷宮是圖：先固定可走方向

- 迷宮格子是頂點，相鄰可通行格之間有邊。此例允許上下左右四向移動，不允許斜走。
- 搜尋狀態至少包含所在格、已訪問資訊，以及重建路徑所需的 predecessor。
- DFS 可用堆疊，找到一條路徑但通常不保證步數最少。
- 若所有移動成本都是 1，BFS 使用佇列，首次到達出口的層數就是最短步數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一張迷宮，對照路徑與距離



座標=(row,column)；鄰居次序：右、下、左、上

座標為 `(row,column)`，左上為 (0,0)，列與欄都從0起算；只允許右、下、左、上，移動成本均為 1。深色為牆，折線為 S 到 G 的5邊路徑。

此圖恰好只有一條 S–G 簡單路徑，所以 DFS 與 BFS 在本例得到相同長度；**不能拿本例證明 DFS 一般也最短**。後面的五點反例會分開比較兩者。

DFS 框架：保存回來後要繼續的位置

若只把目前格放堆疊而不記下一方向，回溯後可能重做鄰居掃描。框架 `(u,next_direction)` 保存遞迴的程式位置，visited 在首次推入時設定。

```
visited[start]=true; parent[start]=none; push((start,0))
while stack 非空:
    (u,i)=top()
    if u==goal: return parent path
    if i==4: pop(); continue
    top().next_direction=i+1
    v=neighbor(u,i)                  // 右、下、左、上
    if v 在界內且非牆且 not visited[v]:
        visited[v]=true; parent[v]=u; push((v,0))
return unreachable
```

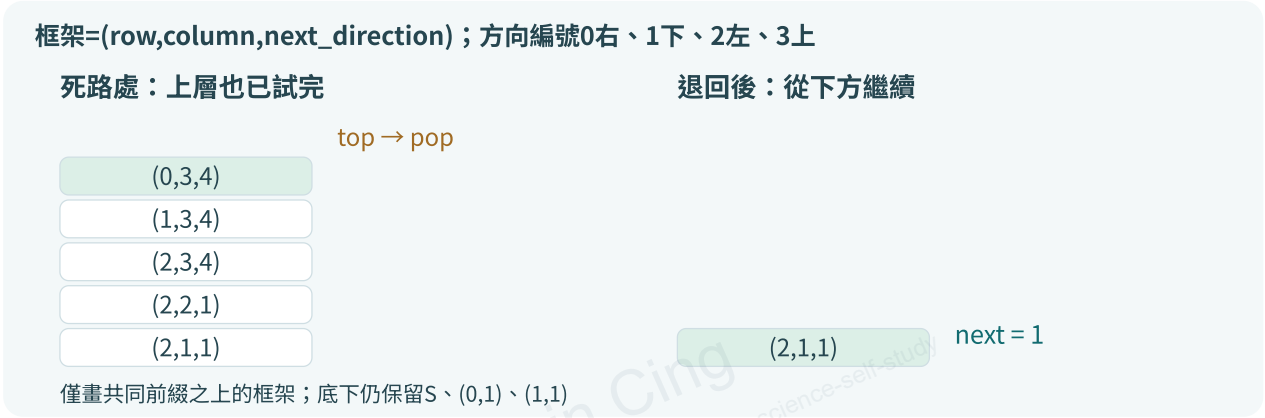
https://github.com/xinhi

DFS 終止與 parent 路徑的理由

本章格圖在首次走到 `(2,1)` 時，右鄰 `(2,2)` 會先被推入，再往右找到 `(2,3)=G`；路徑共5邊。若不在首次抵達 G 時提早停止，完整 DFS 發現次序為 `(0,0),(0,1),(1,1),(2,1),(2,2),(2,3),(1,3),(0,3),(2,0)`。每一框架最多嘗試四方向；有限格數且只首次推入，保證終止。parent 邊沿發現時間嚴格回退，因此不成環並能還原合法路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

回溯不是重來：只彈出走完的框架



為展示真正的 pop，本頁暫不在 G 停止，繼續走到右上死路 (0,3)。它的四方向都處理完後，彈出 `(0,3,4)`；下一個 top 是 `(1,3,4)`，也已無方向可試。

再回到 `(2,3,4)` 後繼續退棧，最後恢復 `(2,1,1)`。這裡的1表示**右方已試過，下一個從下方開始**，接著才會發現左方 (2,0)。visited 不會因退棧而清除，因為本演算法只要求可達性，不是在列舉所有簡單路徑。

BFS：發現時就設定距離與 parent

每個格子最多入列一次。先標記再 enqueue，可避免兩個同層前驅把同一格重複排入。

```
dist[start]=0; parent[start]=none; enqueue(start)
while queue 非空:
    u=dequeue()
    for v in 四向合法鄰居(u):
        if v 尚未發現:
            dist[v]=dist[u]+1; parent[v]=u; enqueue(v)
```

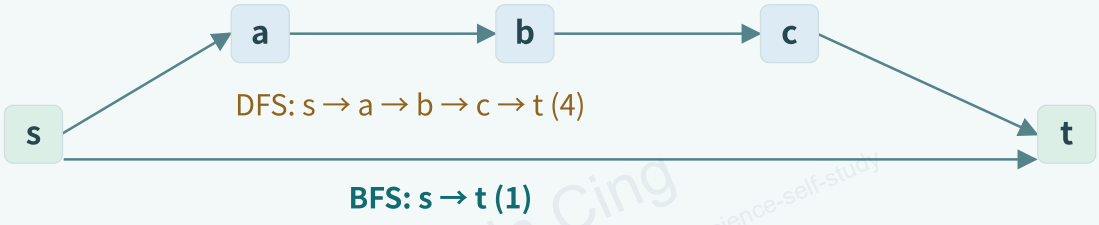
把 dist 的「未定義」當作尚未發現標記。parent 在首次發現時固定；完整距離圖見前頁。若只要找 G，可在首次發現 G 後還原路徑，不必完成整張圖。

BFS 逐次出列：同層先處理，才輪到下一層

出列格	新發現格（距離）	出列後佇列
(0,0)	(0,1):1	[(0,1)]
(0,1)	(1,1):2	[(1,1)]
(1,1)	(2,1):3	[(2,1)]
(2,1)	(2,2):4、(2,0):4	[(2,2),(2,0)]
(2,2)	(2,3):5	[(2,0),(2,3)]
(2,0)	無	[(2,3)]
(2,3)	(1,3):6	[(1,3)]
(1,3)	(0,3):7	[(0,3)]
(0,3)	無	[]

從 parent 還原，再證明最短性

G 的 parent 逆追 (2,2)、(2,1)、(1,1)、(0,1)、(0,0)，反轉得5邊路徑。當距離 d 的格被出列時，佇列只可能還有距離 d 或 d+1 的格；新發現格的距離為 d+1。若存在更短路，其前驅所在的更小層早已被處理，與「尚未發現」矛盾。



固定s的鄰居次序為a、t；箭頭是有向邊，每條成本1

反例另固定 s 的鄰居順序 a、t；DFS 先走長路4邊，BFS 在處理 s 時便發現1邊的 t。

可達性、最短性與成本，要各自證明

- 不變量為每個被探索的格都從起點可達，predecessor 邊連回較早發現的格。
- DFS 若探索所有合法鄰居且沒有未處理的堆疊框架，就已涵蓋所有可達格；出口未被發現則不可達。
- 有 `V` 個可通行格、`E` 條相鄰邊時，鄰接串列模型的 DFS/BFS 為 `O(V+E)`，標記與 predecessor 空間 `O(V)`。
- 四鄰格子中 `E=O(V)`，所以是線性成本。若每步複製整條路徑，實作可能超出此界。

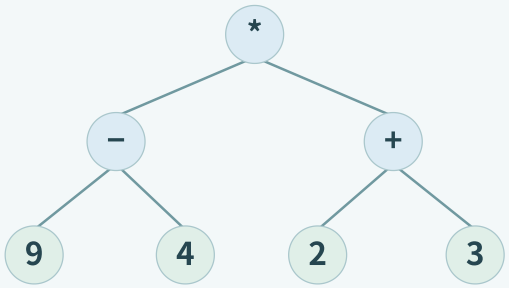
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5　運算樹決定左右，寫法決定掃描順序

- 中序 `a+b*c`、前序 `+ a * b c`、後序 `a b c * +` 代表相同的運算樹。
- 本章先限定二元運算子、整數 token、括號與明訂優先序。負數 token 與一元負號是不同的語法設計。
- 中序需要優先序與結合性。`a-b-c` 預設左結合，代表 `(a-b)-c`。
- 空白不能可靠地切分所有實際程式語言。完整編譯器需先詞法分析，再處理語法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一棵樹，三種讀法



後序求值：先pop右子式值，再pop左子式值

前序：根 → 左 → 右

* − 9 4 + 2 3

後序：左 → 右 → 根

9 4 − 2 3 + *

(9−4) × (2+3) = 5 × 5 = 25

根 * 的左子式是 9−4，右子式是 2+3。前序先根再左、右；後序先左、右再根。中序若要完整保留這棵樹，必須寫成 (9−4)*(2+3)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後序求值：先 pop 的是右運算元

1. 從左到右掃描 token。數值直接 push。
2. 遇到二元運算子，先 pop 右運算元 `b`，再 pop 左運算元 `a`，計算 `a op b` 後 push。
3. 若不足兩個值即拒絕。掃描完成後堆疊必須恰好剩一個值。

原創例子 `8 3 2 * -` 的堆疊依序為 `[8]`、`[8,3]`、`[8,3,2]`、`[8,6]`、`[2]`。除法還需檢查零除與算術範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前序反向掃描，左右角色也要跟著看

- 前序求值可由右到左掃描。遇到運算子時，第一次 pop 是左運算元，第二次是右運算元。
- 原例 `- 8 * 3 2` 由右向左算出 `3*2=6`，最後 `8-6=2`。
- 每個 token 至多 push、pop 固定次數，所以中序轉後序及前後序求值皆為 `O(t)`，`t` 為 token 數。
- 輔助堆疊最壞 `O(t)`。這裡把固定範圍算術視為常數時間，任意精度數值另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把「何時彈出」寫成一個判斷

本節只含非負整數 token、二元 `+ - * / ^` 與括號。`^` 表示**右結合冪次**，不是 C++ 的 XOR；其餘左結合。負號不是一元負號，負數 token 也不在本節輸入文法內。

類別	precedence	associativity
+、−	1	left
*、/	2	left
^	3	right

```
must_pop(top, incoming):
    return top != '(' and
       (prec(top) > prec(incoming) or
        (prec(top) == prec(incoming) and incoming is left-associative))
```

比較的是 incoming 的結合性。 遇括號另作配對處理，不把括號當普通二元運算子。

中序轉後序：逐 token 維護兩個容器

output 保存已確定的後序片段；ops 保存待決運算子。need_operand 防止兩個數值相鄰、空括號與缺少右運算元。

```
output = []; ops = []; need_operand = true
for token in tokens:
    if token 是數值:
        require need_operand; output.append(token); need_operand=false
    else if token == '(':
        require need_operand; ops.push('(')
    else if token == ')': close_group()
    else if token 是二元運算子 op:
        require not need_operand
        while ops is not empty and must_pop(ops.top,op):
            output.append(ops.pop())
        ops.push(op); need_operand=true
    else: reject
```


右括號：只清空當前括號內的待決運算

```
close_group():
    require not need_operand
    while ops 非空且 ops.top != '(': output.append(ops.pop())
    require ops 非空; ops.pop()      // 丟棄配對左括號
    need_operand=false
```

這個 helper 共用 output、ops、need_operand。成功後 need_operand=false；空括號、缺少右運算元或無左括號可配時，都拒絕。括號本身不會輸出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

掃描完，不代表轉換已完成

接續上一頁的同一個 output、ops 與 need_operand：

```
require not need_operand                  // EOF，拒絕空式或缺最後運算元
while ops 非空:
    require ops.top != '('                // 未配對左括號，拒絕
    output.append(ops.pop())
return output
```

每個 require 失敗都拒絕整個輸入。即使已產生部分 output，也不能把它當成轉換成功的結果。while 不能改成 if：一個新運算子可能要求連續彈出多個舊運算子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小例子：9−4 也需要 EOF

9-4

讀入 token	運算子堆疊（底→頂）	已輸出
9	∅	9
-	-	9
4	-	9 4
EOF	∅	9 4 -

EOF 前只有 9 4，不是完整後序式。EOF 彈出 − 得 9 4 −，值為5；前序為 − 9 4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

括號：先完成內部子式

`(9-4)*(2+3)`

讀入 token	運算子堆疊（底→頂）	已輸出
(	(	∅
9	(	9
-	(-	9
4	(-	9 4
)	∅	9 4 -

右括號使第一個子式 9−4 完成，輸出 9 4 −；這時 ops 為空，接著讀乘號與第二個左括號。

接續第二個括號，最後清空乘號

`(9-4)*(2+3)`

讀入 token	運算子堆疊（底→頂）	已輸出
*	*	9 4 -
(	* (	9 4 -
2	* (	9 4 - 2
+	* (+	9 4 - 2
3	* (+	9 4 - 2 3
)	*	9 4 - 2 3 +
EOF	∅	9 4 - 2 3 + *

後序求值堆疊為 `[9]→[9,4]→[5]→[5,2]→[5,2,3]→[5,5]→[25]`。前序 `* - 9 4 + 2 3` 也算得25。

連續彈出與右結合，分別追蹤

`9-4*2/1+3`

讀入 token	運算子堆疊（底→頂）	已輸出
9	∅	9
-	-	9
4	-	9 4
*	- *	9 4
2	- *	9 4 2
/	- /	9 4 2 *
1	- /	9 4 2 * 1
+	+	9 4 2 * 1 / -
3	+	9 4 2 * 1 / - 3
EOF	∅	9 4 2 * 1 / - 3 +

同級要不要彈出？看新運算子的結合性

讀 `+` 時先彈 `/`，再彈同級左結合 `-`；只彈一次會錯。正確值 `9-(4×2)/1+3=4`。右結合 `2^3^2` 則在第二個 `^` 不彈同級，後序為 `2 3 2 ^ ^`，值512，不是64。

右結合：2^(3^2)=2^9=512；左結合：(2^3)^2=8^2=64。這是兩棵不同的運算樹，不是求值順序可以任意替換。

拒絕非法語法，也檢查算術

輸入	失敗時刻	拒絕理由
`(9-4`	EOF	彈 – 後仍有 (，左括號未配對
`9-4)`	讀)	彈 – 後無 (可配
`9-`	EOF	need_operand 仍為真
`9 4`	讀第二個數值	缺二元運算子
`()`	讀)	括號內缺運算元

不可用堆疊值數量猜一元或二元語法。求值另拒絕除零與固定整數溢位。

不變量與線性時間，為何成立？

把尚未配對的左括號視為堆疊分段邊界。每一分段內，已堆疊運算子由底到頂的優先序非遞減；相等優先序只有等待右結合運算時可同時保留。輸出是已確定子運算式的後序排列；堆疊運算子尚在等待右運算元或更高優先子式。

讀入新 op 時反覆 pop，恰好把語法上應先完成的運算子輸出。右括號把該括號內全部待辦運算輸出，保留外層的運算子；EOF 則不再有未讀運算元，必須將全部待辦輸出。若還有左括號，文法就沒有合法配對，不能當運算子輸出。

每個運算子至多推入一次、彈出一次，括號亦如此。雖然某一 token 觸發多次 pop，整段 t 個 token 仍 O(t) 時間、O(t) 空間。這是跨 token 的總量論證，不是宣稱每次迴圈皆最壞常數。剖析負整數 token、一元運算、函式呼叫與浮點特殊值，需擴充文法及對應正確性，不在此二元版默認支援。

括號配對：堆疊記住尚未關閉的種類

輸入只含 `()[]{}`。掃描左括號時 push；掃描右括號時，若堆疊為空，或 top 不是相應的左括號，立即拒絕；否則 pop。掃描結束時，只有堆疊為空才接受。若允許其他字元，必須另訂是否忽略或拒絕，不能默默假設。

括號配對：右括號必須匹配堆疊頂端

輸入：([)]

掃描到的位置	(	[	)
堆疊（底→頂）	(	([	([

讀到) 時，top 是 [，期待的是]。

立即拒絕；左右括號數量相同仍可能巢狀錯誤。

若輸入為 ([])，掃描後堆疊為空，才接受。

粗框標出失配；不依賴顏色區分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼只比較數量不夠？

`([)]` 中各種左、右括號數量相同，但讀到第三個字元 `)` 時，堆疊自底至頂為 `(`、`[`，頂端期待 `]`，因此拒絕。相對地，`([])` 的狀態依序為 `[ ( ]`、`[ (, [ ]`、`[ ( ]`、`[]`，最後接受。

不變量：掃描過的前綴若尚未拒絕，堆疊由底至頂保存所有未配對左括號，且頂端是下一個必須先關閉的括號。每個符號最多 push 或 pop 一次，時間 $O(n)$、額外空間 $O(n)$；空字串在此契約下接受。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

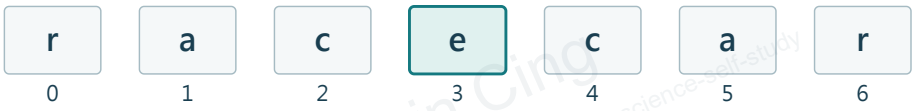
已知長度的迴文：先反轉前半段

輸入是長度已知的符號序列 S。先把前 $\lfloor n/2 \rfloor$ 個符號依序 push；若 n 為奇數，跳過正中間的一個符號；再由左至右掃描後半段，每次與 top 比較並 pop。遇到不等立即拒絕，全部相等才接受。

以 racecar 為例：先 push r,a,c，跳過 e，再依序把 c,a,r 與堆疊頂端比較。空序列和單一符號都接受。比較單位必須先指定：byte、Unicode 碼點與使用者感知的字形簇可能有不同結果。

迴文：前半入堆疊，後半逐一比較

racecar：長度 7，跳過中央 e



前半 push r,a,c；堆疊底→頂為 [r,a,c]。

跳過 e；依序比較 c、a、r，三次都相同。

每次比較後 pop；掃描完成且全相同，接受。

空序列與單一符號也接受；已知長度是此方法的前提。

迴文正確性與成本

推入前半段後，top 是中央左邊第一個符號；第 k 次 pop 比較的是相對中央對稱的兩個位置。每一對都相等，恰好等價於序列與其反轉相同。時間 $O(n)$，堆疊最多 $\lfloor n/2 \rfloor$ 個符號。

這個版本事先知道長度並能掃描相應位置。若是單向串流且事前不知道長度，不能在讀到中間前憑空知道何時停止 push；需要不同的輸入假設或演算法。

判定 $a^n b^n$：數量相同，順序也要對

定義語言 $L = \{a^n b^n \mid n \geq 1\}$，輸入字母表只含 a,b。先讀一個以上的 a，每個 push 一個標記；讀到第一個 b 後進入第二階段，每個 b pop 一個標記。第二階段再看到 a、遇到其他符號或空堆疊時讀到 b 都拒絕。結束時必須見過 a 與 b，且堆疊為空。

aabb 接受；aab 留下一個標記而拒絕；abb 第二個 b 無標記可 pop；abab 在第二階段又讀到 a 而拒絕。空字串不屬於此處的 L；若改定義 $n \geq 0$，才應接受空字串。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6　堆疊排列：不是所有輸出次序都可行

輸入固定依序為 1、2、3，只能把下一個輸入 push，或把當前 top pop 到輸出；禁止直接越過堆疊。

目標	必要操作	結果
2,1,3	push1、push2、pop2、pop1、push3、pop3	可行；最大深度2
3,1,2	為先輸出3，必須 push1、push2、push3、pop3	不可行：top=2 擋住下一個1

一般判定：依目標值 x，若 top≠x，就持續推入尚未讀取的輸入；若輸入耗盡仍 top≠x，立即拒絕，否則 pop x。**正確性來自操作被迫性**：top 不是 x 時不能先輸出其他值；若尚可產生 x，唯一前進方式是 push 下一個輸入。每個值至多進出一次，時間 O(n)、空間 O(n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何 n=3 有5種，而不是6種？

令 push 為 U、pop 為 D。合法操作有 n 個 U、n 個 D；每一前綴皆滿足 U 數≥D 數，最後兩者相等。這正是合法括號序列（U 對應左括號）。

把第一次 push 與讓它離棧的 pop 配成 `U A D B`。A 內若有 k 次 push，B 就有 n−1−k 次；兩者各自合法，分解也唯一。因此

$$C_0 = 1, \qquad C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}.$$

依序算得 C_1=1、C_2=2、C_3=2+1+2=5、C_4=5+2+2+5=14。固定互異輸入時，輸出唯一決定操作序列（前頁的被迫性），因此也在計算不同的可行輸出，而不只是數操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Catalan 公式：扣掉會對空堆疊 pop 的路徑

不限制前綴時，共有 $\binom{2n}{n}$ 種 U/D 排列。對任何非法路徑，把「第一次下降到−1」以前的 U/D 全互換；原前綴多一個 D，交換後整條路徑共有 n+1 個 U、n−1 個 D。

反向則取這類路徑第一次升到+1的前綴再交換，恰好還原一條非法路徑，所以是一一對應。非法路徑數為 $\binom{2n}{n-1}$。因此

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}.$$

n=0 另有一個空操作序列。互異值、固定入列順序與無旁路，是這裡把 Catalan 數用於堆疊輸出的前提；重複值的不同輸出序列數不能直接套用。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 stack、queue；MIT 6.006，Lecture Notes 9–10。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

前置知識與本章目標

- 前置知識為指標、物件生命週期、Stack/Queue ADT 與第 1 章不變量。
- 能區分節點位置、節點值與節點所有權，並說明哪個操作真的需要搜尋。
- 能實作與追蹤單向、環狀、雙向串列，以及鏈結堆疊、佇列與多項式相加。

單向串列的合法狀態

- 節點包含值與 `next`，`head` 指向第一個節點。空串列使用 null。
- 不變量為從 `head` 沿 `next` 形成有限無環鏈，最後指向 null，每個節點由唯一容器負責釋放。
- 若保留 `tail`，非空時它必須指向最後節點且 `tail.next=null`；空時 `head`、`tail` 同為 null。
- 外部指標可作非擁有參照，但被指向節點刪除後即失效。有效位址不等於有效物件生命週期。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

已知前驅時的插入

在 `p` 後插入新節點 `x`。

1. 配置並建構 `x`，令 `x.next=p.next`。
2. 令 `p.next=x`。若 `p` 原先是尾節點，更新 `tail=x`。
3. 更新元素數。若建構失敗，原串列仍保留。

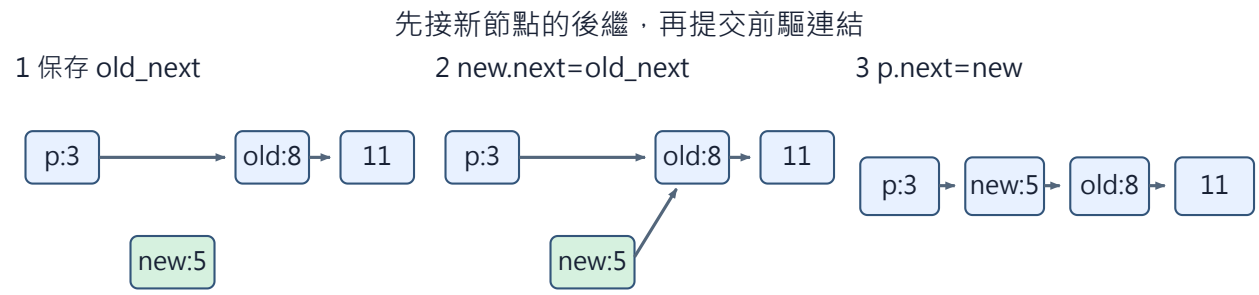
此操作在已知合法前驅、配置成本視為常數時為 `O(1)`。若先用索引找第 `i` 個前驅，搜尋成本為 `O(i)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單向串列插刪的完整所有權轉移

```
insert_after(p,value):
    require p is a live node of this container
    old_next=p.next
    x=allocate_and_construct(value)
    x.next=old_next
    p.next=x
    if tail==p: tail=x
    size+=1
erase_after(p):
    require p is live and p.next!=null
    victim=p.next; after=victim.next
    p.next=after
    if tail==victim: tail=p
    destroy_and_release(victim); size-=1
```


單向串列插刪的完整所有權轉移 （續）



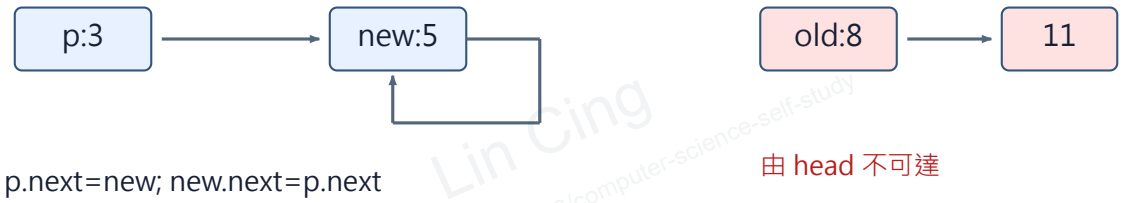
對p值3、後繼8→11插入5。配置後x未從head可達；設定x.next=8後，舊鏈仍3→8→11且新節點也能到8；最後p.next=x提交為3→5→8→11。若倒過來先p.next=x再x.next=p.next，會得到x.next=x自環，8與11失去從head的路徑。這是明確可達性破壞，不只結果值排列錯誤。

頭插／頭刪需獨立處理head，或用不含資料的dummy前驅統一操作。若dummy成為最後剩餘節點，對外空狀態應head=null,tail=null，不把dummy當使用者資料。已知前驅的連結更新O(1)；外部任意pointer不滿足此契約，依位置搜尋前驅仍O(n)。

錯序插入的可達性反例

```
old_next=p.next
p.next=x              // wrong first update
x.next=p.next         // reads x, not old_next
```

錯序造成 new 自環，失去舊後綴



p.next=new; new.next=p.next

由 head 不可達

原head=p且鏈3→8→11。錯序執行後由head可走3→5→5→…；原8與11仍配置在記憶體卻失去容器的擁有路徑，形成遺漏與可能洩漏。即使某個外部變數仍持有8的位址，也不代表容器表示有效。正確順序是先以已保存old_next接x，再提交p.next=x。若多執行緒可同時觀察，這兩個寫入也不自動成為原子交易；本章契約限定單執行緒結構更新。

刪除必須先保存下一個位置

- 刪除 `p` 後的節點時，先保存 `victim=p.next` 與 `after=victim.next`，再令 `p.next=after`，最後釋放 victim。
- 若 victim 是尾節點，更新 `tail=p`。刪除唯一節點則須同時使 head、tail 為空。
- 解參照 victim 必須發生在釋放前。釋放後再讀取 `victim.next` 是 use-after-free。
- 只有待刪節點指標、沒有前驅時，一般無法在單向串列 `O(1)` 刪除尾節點。「複製下一節點的值」也會改變節點身分語義。

頭端、佇列端點與迭代清除

```
push_front(v):
    x=Node(v,next=head); head=x
    if tail==null: tail=x
    size+=1
pop_front():
    require head!=null
    x=head; result=copy(x.value); head=x.next
    if head==null: tail=null
    release(x); size-=1; return result
enqueue(v):
    x=Node(v,next=null)
    if tail==null: head=x
    else: tail.next=x
    tail=x; size+=1
clear():
    while head!=null:
        x=head; head=x.next; release(x)
    tail=null; size=0
```


頭端、佇列端點與迭代清除 （續）

這四個操作共用同一個擁有節點的單向表示，鏈結堆疊以 push_front/pop_front 操作，鏈結佇列以 enqueue/pop_front 操作。前提為單執行緒、節點配置與值複製成功；若複製 result 會拋出例外，必須在修改連結前完成，失敗時原串列保留。每次釋放前先保存下一個節點，不能讀取已釋放的 x.next。

從空狀態 (head,tail,size)=(null,null,0) 入列4，兩端同指4且 size=1；入列9得4→9、tail=9、size=2；出列4得 head=tail=9、size=1；再出列9得到兩端 null、size=0；再入列7可安全建立新鏈。若只清 head 不清 tail，最後一步將沿已釋放尾節點寫入。單點與空鏈是表示契約的一部分。

歸納檢查：新節點只由原尾或 head 連入一次，無環且末端 next=null；刪頭只移除第一點，剩餘次序與所有權不變；size 與可達節點數同步。每項端點操作在固定大小值和常數配置模型下最壞 O(1)。clear 每輪恰釋放一個可達節點，時間 Θ(n)、額外堆疊 O(1)，可避免逐節點遞迴析構的線性呼叫深度。

三指標反轉的不變量

初始 `prev=null,curr=head`，重複保存 `next=curr.next`、令 `curr.next=prev`，再前移 `prev,curr`。

- 不變量為 `prev` 指向已反轉前綴，`curr` 指向尚未處理後綴，兩者節點集合互斥且聯集為原串列。
- 原串列 `3→8→11` 經一輪成 `3→null`，未處理部分為 `8→11`。
- 終止時 `curr=null`，令 `head=prev`。若維護 tail，舊 head 成為新 tail。
- 每個節點處理一次，時間 `O(n)`、輔助指標空間 `O(1)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

反轉的完整程序與節點分割不變量

```
reverse(head):
    old_head=head; prev=null; curr=head
    while curr!=null:
        after=curr.next; curr.next=prev
        prev=curr; curr=after
    head=prev; tail=old_head
```

反轉3→8→11的(prev,curr)依序為(空,3→8→11)、(3,8→11)、(8→3,11)、(11→8→3,空)。兩段互斥且覆蓋原節點，是保留全部資料的證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐步反轉鏈結串列

單向串列反轉：兩段互斥，合起來保有所有節點

原始：3 → 8 → 11 → null

輪次	prev：已反轉前綴	curr：未處理後綴
0	null	3 → 8 → 11 → null
1	3 → null	8 → 11 → null
2	8 → 3 → null	11 → null
3	11 → 8 → 3 → null	null

每輪先保存 curr.next，再反轉連結；完成後 head=prev，tail=原 head。

空鏈和單節點也適用；圖中的數字表示節點身分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C++ 所有權與複製語義

- 用 `unique_ptr` 表示單一所有權，可以讓節點釋放責任隨所有權轉移。非擁有 tail 可用一般指標但必須維持有效性。
- 含 owning raw pointer 的類別若採預設淺複製，兩個容器可能重複釋放同一節點。必須定義深複製或明確禁用複製。
- `shared_ptr` 不是所有鏈結結構的預設答案。雙向或環狀 owning links 會形成參照計數循環。
- 長鏈的遞迴析構可能造成很深的呼叫堆疊；授課實作可採迭代清除，並說明生命週期處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

鏈結堆疊與佇列

結構	指標安排	更新成本
堆疊	head 為頂端	head 插入、刪除各 O(1)
佇列	head 出列、tail 入列	保留 tail 時各 O(1)

- 鏈結佇列從空狀態 enqueue 時，head、tail 都指向新節點。
- dequeue 最後一個節點時，必須把 tail 也清成 null，否則下一次入列可能沿懸空指標寫入。
- 操作成本不包含任意大小值的複製。配置器的真實延遲也不由 ADT 保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多項式的正規表示

- 每個節點存 `(exponent,coefficient)`，指數為非負整數，按指數嚴格遞減，係數非零。
- $P=4x^5-2x^2+7$ 表示為 `(5,4)→(2,-2)→(0,7)`。
- 同指數多次出現屬於未正規化輸入。需合併係數，係數抵消成零時移除節點。
- 零多項式用空串列表示，不把 `(0,0)` 與空串列混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多項式相加是有序合併

- 令 $Q=-4x^5+3x^3+2x^2$。兩游標從最高指數開始，比較目前指數。
- 指數較大的一項直接輸出並前移。指數相同則相加，僅在係數非零時輸出，再同時前移。
- 本例五次項與二次項抵消，得到 $P+Q=3x^3+7$。
- 不變量為輸出已正規化，且等於兩輸入已消耗部分的和。每個節點最多前進一次，時間 $\Theta(p+q)$，輸出空間 $O(p+q)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多項式合併的逐項消耗

```
add_polynomials(p,q):
    require both normalized, descending exponents
    out=[]
    while p!=end and q!=end:
        if p.exp>q.exp: append_copy(out,p); advance(p)
        else if q.exp>p.exp: append_copy(out,q); advance(q)
        else:
            c=p.coeff+q.coeff
            if c!=0: append(out,(p.exp,c))
            advance(p); advance(q)
    append_copy(out,remaining(p)); append_copy(out,remaining(q))
    return out
```

同一$P=4x^5-2x^2+7$與$Q=-4x^5+3x^3+2x^2$：先比較5與5，係數$4+(-4)=0$不輸出；再比較2與3，輸出$3x^3$；再比較2與2，$-2+2=0$不輸出；Q結束後輸出剩餘常數7，結果$3x^3+7$。游標位置每次至少前進一個，輸出嚴格遞減且無零係數。

若係數採固定整數，必須指定溢位如何處理；若採浮點數，近零合併的容差不是精確代數等於零，會改變抽象值。程式應先選精確整數、有理數或明確的近似模型，不能以浮點容差悄悄宣稱代數恆等。

環狀串列的終止條件

- 非空環狀單向串列滿足 `tail.next=head`，不以 null 終止。走訪應在回到起點時停止。
- 保留 tail 時，head 為 `tail.next`。在頭尾插入均可改動固定數目的 links。
- 原例 `A→B→C→A`，從 B 開始走訪一圈應得到 B、C、A，而不是永遠等待 null。
- 若採 sentinel，空串列可用 sentinel 指回自身。sentinel 是結構節點，不屬於使用者資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀串列的尾插與端點成本

```
circular_push_back(value):
    x=new_node(value)
    if tail==null: x.next=x; tail=x
    else: x.next=tail.next; tail.next=x; tail=x
```

環狀空鏈加入A後A.next=A；再加入B後tail=B、B.next=A、A.next=B。走訪用do/while在回到head時停止，空鏈必先檢查tail=null。只保留tail時刪頭O(1)，刪尾須找前驅O(n)，不能因頭尾插入快就宣稱全部端點操作都快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀佇列：唯一節點出列要清空 tail

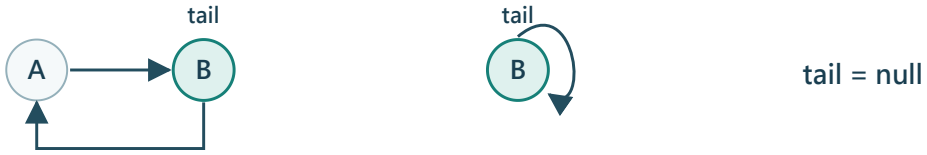
只保留 `tail`，非空時首節點為 `tail.next`。若 `tail.next==tail`，代表只有一個節點；出列並釋放它之前，必須把 `tail` 設為 null。一般情況則把 `tail.next` 改為舊首節點的後繼，再釋放舊首節點。

```
dequeue():
    if tail == null: return empty
    victim = tail.next
    value = copy(victim.value)
    if victim == tail: tail = null
    else: tail.next = victim.next
    destroy_and_release(victim)
    return value
```

例：A→B→C→A 且 tail=C，出列後為 B→C→B，tail 仍是 C；若原來只有 A→A，出列後 tail 必須為 null。假設值複製、釋放與回傳均成功；泛型值若可能拋例外，需另訂完整例外契約。

環狀佇列：最後節點出列

環狀佇列：刪除最後節點的前後



① 原狀 A→B→A　② 出列 A：B→B　③ 出列 B：空

victim=tail.next；只有一個節點時，先將 tail 設為 null，再釋放 victim。

之後入列 C，會建立 C→C；不可使用已釋放的 B。

粗框標示尾節點；空狀態沒有有效的 tail。

Floyd 快慢指標：找到環或證明走到底

兩指標都從 head 開始。每輪先確認 fast 與 fast.next 皆非 null，再令 slow 走一步、fast 走兩步；若相遇，結構有環。若檢查失敗，已到達無環鏈的末端。此處假設節點在演算法執行期間保持有效，沒有並行修改。

```
slow=head; fast=head
while fast != null and fast.next != null:
    slow=slow.next; fast=fast.next.next
    if slow==fast: return meeting
return no_cycle
```

空串列、單節點末端為 null 都回報無環；單節點指向自己則第一輪相遇。演算法不改動節點連結。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼再次相遇恰在環入口？

設 head 到環入口有 μ 條邊、環長 $\lambda \geq 1$。第一次相遇發生在環內；設 slow 已走 t 步，fast 走 $2t$ 步，兩者在環上的位置相同，因此 $t \equiv 0 \pmod{\lambda}$。

令一個指標回 head，另一個留在相遇點，兩者每輪各走一步。走 μ 步後，前者抵達入口；後者的位置是 $(t - \mu + \mu) \bmod \lambda = 0$，也在入口。在這之前，從 head 出發的指標尚未進環，不可能與環內指標相遇。因此第二次相遇恰是入口。

從 head 到首次相遇、再到入口，時間 $O(\mu + \lambda)$，額外指標空間 $O(1)$。無環長度為 n 時，時間 $O(n)$；兩種情況都不依賴節點值是否重複。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

數值追蹤：前綴長 2、環長 3

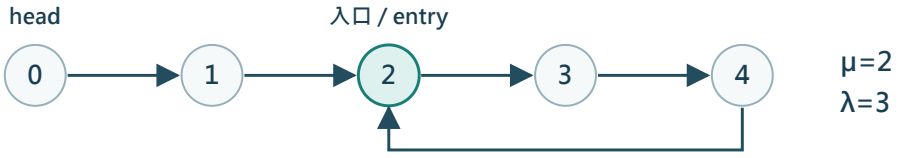
指標鏈為 `0→1→2→3→4→2`，所以 $\mu = 2$、$\lambda = 3$。第一次相遇前，各輪 `(slow,fast)` 為 `(1,2)`、`(2,4)`、`(3,3)`；回 head 後，兩指標依序為 `(0,3)`、`(1,4)`、`(2,2)`，入口是 2。

不可只因「兩個節點的值相同」就判為相遇：比較的是節點身分。找環入口與刪除環是不同操作；本演算法只回傳入口，不修改 `next`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快慢指標的兩次相遇

Floyd：第一次相遇，再尋找環入口



第一階段：每輪 slow 一步，fast 兩步

(1,2) → (2,4) → (3,3)

重設一個指標到 head；兩者都每輪一步

(0,3) → (1,4) → (2,2)

比較節點身分；兩次相遇的節點不一定相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向串列的局部一致性

- 每個節點有 prev、next，鄰接節點須滿足 `x.next.prev=x` 及 `x.prev.next=x`，端點按 sentinel 策略處理。
- 在相鄰節點 p、q 間插入 x，設定 `x.prev=p,x.next=q,p.next=x,q.prev=x`。
- 刪除已知節點 x，讓 `x.prev.next=x.next`、`x.next.prev=x.prev`，再釋放 x。
- 若節點已知，更新為 `O(1)`。依值找節點仍是 `O(n)`，不能把「移除已知節點」偷換成「搜尋並移除」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向串列刪除與參照生命週期

```
doubly_erase(x):
    require x is a live owned data node, x!=H and x!=T
    p=x.prev; q=x.next
    p.next=q; q.prev=p
    release(x); size-=1
```

本段固定採兩個不含資料的 sentinel H、T。H.prev=null、T.next=null；空串列為 H.next=T、T.prev=H，size=0。每個資料節點的 prev、next 均非 null，因此以下 p.next 與 q.prev 可安全解參照。若改採無 sentinel 的 null 端點，刪首／尾節點必須另更新 head／tail，不能直接套此程式。

只有一筆資料時 H↔x↔T，刪除取 p=H、q=T，接回 H.next=T、T.prev=H，size 由 1 降 0；H、T 保留且絕不釋放。刪多筆串列的第一或最後資料同樣由鄰接 sentinel 接回。合法節點 handle 與所屬容器的確認在進入指標操作前完成。

雙向鏈p↔x↔q刪除後p.next=q、q.prev=p，故兩個互反條件同時恢復。剩餘節點的外部參照保持有效，x的所有參照失效；不能只改一側，否則反向與正向走訪會看到不同序列。

雙向串列：刪除唯一節點

雙向 sentinel：刪除唯一資料節點

① 保存 p=x.prev=H、q=x.next=T
② 設 p.next=q；設 q.prev=p
③ 釋放 x；size 由 1 變 0

H.next=T
T.prev=H

H、T 保持有效；x 的外部指標不得再解參照。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

串列與陣列的選擇

操作與限制	動態陣列	單向串列
依索引讀取	O(1)	O(n)
已知前驅插入	O(n) 搬移	O(1) links
連續掃描	通常有儲存局部性	需追蹤指標
額外每項空間	容量閒置	next 與配置開銷

只有完整工作負載才能決定取捨。沒有實測時，不能以 links 數直接宣稱實際較快。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 forward_list、list 與 unique_ptr。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

前置知識與目標

- 前置知識為遞迴、鏈結結構、堆疊與歸納證明。
- 能以精確術語描述樹，追蹤走訪、BST 更新與堆積操作，並維護各自的不變量。
- 能說明一般二元樹、二元搜尋樹及二元堆積的差異，而不只依外形判斷。

樹的數學定義與術語

- 無向樹是連通且無環的簡單圖。任意兩點之間恰有一條簡單路徑，含 `n` 點時有 `n-1` 邊。
- 選定 root（根）後，每個非根節點具有唯一 parent（父節點）。children（子節點）是距根多一條邊的鄰點。
- depth（深度）為根到節點的邊數；height（高度）為節點到最深後代葉的邊數。整棵樹高度是根高度。
- leaf（葉）沒有子節點。森林是互不相交的樹集合。空樹的高度約定為 −1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二元樹左右位置有語義

- 二元樹每個節點有至多一個左子節點及一個右子節點。只有左子與只有右子是不同結構。
- 高度 `h≥0` 的二元樹最多有 `2^{h+1}-1` 點，因第 `d` 層最多有 `2^d` 點。
- full binary tree 在本教材指每個內部節點恰有兩子。complete 指最後一層以外填滿，最後一層由左連續填入。
- perfect 指各葉同深且內部節點都有兩子。這些名稱在不同教材可能有變體，必須先固定定義。

有些教材將 full 用於所有層都填滿的 perfect tree，並另稱每個內部節點恰有兩子的樹為 strict。閱讀時以定義而非名稱判別。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

葉節點與分支數：先數邊，再數節點

非空二元樹以 n_0、n_1、n_2 分別表示 0、1、2 個子節點的節點數。每個非根節點恰對應一條父子邊，因此邊數既是 n−1，也是 n_1+2n_2。

代入 $n = n_0 + n_1 + n_2$，得到 $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$，消去 n_1 即得 **$n_0=n_2+1$**。這個等式不要求樹為完全樹或每個內部節點都有兩個孩子；空樹不符合此結論。

例：55 個節點、其中 22 個只有一子。由 $55 = n_0 + 22 + (n_0 - 1)$，得 $n_0=17$、$n_2=16$。對一般根樹，同樣的邊數法給出 $n_0 = 1 + \sum_{d\ge 2}(d-1)n_d$，求和範圍 d≥2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

高度界與完整樹的索引

本章高度以邊數計，根的 depth 為 0。深度 d 最多 2^d 點，歸納步是每個深度 d 節點至多生兩個孩子。加總得 $n \le 2^{h+1} - 1$，故非空樹 $h \ge \lceil \log_2(n+1) \rceil - 1$；鏈狀樹達到最大高度 n−1。

若使用 root level=1、height=層數的慣例，上式的高度數字全都加 1。不要把兩種慣例混在同一次計算。

0 起始完整樹以 i 表示節點時，左子 2i+1、右子 2i+2、父節點 ⌊(i−1)/2⌋；子索引必須小於 n。n=10 時最後有子節點為 i=4，葉索引 5 至 9，共 5 個。1 起始版本的父子公式依序為 ⌊j/2⌋、2j、2j+1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

鏈結與陣列表示

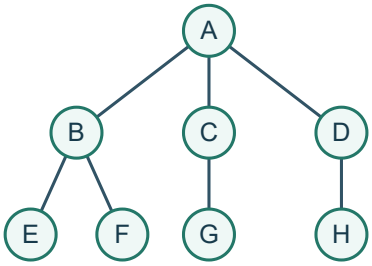
- 鏈結二元樹以 `{key,left,right}` 表示，可自然容納不規則形狀。
- 完全二元樹可放在 0 起始陣列，節點 `i` 的左右子索引是 `2i+1`、`2i+2`，父索引是 `⌊(i-1)/2⌋`，父式只對 `i>0` 使用。
- 高度 `h` 的單鏈若硬套完整位置編號，索引可達指數規模。陣列表示不是所有二元樹都省空間。
- 由索引算子節點時，要在乘法或加法前確保不溢位，並檢查是否小於元素數。

本章堆積採 0 起始索引；演算法課常採 1 起始，父、子公式分別為 ⌊j/2⌋、2j、2j+1。兩者表示同一結構，代入前先確認索引起點。

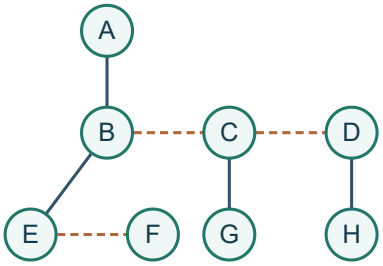
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左子右兄弟：保留有序一般樹

一般樹與左子右兄弟表示



原樹：A 的孩子依序為 B、C、D



實線 left = 第一個孩子

虛線 right = 下一個兄弟

右側的 C 是 B 的兄弟，不是 B 在原樹中的孩子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左子右兄弟的讀法與樹的計數

每個節點的 left 指向第一個孩子，right 指向下一個兄弟；沿第一個孩子的 right 鏈可依序列出原樹所有孩子。若要表示森林，根之間也可依序以 right 相連。這個對應保留的是 **有序** 根樹；忽略原本兄弟順序會失去一對一關係。

令 C_n 是 n 個無標記節點的有序二元樹形狀數，包含 $C_0=1$。根分配 i 個節點給左子樹，其餘 n−1−i 個給右子樹，故 $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$。逐項算得 $C_1=1$、$C_2=2$、$C_3=5$、$C_4=14$。

對固定的 n 個互異且全序的 BST 鍵，每一種二元樹形狀都只有一種符合中序遞增的標鍵方式，所以 BST 的形狀數同為 C_n。若節點標籤可任意排列且視為不同，一般二元樹則有 $n!C_n$ 種；不能混淆這兩個計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Catalan 閉式：用括號路徑計數

將空樹編碼為空字串；非空樹編為 (、左子樹編碼、)、右子樹編碼。匹配根的第一對括號，可唯一切回左右子樹，故 n 節點二元樹形狀與 n 對合法括號一一對應。

將左括號視為上步、右括號視為下步，全部上、下各 n 步有 $\binom{2n}{n}$ 種。對第一次跌到 −1 的路徑，反射到該位置以前的步驟，得到上步 n+1、下步 n−1 的路徑；反射可逆，故不合法路徑有 $\binom{2n}{n-1}$ 種。

合法數為 $C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}$。其中 n=0 得 1，表示空樹也計作一種形狀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種深度優先走訪

- preorder（前序）依「根、左、右」。inorder（中序）依「左、根、右」。postorder（後序）依「左、右、根」。
- 例樹根為 8，左子 3 的左右子為 1、6，右子 12 的左子為 10。
- 前序為 `8,3,1,6,12,10`。中序為 `1,3,6,8,10,12`。後序為 `1,6,3,10,12,8`。
- 對每個子樹按相同規則遞迴，歸納即可證明每個節點恰訪問一次，時間 `O(n)`。

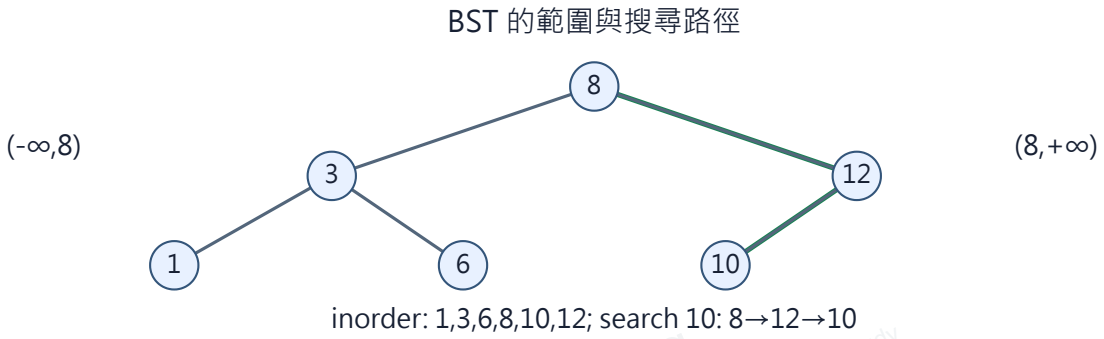
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴走訪與迭代中序的完整狀態

```
visit(x,order):
    if x==null: return
    if order==pre: emit(x)
    visit(x.left,order)
    if order==in: emit(x)
    visit(x.right,order)
    if order==post: emit(x)
inorder_iterative(root):
    stack=[]; x=root
    while x!=null or stack is not empty:
        while x!=null: stack.push(x); x=x.left
        x=stack.pop(); emit(x); x=x.right
```

https://github.com/xinhi6716/com

遞迴走訪與迭代中序的完整狀態 （續）



同一樹的中序堆疊底→頂與輸出：先push8,3,1得[8,3,1]；pop1輸出[1]；pop3輸出[1,3]後push6得[8,6]；pop6輸出[1,3,6]；pop8後進12的左鏈得[12,10]；pop10後pop12，結束為[1,3,6,8,10,12]。堆疊保存待輸出的祖先，每個節點push/pop恰一次，總Θ(n)，輔助O(h+1)。

一般二元樹的中序不保證排序；只有全子樹範圍成立的BST才有此性質。不能用本例的已排序結果替代BST表示不變量。

從前序與中序還原樹

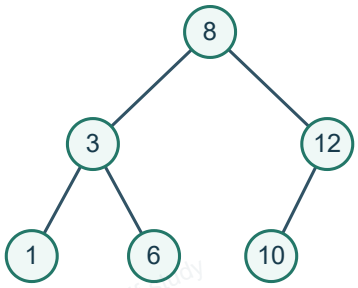
前序與中序如何切出子樹

pre: 8 | 3, 1, 6 | 12, 10
in: 1, 3, 6 | 8 | 10, 12

根 8 將中序切成左 3 個鍵、右 2 個鍵

左：pre 3,1,6；in 1,3,6

右：pre 12,10；in 10,12



重複套用：前序首鍵是根；中序決定左右子樹大小。

鍵必須互異；若輸入不相容，重建程序必須拒絕。

還原樹的正確性與限制

鍵互異且兩序列確實來自同一棵樹時，前序首項唯一決定根；根在中序的左、右段唯一決定兩子樹的鍵集合及大小。據此在前序切出同長區段，再對左右段遞迴。對節點數歸納，根及左右子樹均唯一，整棵樹因此唯一。

實作先建立 key→中序位置的映射，使用區間邊界與一個前序游標。每個鍵處理一次，期望 Θ(n) 時間、Θ(n) 映射空間與 O(h+1) 呼叫堆疊；雜湊表須另註最壞界。若鍵重複、集合不一致，或根的位置落在當前中序區間外，就拒絕輸入。

僅有前序與後序通常不唯一。單鍵 A 的唯一子節點 B 可放左或右，兩者的前序皆 A,B，後序皆 B,A。若另外保證每個內部節點恰有兩子且鍵互異，則可藉前序下一鍵在後序中定位左子樹，遞迴唯一還原。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

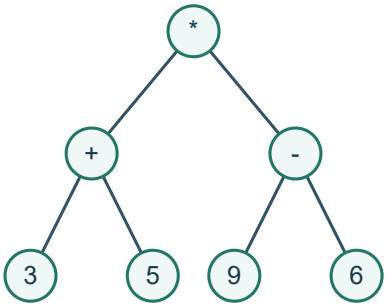
走訪的堆疊與佇列成本

- 非空樹遞迴 DFS 的額外堆疊 `O(h+1)`，以免 h=0 的單點例外被省略。退化鏈的 `h=n-1`，不能一律宣稱 `O(log n)`。
- 中序迭代法先把沿途左鏈 push，pop 一點並輸出後，再處理其右子樹。
- level-order（層序）以佇列逐層處理。原例結果是 `8,3,12,1,6,10`。
- 層序佇列需 `O(w)`，`w` 為最大層寬。平衡樹可能有 `w=Θ(n)`，與 DFS 堆疊不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

運算式樹與走訪順序

運算式樹：走訪順序與計算



中序 inorder： (3 + 5) × (9 − 6)

前序 preorder： × + 3 5 − 9 6

後序 postorder： 3 5 + 9 6 − ×

後序求值：8 × 3 = 24

中序要依運算子的優先順序及結合性加括號，才能保留原樹的語意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

運算式樹如何保留語意

運算子節點的左右孩子分別是左、右運算元。後序會先取得兩個運算元，才執行父運算子；例如後序 `3 5 + 9 6 − ×` 依序計算 8、3、24。二元減法與除法不可交換：先彈出的是右運算元，後彈出的是左運算元。

前序對應 prefix 表示法，後序對應 postfix。中序若直接省略括號，可能改變運算順序：樹 `(3+5)×(9−6)` 若寫成 `3+5×9−6` 就不是相同運算式。輸出中序時，要保留足夠括號，並考量運算子的優先順序及結合性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 不變量與搜尋

- 本章鍵互異。對任一節點 x，左子樹所有鍵 `<x.key`，右子樹所有鍵 `>x.key`。
- 搜尋 k 時，依比較結果丟棄根與其中一個子樹。被丟棄子樹不含 k，理由來自全子樹不變量。
- 上述例樹搜尋 10 的路徑為 `8→12→10`。搜尋 11 最後落到 10 的空右子位置。
- 搜尋、插入與刪除時間均為 `O(h+1)`。沒有平衡機制時，依序插入遞增鍵可使高度線性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 插入與重複鍵政策

- 沿搜尋路徑找到空子位置，插入新葉即可保留所有祖先的鍵範圍限制。
- 在原例插入 11，成為 10 的右子。只需檢查沿路界線 `8<11<12`、`10<11`。
- 若允許重複，不能一邊說嚴格不等式，一邊把相同鍵放任意子樹。可採每鍵計數、複合唯一鍵或明訂非嚴格規則。
- 實務容器若儲存 key-value，更新既有 key 的 value 與新增節點是不同操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 查找、插入與全域範圍驗證

```
find(root,k):
    x=root
    while x!=null:
        if k==x.key: return x
        if k<x.key: x=x.left
        else: x=x.right
    return not_found
insert(x,k,v):
    if x==null: return Node(k,v,left=null,right=null)
    if k<x.key: x.left=insert(x.left,k,v)
    else if k>x.key: x.right=insert(x.right,k,v)
    else: x.value=v
    return x
valid(x,lower,upper):
    if x==null: return true
    return lower<x.key<upper and valid(x.left,lower,x.key)            and
valid(x.right,x.key,upper)
root=insert(root,k,v)
```


BST 查找、插入與全域範圍驗證 （續）

此介面對同鍵更新 value，不配置第二個節點。valid 的上下界是可選的數學界，起始為 −∞、+∞；若機器鍵域已包含極值，實作要用「沒有界」的標記，不能拿最小／最大可表示鍵冒充無窮。

原樹查11的比較為8<11、12>11、10<11，進10.right的空位置而失敗；插11沿相同路徑配置葉，逐層接回10.right、12.left、8.right，外部也接回 root。已知的祖先區間依序為(8,+∞)、(8,12)、(10,12)，故11滿足全部限制。再次插11只更新該節點 value，節點數不變。

反例根8、左子3、3右子9：每條父子邊的局部大小都對，但 valid 在9處帶著(3,8)而拒絕。搜尋正確性同樣使用祖先區間；若 k<x.key，整棵右子樹的鍵都大於 x.key，不能含 k。查找與插入走一條高 h 的路徑，最壞 O(h+1)；遞迴插入另用 O(h+1) 堆疊。全域驗證必看全部 n 個節點，時間 Θ(n)，不是 O(h)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 刪除的三種情況

1. 葉節點直接從父連結移除。
2. 單一子節點以該子樹取代被刪位置。
3. 兩子節點以右子樹最小鍵，即中序後繼，取代邏輯內容，再刪除原後繼位置。

後繼不可能有左子，因此最後只需處理前兩種。若節點身分被外部 handle 使用，搬值與搬節點必須選擇一致語義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩子刪除的原創追蹤

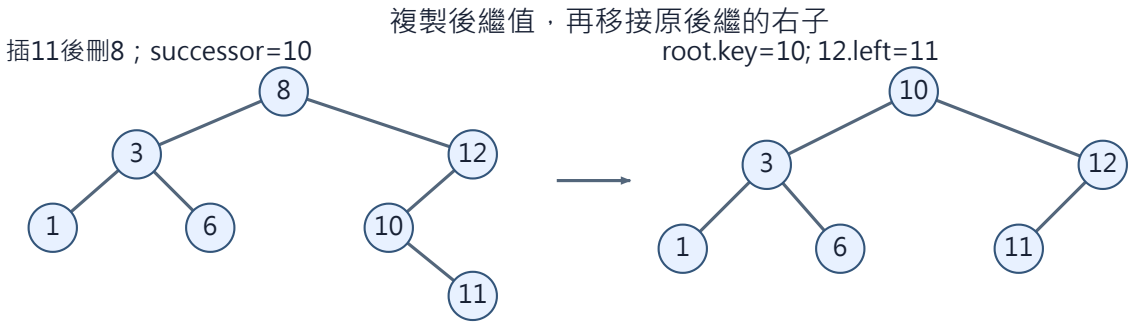
- 從原樹刪除 8，右子樹最小鍵是 10。令根鍵為 10，再刪除 12 的左子 10。
- 新中序為 `1,3,6,10,12`，恰是原有序序列移除 8。
- 後繼大於原根左子樹所有鍵，且不大於右子樹剩餘鍵，因此替換不破壞 BST 次序。
- 只檢查父與子大小並不足以驗證 BST。例如根 8 的左子 3 若有右後代 9，局部邊比較成立，整體仍違規。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 刪除的完整遞迴與身分契約

```
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x.left==null: y=x.right; release(x); return y
        if x.right==null: y=x.left; release(x); return y
        s=x.right
        while s.left!=null: s=s.left
        x.key_and_value=copy(s.key_and_value)
        x.right=erase(x.right,s.key)
    return x
root=erase(root,k)
```


BST 刪除的完整遞迴與身分契約 （續）



先在10右插11。刪8時後繼10沒有左子，但有右子11；複製10到原根後，遞迴刪原10位置，回傳11給12.left。根的左右範圍仍分別小於、大於10，中序恰少8。若刪根而只在函式內改區域指標，外部root可能指向已釋放節點，所以呼叫端必須接回回傳根。

此版承諾key-value集合，不承諾節點handle身分。若要求所有未刪元素的handle穩定，應以節點transplant更新parent及root，而不是複製後繼內容。泛型payload複製例外也需另定提交政策。成本是搜尋加後繼左鏈，共O(h+1)，無平衡時可Θ(n)。

一般後繼的單次成本與整段走訪

```
successor(x):
    require x is a live node and parent links are maintained
    if x.right!=null:
        x=x.right
        while x.left!=null: x=x.left
        return x
    p=x.parent
    while p!=null and x==p.right: x=p; p=p.parent
    return p
```

後繼是嚴格大於目前鍵的最小鍵，不是比較數值加1。若有右子樹，該子樹最左點是答案；否則沿父鏈跳過「剛走完其右子樹」的祖先，第一個把目前子樹放左側的祖先才是答案；跳到根外表示目前是最大鍵。此頁採普通child指標加parent，與本章的thread標記是不同表示。

平衡例根11、左5(2,7)、右17(13,19)：successor(5)沿右子到7；successor(7)先由7回5，再由5回11，答案11；successor(19)經17、11後回null。即使給了節點handle，一次successor仍可能走高度h，最壞O(h+1)，沒有自動的O(1)界。

從最左鍵開始反覆successor，輸出2,5,7,11,13,17,19。每條樹邊向下至多一次、向上至多一次，故初始化加完整走訪總Θ(n)，而非n次最壞O(h)必然等於Θ(nh)。若僅作不連續的任意查詢，不能把完整走訪的平均成本直接指派給每次查詢。若沒有parent、thread或顯式祖先堆疊，介面還須從root重搜祖先。

有序來源合併與線性建平衡樹

```
merge_unique(streams):
    H=min_heap of each nonempty stream's first record; pending=none
    while H not empty:
        (key,value,source)=H.pop_min()
        if pending==none: pending=(key,value)
        else if key==pending.key: pending.value+=value
        else: emit pending; pending=(key,value)
        advance source; if nonempty: H.push(next record from source)
    if pending!=none: emit pending
build_sorted(count):
    if count==0: return null
    left_count=count//2
    left=build_sorted(left_count)
    (k,v)=next(unique_sorted_stream)
    x=Node(k,v,left=left,right=null,parent=null,size=count)
    if left!=null: left.parent=x
    x.right=build_sorted(count-left_count-1)
    if x.right!=null: x.right.parent=x
    return x
```

有序來源合併與線性建平衡樹 （續）

先固定同鍵政策，例如value為出現次數、同鍵相加。三個來源[2,7,17]、[5,7,13]、[11,19]合併8筆，消去重複鍵後得到7筆2,5,7,11,13,17,19，其中7的value=2。k個來源、總輸入N筆時，heap合併O(k+N log(k+1))（含檢查k個來源的入口成本），k為常數時線性；不能對任意大k仍稱O(N)。不同鍵數U需先知道：可重讀來源時先計數再重播；一次性來源則先存U筆緩衝，不能憑空知道U。

build_sorted(7)先建左3筆：以5為根、2與7為子；下一筆11成全根，再建右3筆17(13,19)。每個節點在中序時恰消耗一筆，所以輸出中序等於輸入有序序列，歸納即證全子樹BST範圍。左右節點數相差至多1，遞迴高度O(log(U+1))，建樹$T(U)=T(\lfloor U/2\rfloor)+T(U-\lfloor U/2\rfloor-1)+\Theta(1)=\Theta(U)$。節點輸出空間Θ(U)、額外堆疊O(log(U+1))。

反例是把已排序U鍵逐個插入沒有平衡機制的BST，形成單鏈，總比較$1+2+\cdots+(U-1)=\Theta(U^2)$。輸入已排序可以幫助直接建構，不會自動讓一般insert變快。從有序陣列取中點也可達相同線性建樹；此串流版說明不必具備隨機索引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

引線二元樹重用空指標

- 以標記區分 child link 與 thread（引線），把原本空的左、右位置用來指向中序前驅或後繼。
- 找中序後繼時，若右連結是 thread，直接跟隨；若是真右子樹，先進右子，再沿真左子走到底。
- 完整走訪可不用遞迴或顯式堆疊，總時間 `O(n)`、額外走訪狀態 `O(1)`。
- 插入與刪除必須同步更新前後繼引線。若把 thread 當 child 遞迴，可能形成循環而不終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

引線走訪與插入的具體追蹤

- 在原創 BST 中，葉 6 的左引線指向 3，右引線指向 8；葉 10 的左引線指向 8，右引線指向 12。
- 走到 6 後，右標記為 thread，故直接到 8。8 的右標記是真 child，先到 12，再沿真左 child 到 10。
- 若在 10 右側插入新鍵 11，原本該位置是指向 12 的 thread。新 11 的左、右引線分別指 10、12，再將 10 的右連結改為 child 並指向 11。
- 走訪結果改為 `1,3,6,8,10,11,12`。若仍把 10 的右標記當 thread，搜尋與走訪會採錯語義。資料指標與標記必須作為同一更新維護。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向引線的中序後繼與右插入

```
successor(x):
    if x.right_tag==thread: return x.right
    x=x.right
    while x.left_tag==child: x=x.left
    return x
insert_right(p,k):
    require p.right_tag==thread and k>p.key
    s=p.right
    require s==null or k<s.key
    x=Node(k,left=p,right=s,left_tag=thread,right_tag=thread)
    p.right=x; p.right_tag=child
    if s!=null and s.left_tag==thread and s.left==p: s.left=x
```

本章不使用header sentinel，最左前驅與最右後繼為null。從最左真child開始，不斷successor直到null。插11於10的右thread位置：原10.right=12且tag=thread；新11.left=10、11.right=12皆thread；改10.right=11且tag=child。12.left若是真child仍指10，不能任意改成11，否則會丟失真子樹；只有它原本是指10的前驅thread才更新。

某次successor可沿左鏈O(h)，但全程每條真子邊與引線只被有界次數使用，總O(n)，額外O(1)走訪狀態。更新時tag與pointer是共同表示；只改指標卻不改tag，會讓新11被當成thread跳過或被誤當遞迴child造成循環。與普通BST刪除共享抽象排序，但引線修復不能省略。

引線二元樹的刪除、端點與身分契約

採前節的雙向引線表示，無 header、最外側 thread 為 null，鍵互異。搜尋只沿標記為 child 的指標下降。先把兩子刪除化成後繼節點的至多一子刪除，再修前後繼及父連結；不能沿 thread 遞迴刪除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

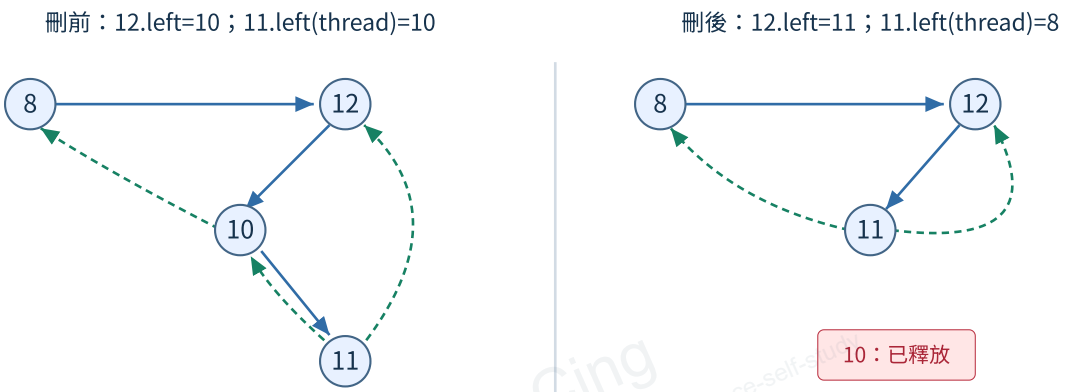
引線二元樹的刪除、端點與身分契約 （續）

```
predecessor(x):
    if x.left_tag==thread: return x.left
    x=x.left
    while x.right_tag==child: x=x.right
    return x
erase_threaded(k):
    p=null; z=root
    while z!=null and z.key!=k:
        if k<z.key:
            if z.left_tag==thread: return absent
            p=z; z=z.left
        else:
            if z.right_tag==thread: return absent
            p=z; z=z.right
    if z==null: return absent
    if z.left_tag==child and z.right_tag==child:
        p=z; s=z.right
        while s.left_tag==child: p=s; s=s.left
```

引線二元樹的刪除、端點與身分契約 （續）

```
        z.key_and_value=copy(s.key_and_value); z=s
    pred=predecessor(z); succ=successor(z)
    replacement=null
    if z.left_tag==child: replacement=z.left
    else if z.right_tag==child: replacement=z.right
    if pred!=null and pred.right_tag==thread and pred.right==z:
        pred.right=succ
    if succ!=null and succ.left_tag==thread and succ.left==z:
        succ.left=pred
    if p==null: root=replacement
    else if p.left_tag==child and p.left==z:
        if replacement!=null: p.left=replacement
        else: p.left=pred; p.left_tag=thread
    else:
        if replacement!=null: p.right=replacement
        else: p.right=succ; p.right_tag=thread
    release(z); return removed
```


引線二元樹的刪除、端點與身分契約 （續）



藍實線：真 child；綠虛線：thread。最外側 null thread 省略。
8.left 下的子樹 {1,3,6} 沒有改變，圖中省略。

具體刪除原樹中有右子 11 的 10：p=12、pred=8、succ=11、replacement=11。先把 11.left 的 thread 從 10 改指 8，12.left 再改真 child 11，11.right 的 thread 仍指 12，最後才釋放 10。完整中序由 [1,3,6,8,10,11,12] 變成 [1,3,6,8,11,12]，successor(8) 沿 12 的真左子到 11，predecessor(11) 直接跟 thread 到 8。

引線二元樹的刪除、端點與身分契約 （續）

刪葉 6 時，原父 3 的右 child 變成指向 successor=8 的 thread，不能保留 child 標記。刪唯一節點時 pred=succ=replacement=null，令 root=null 後釋放；空樹或找不到鍵不修改狀態。兩子分支複製完整 key-value 到目標位置，實際釋放的是後繼節點；此版只承諾字典值，不承諾未刪元素的節點 handle 穩定，複製成本與例外另計。

正確性分三層：後繼無真左子，故化簡後至多一子；移接 replacement 保留其他真 child 子樹及其中序順序；只有 pred 的後繼 thread、succ 的前驅 thread 及父的空側標記可能指向被刪節點，逐一修復後不留懸空 thread。所有 pointer 與 tag 一起維持，不能把真 child 無條件覆寫成 thread。搜尋、找後繼及前後驅各走至多高度 h 的路徑，總 O(h+1) 時間、O(1) 額外指標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小堆積有形狀與順序兩項不變量

- binary min-heap 是完全二元樹，且每個父鍵不大於子鍵。根因此是全域最小值。
- 堆積不要求左子小於右子，也不要求整棵左子樹小於右子樹，所以不能用 BST 搜尋方式找任意鍵。
- 合法陣列為 `[2,5,4,9,7,8]`。它不是整體排序，任意鍵搜尋最壞仍為 `O(n)`。
- Priority Queue ADT 支援 insert、find-min、delete-min。binary heap 是其中一種實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

sift-up 與 sift-down

- 插入 1，先放最後一格，再與父交換直到父鍵不大於它。本例從索引 6 經 2 到 0，結果 `[1,5,2,9,7,8,4]`。
- 刪最小值時，以末元素放根，再與較小子交換直到堆序恢復。只與任意一個子交換可能留下較小子在父下面。
- 沿更新路徑以外的邊未改變，路徑上的違規逐層修復，故保留堆序與完全性。
- insert、delete-min 的堆積重整最壞 `O(log n)`，find-min `O(1)`。若底層陣列擴容，某次 insert 另需線性搬移；該配置與搬移成本另以攤銷分析處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二元最小堆積的完整上浮與下沉

```
insert(v):
    A.append(v); i=len(A)-1
    while i>0:
        p=(i-1)//2
        if A[p]<=A[i]: break
        swap(A[p],A[i]); i=p
delete_min():
    require len(A)>0
    result=A[0]; last=A.pop()
    if len(A)==0: return result
    A[0]=last; sift_down(0,len(A)); return result
sift_down(i,n):
    while i<n//2:
        j=2*i+1
        if j+1<n and A[j+1]<A[j]: j+=1
        if A[i]<=A[j]: break
        swap(A[i],A[j]); i=j
build_heap():
    for i=n//2-1 down to 0: sift_down(i,n)
```

二元最小堆積的完整上浮與下沉 （續）

以本章[2,5,4,9,7,8]，插1後附加到6號、交換6↔2再2↔0，得[1,5,2,9,7,8,4]。刪最小時拿末4補根，得[4,5,2,9,7,8]，比較兩子5、2後換較小2，得[2,5,4,9,7,8]，下一子8不小於4而停止。單點heap刪除後直接回傳，不能在空A上寫A[0]。

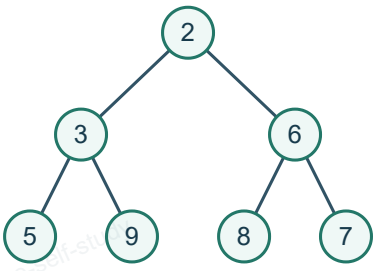
上述sift_down的迴圈條件 i<n//2（整數除法）恰好表示至少有左子，避免先算2i+1時溢位。n=3時只有i=0為內點；n=4時i=0、1都是內點。進入迴圈後才算j=2i+1並檢查j+1<n，避免缺右子時讀越界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建堆的每次比較與交換

自底向上建最小堆

節點與比較	交換後陣列（索引從 0 開始）
i=2 6 ≤ 7, 8	[9, 5, 6, 2, 3, 8, 7]
i=1 5 ↔ 2	[9, 2, 6, 5, 3, 8, 7]
i=0 9 ↔ 2	[2, 9, 6, 5, 3, 8, 7]
i=0 9 ↔ 3	[2, 3, 6, 5, 9, 8, 7]



i=2 的子堆已有效；每次下沉前，兩個子樹均為堆。

最後一列是完整陣列，右圖是同一狀態的樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自底向上建堆為何線性

- 從最後內部節點 $\lfloor n/2 \rfloor - 1$ 到根依序 sift-down。開始處理某節點時，其左右子樹已是堆。
- 高度為 h 的節點數為 $O(n/2^h)$，其下沉成本 $O(h)$。此寫法包含取整常數，不把近似節點數當成精確上界。
- 總成本 $O(n \sum_{h \geq 0} h/2^h) = O(n)$，而讀取輸入需 $\Omega(n)$，所以建堆為 $\Theta(n)$。
- 不能以「有 n 個點，每個 O(log n)」就宣稱緊界 $\Theta(n \log n)$。該乘法僅提供較鬆上界。

`std::priority_queue<int>` 預設取最大值。最小堆可指定 `std::greater<int>`，應確認比較方向與 min-heap 定義一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

怎麼選擇樹的表示？

結構	必須維護的條件	適合的操作
一般二元樹	左右子位置有意義	表達運算式或遞迴結構
BST	左子樹鍵 < 根鍵 < 右子樹鍵	搜尋、範圍與有序走訪；成本與高度有關
min-heap	完全形狀、父鍵 ≤ 子鍵	取最小值、插入和刪除最小值
雙向引線樹	child / thread 標記與指標一致	不用外部堆疊的中序走訪

先確認介面要保證什麼，再選表示。min-heap 的局部父子順序不能用來剪除整個子樹，因此任意鍵查找一般仍要線性時間。BST 若不維持平衡，搜尋也可能退化成線性。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；ISO C++ 草案 priority_queue。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

前置知識與目標

- 前置知識為集合、樹、堆疊、佇列、優先權佇列與歸納證明。
- 能選擇圖表示，證明搜尋與貪婪選邊的關鍵不變量，依邊權條件選擇最短路徑算法。
- 能區分最小生成樹、最短路徑、拓樸次序及關鍵路徑的不同最佳化目標。

Graph ADT 與問題契約

- 圖 $G=(V,E)$。有向邊 (u,v) 有方向，無向邊 $\{u,v\}$ 不區分方向。權重為映射 $w:E\to R$。
- 本章除另註外採無自環、無平行邊的圖。演算法可延伸到 multigraph，但表示與去重政策要明示。
- ADT 可提供頂點列舉、鄰居列舉、加邊與權重查詢。不同表示對這些操作有不同成本。
- 邊權 0 是合法數值。不可在加權矩陣中同時用 0 表示「無邊」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

鄰接矩陣與鄰接串列

表示	空間	判定特定邊	列舉 u 的鄰居
矩陣	$\Theta(V^2)$	$O(1)$	$\Theta(V)$
未排序串列	$\Theta(V+E)$	$O(\deg(u))$	$\Theta(\deg(u))$

此表以 $V=|V|, E=|E|$ 簡寫。無向圖每邊通常儲存兩筆鄰接項。串列中加邊是否需檢查重複，會改變插入成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS 的層數不變量

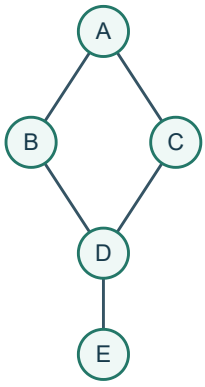
1. 起點 s 設 `dist[s]=0` 並入列，其餘為未發現。
2. 出列 u，對每個未發現鄰點 v 設 `dist[v]=dist[u]+1`、`parent[v]=u` 並入列。
3. 在入列時標記發現，避免同一點多次入列。

佇列的待處理距離保持非遞減。首次發現 v 時，任何更短路徑的前驅都應更早被處理，故該距離為最少邊數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS 的佇列與前驅

BFS：入列時標記，逐輪追蹤



出列 / 之後的佇列 / 新 parent

A	[B, C]	B←A, C←A
B	[C, D]	D←B
C	[D]	—
D	[E]	E←D
E	[]	—

距離 A,B,C,D,E = 0,1,1,2,3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS、時間戳與有向環

- DFS 把頂點分為白色未發現、灰色正在處理、黑色已完成。遞迴堆疊對應灰色路徑。
- 有向圖中，指向灰色祖先的 back edge 證明存在有向環。指向黑色頂點本身不表示有環。
- 完成時間可支援拓樸排序。若圖無環，依完成時間遞減排列是一個合法拓樸次序。
- 只從單一起點搜尋不保證涵蓋全圖。要建立搜尋森林，需對剩餘白色頂點再次啟動。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋成本與可重現追蹤

- 鄰接串列中，每個頂點與每筆鄰接項處理固定次數，完整 BFS/DFS 為 $\Theta(V+E)$，輔助空間 $O(V)$。
- 矩陣版本對每點掃整列，完整搜尋為 $\Theta(V^2)$。不可不加條件套用串列成本。
- 原創無向圖邊為 `AB,AC,BD,CD,DE`，鄰居按字母順序。從 A 做 BFS 為 `A,B,C,D,E`。
- 同圖遞迴 DFS 為 `A,B,D,C,E`。走訪順序受鄰居次序影響，距離與可達性結論不受影響。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS 與 DFS 的完整狀態機

```
BFS(s):
    for v in V: dist[v]=infinity; parent[v]=none
    dist[s]=0; Q.push(s)
    while Q not empty:
        u=Q.pop_front()
        for v in adj[u]:
            if dist[v]==infinity:
                dist[v]=dist[u]+1; parent[v]=u; Q.push(v)
DFS(u):
    color[u]=gray; discover[u]=++clock
    for v in adj[u]:
        if color[v]==white: parent[v]=u; DFS(v)
        else if directed and color[v]==gray: report back edge
    color[u]=black; finish[u]=++clock
```

對無向AB,AC,BD,CD,DE、字母序鄰居，BFS出列後的queue為：A後[B,C]；B後[C,D]；C後[D]；D後[E]；E後[]。d=(0,1,1,2,3)，parent=(無,A,A,B,D)。DFS時間戳為A:(1,10)、B:(2,9)、D:(3,8)、C:(4,5)、E:(6,7)，parent C=D、E=D。

有向圖灰邊表示回祖先形成環；無向圖必須忽略回parent的同一條樹邊，不能把每條無向邊的反向鄰接項當成新環。完整森林先將全圖白化，再對每個白點呼叫DFS；單源BFS不會替其他分量定距離。深度、queue與parent皆需Θ(V)最壞空間，不包含輸入。

MST 的目標與割性質

- 對連通無向加權圖，生成樹含全部頂點且恰有 V−1 邊；MST 最小化總權重，允許負權。非連通圖改求各分量的最小生成森林。
- 維持不變量「存在一棵 MST T，包含目前全部已選邊 F」。割 (S,V\S) 尊重 F，意指 F 沒有跨割邊。令 e 是一條跨割最輕邊。

若 e∈T，不變量直接延續。否則 T+e 有唯一環；該環除 e 外還有跨同一割的邊 f。因割尊重 F，f∉F；又 w(e)≤w(f)。故 T′=T−f+e 仍是生成樹，權重不大於最優 T，只能同為最優，並包含 F∪{e}。交換沒有丟掉先前選擇，這才證明 e 對目前 F 安全。

不能只證「每條邊各自屬於某棵 MST」。三角形三邊權皆 1，每條各自出現在某棵 MST，但三條一起成環，沒有任何 MST 能同時包含它們。安全性必須相對整組 F 而言。

Prim 逐步擴張一個頂點集合

- 令 S 為已納入頂點，為每個外部點保留連到 S 的最小邊權 `key[v]` 與對應 parent。
- 每次選最小 key 的外部點加入 S，再更新鄰邊。選入邊是割 `(S,V\S)` 的最輕邊。
- 二元堆與鄰接串列為 `O((V+E)log V)`；連通非單點圖可簡寫 `O(E log V)`。矩陣線性挑點版本為 `O(V²)`。
- key 是「連到目前樹的單邊成本」，不是從起點累計距離，因此不可與 Dijkstra 的 dist 混用。

歸納從 F=∅ 開始。Prim 的 F 全在目前 S 內，故割 (S,V\S) 尊重 F；取其最輕跨邊可由安全邊定理維持「某棵 MST 包含 F」。連通圖最後有 V−1 條無環邊，F 本身就是那棵 MST。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kruskal 與互斥集合

1. 將全部邊依權重排序。
2. 從輕到重處理邊 `(u,v)`。若兩端在不同分量，加入邊並 union。
3. 連通圖選滿 `V-1` 邊即可停止。否則完成最小生成森林。

使用 path compression 及 union by rank 的互斥集合，m 次操作可用 `O(m α(V))` 攤銷界。排序通常主導 `O(E log E)`。此處 α 為反 Ackermann 函數，與雜湊負載率不是同一符號。

Kruskal 令 S 為目前 F 中含 u 的整個連通分量；待選 e=(u,v) 兩端不同分量，故 e 跨此割且割尊重 F。若有更輕跨邊，它已較早被處理，當時兩端也不可能同分量，應已加入並合併，與現在仍跨割矛盾。因此 e 是該割最輕邊，安全邊定理維持同一包含不變量。Θ(V) 初始化不能省略；一般總界為 O(V+E log(E+1)+E α(V))，DSU 另占 Θ(V) 空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

互斥集合的完整森林操作

互斥集合（disjoint-set union, DSU）表示頂點的分割。每個非空集合恰有一個根，以 parent[root]=root 表示；find 回傳代表根，union 只合併兩個原本不同的集合。陣列在 make_set 前配置 V 格，頂點 ID 必須有效。

```
make_set(v): parent[v]=v; rank[v]=0
find(v):
    r=v
    while parent[r]!=r: r=parent[r]
    while parent[v]!=v:
        next=parent[v]; parent[v]=r; v=next
    return r
union(u,v):
    a=find(u); b=find(v)
    if a==b: return false
    if rank[a]<rank[b]: swap(a,b)
    parent[b]=a
    if rank[a]==rank[b]: rank[a]+=1
    return true
```


互斥集合的完整森林操作 （續）

第二個 find 迴圈必先保存 next，否則覆寫 parent 後會丟掉尚未壓縮的路徑。rank 是合併歷史的高度上界，不是壓縮後的實際高度；壓縮後不重新降低 rank。union 的回傳值恰好可作為 Kruskal 是否收邊的條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

互斥集合的逐輪狀態與可自足成本界

沿同一 MST 的選邊次序 BC、AB、DE、CD 呼叫 union；rank 相等時保留第一個根。下表 parent 欄均依 A,B,C,D,E 排列。

已完成操作	parent[A..E]	根及其 rank
五次 make_set	[A,B,C,D,E]	A0,B0,C0,D0,E0
union(B,C)	[A,B,B,D,E]	A0,B1,D0,E0
union(A,B)	[B,B,B,D,E]	B1,D0,E0
union(D,E)	[B,B,B,D,D]	B1,D1
union(C,D)	[B,B,B,B,D]	B2
find(E)	[B,B,B,B,B]	B2

最後一次 find 的第一輪走 E→D→B，第二輪把 E 直接接 B；沒有把 E 的集合改掉。再遇 BD、AC、CE 時兩端都回 B，union 回 false，不能增加 rank，也不能再收邊。

互斥集合的逐輪狀態與可自足成本界 （續）

正確性由森林不變量歸納：find 只接往同一棵樹的祖先根，所以不改分割且不產生環；union 只把一個根接到另一個根，恰合併兩集合。每個非根到父的 rank 嚴格增加：低 rank 接高 rank，平手時勝根 rank 加一；壓縮接更高祖先仍保持。初始 rank0 集合恰含一點。rank 為 r 的根所代表集合至少含 2^r 點，因升 rank 只發生在兩個至少各 2^{r-1} 點的集合合併。因此 $\mathrm{rank} \le \lfloor \log_2 V \rfloor$，路徑長與單次 find/union 皆為 O(1+log(V+1))，初始化 Θ(V)、空間 Θ(V)。這已完整支持 Kruskal 的 O(V+E log(E+1)+E log(V+1)) 上界。

若進一步使用反 Ackermann 分層定理，V 次 make_set 後的 m 次 find/union 有更細的 O(V+m α(V)) 序列界；該精細定理不由上述 rank 高度證明推出。本章的演算法與較鬆對數界不依賴省略的精細證明，排序與所有頂點初始化仍需計入。

MST 追蹤與反例

- 邊權為 AB=2,AC=5,BC=1,BD=4,CD=3,CE=7,DE=2。
- Kruskal 可依序選 BC,AB,DE,CD，總權重 1+2+2+3=8。平手順序可不同，合法最優總權重相同。
- Prim 從 A 可選 AB,BC,CD,DE，得到同一組邊但順序不同。
- MST 不保證起點到各點最短。例如三角形 AB=2,BC=2,AC=3 的 MST 使 A 到 C 為 4，原圖最短距離為 3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

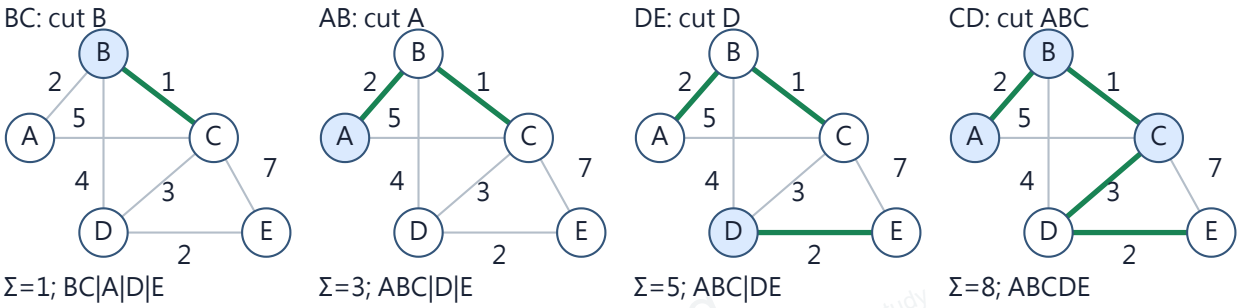
Prim 與 Kruskal 的完整選擇與狀態

```
Prim(s):
    key[*]=infinity; parent[*]=none; key[s]=0
    while some vertex is unselected:
        u=unselected vertex with minimum key
        if key[u]==infinity: start a new component or report disconnected
        select u
        for (u,v,w) in adj[u]:
            if v unselected and w<key[v]: key[v]=w; parent[v]=u
Kruskal():
    make_set(v) for all v; result=[]
    for (u,v,w) in sorted_edges:
        if find(u)!=find(v): result.append((u,v,w)); union(u,v)
    return result
```

https://github.com/xinhi

Prim 與 Kruskal 的完整選擇與狀態 （續）

Kruskal：藍為割側，綠為累積選邊



同一AB2,AC5,BC1,BD4,CD3,CE7,DE2圖，Prim從A的外部key依序：選A後B2,C5,D∞,E∞；選B後C1,D4,E∞；選C後D3,E7；選D後E2；選E結束。parent依次B←A、C←B、D←C、E←D，邊總重8。這些key從5降1、7降2，代表單邊連結成本，不是累加距離。

Kruskal完整排序BC1,AB2,DE2,CD3,BD4,AC5,CE7；前四條各合併不同分量，後三條兩端已同分量而拒絕。DSU只負責環與分量，不負責權重最優；最優性來自前述割交換定理。相等權重可有不同MST，不能要求唯一樹。

最短路徑與鬆弛

- 路徑長度為邊權和，$\delta(s,v)$ 是 s 到 v 的最小路徑長度。不可達記為 $+\infty$。
- 若 s 可達某個負環，且負環又可達 v，則 v 的路徑長度可無界下降，沒有有限最短值。
- 鬆弛 (u,v) 為若 $d[u]$ 有限且 $d[u]+w(u,v)<d[v]$，就更新 d[v] 與 parent。
- 每個有限 d 都是某條已發現路徑長度，因此是最短距離的上界。機器整數相加需另防溢位。

Dijkstra 的非負權條件

- 初始化 d[s]=0。每次選出尚未確定且 d 最小的 u，將 u 標為 settled，再鬆弛其出邊。
- 正確性關鍵是尚未處理路徑的後續邊不會使長度下降。這要求所有相關邊權非負。
- 反例 `s→a=2,s→b=5,b→a=-4`。若先把 a 的距離 2 永久確定，就錯過真正距離 1。
- 零權邊合法。不可把「正權」誤寫成必要條件。負權圖應改用符合條件的演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 原創 trace 與實作差異

有向邊 `s→a=4,s→b=1,b→a=2,a→t=1,b→t=6`。

settled	d[a]	d[b]	d[t]
s	4	1	∞
b	3	1	7
a	3	1	4
t	3	1	4

indexed binary heap 為 `O((V+E)log V)`。採重複入堆而不 decrease-key 時，要跳過舊距離，空間可達 `O(V+E)`，時間保守寫 `O((V+E)log(V+E))`。

矩陣版每輪線性挑選未確定的最小距離，並掃描該頂點整列，時間 $O(V^2)$、矩陣空間 $\Theta(V^2)$；鄰接串列搭配 indexed heap 使用不同的選取與更新方式。兩版的鬆弛規則和非負權前提相同，不能只看名稱就混用成本。

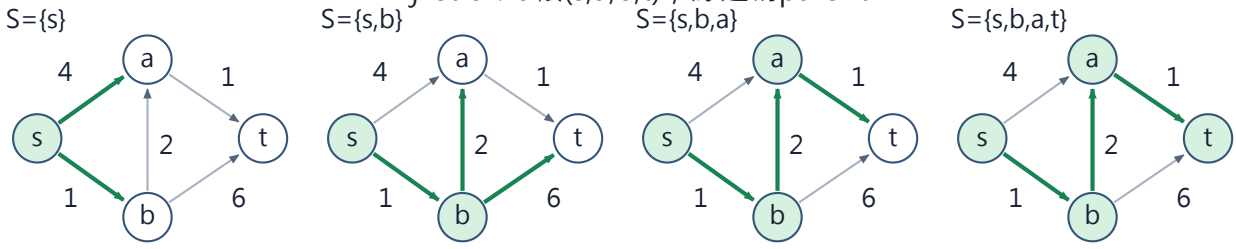
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 的優先佇列、舊項與前驅

```
Dijkstra(s):
    require all relevant edge weights>=0
    d[*]=infinity; parent[*]=none; d[s]=0
    H.push((0,s)); settled={}
    while H not empty:
        (du,u)=H.pop_min()
        if du!=d[u] or u in settled: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            if d[u]+w<d[v]:
                d[v]=d[u]+w; parent[v]=u; H.push((d[v],v))
```


Dijkstra 的優先佇列、舊項與前驅 （續）

Dijkstra：d依(s,a,b,t)；綠邊為parent



同一s→a4,s→b1,b→a2,a→t1,b→t6圖，初H=[(0,s)]。取s後H含(1,b),(4,a)；取b後加入(3,a),(7,t)；取a=3後加入(4,t)。舊(4,a)再彈出時因du≠d[a]而跳過；t=4確定後舊(7,t)也跳過。最終d=(s0,a3,b1,t4)，parent a=b,b=s,t=a。

證明settle u正確：取一條更短s→u路徑上第一個未確定點y，其前驅x已確定且已鬆弛，故d[y]≤該路徑前綴長；非負後續邊使該前綴≤整條更短路長，與u為最小d矛盾。負邊反例s→a2,s→b5,b→a−4沒有負環卻破壞這一步。距離相等時不必任意換parent，避免零權邊下人為形成前驅循環。

Bellman–Ford 的邊數歸納

- 做 `V-1` 輪全邊鬆弛。以同步陣列版本而言，第 k 輪後得到至多 k 邊的最短 walk 長度。
- 以原陣列立即更新的版本可能提早傳播，但每輪後至少已涵蓋所有至多 k 邊路徑，所以同樣正確。
- 沒有可達負環時，最短路徑可取簡單路徑，至多 `V-1` 邊。再一輪仍可鬆弛，即證明 s 可達負環。
- 原地鬆弛版時間 O(V+VE)；本章同步版每輪複製 V 格，時間 O(V(V+E))，另用 O(V) 距離與 parent 空間。只從 s 初始化，偵測的是 s 可達負環，不能宣稱找出全圖所有負環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floyd–Warshall 的中介點遞迴

- 在沒有相關負環的條件下，定義 `D^(k)[i,j]` 為中介點只允許 `{0,…,k-1}` 的最短距離；若允許可重複經過負環的 walk，有限最小值本身就不存在。
- 遞迴式為 `D^(k+1)[i,j]=min(D^(k)[i,j],D^(k)[i,k]+D^(k)[k,j])`。
- 初始化對角為 0，直接邊用其權重，其餘為無窮大。迴圈順序以 k 在最外層，才能對應中介點不變量。
- 時間 `Θ(V³)`、空間 `Θ(V²)`。結束後 `D[v,v]<0` 表示有負環；若 i 可達這樣的 v 且 v 可達 j，該 i→j 的 walk 長度無下界，不能把陣列中的有限值當成有限最短距離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman–Ford 邊數層與 Floyd 中介點層

```
BellmanFord_synchronous(s):
    d[*]=infinity; d[s]=0
    repeat V-1 times:
        next=copy(d)
        for (u,v,w) in E:
            if finite(d[u]): next[v]=min(next[v],d[u]+w)
        d=next
    if any finite(d[u]) and d[u]+w<d[v]: report reachable_negative_cycle
Floyd():
    D=direct_edges_and_zero_diagonal
    for k in V:
        for i in V:
            for j in V:
                if finite(D[i,k]) and finite(D[k,j]):
                    D[i,j]=min(D[i,j],D[i,k]+D[k,j])
```

負邊無環例s→a2,s→b5,b→a−4,a→t3的同步BF距離(s,a,b,t)：第0輪(0,∞,∞,∞)，第1輪(0,2,5,∞)，第2輪(0,1,5,5)，第3輪(0,1,5,4)，額外輪無改善。原地版本可能第1輪已達最終值，不能把同步等號不變量誤套到原地更新。

Bellman–Ford 邊數層與 Floyd 中介點層 （續）

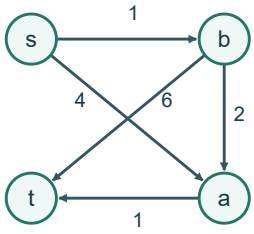
Floyd對非負例以(s,a,b,t)排序，初矩陣為 `[[0,4,1,∞],[∞,0,∞,1],[∞,2,0,6],[∞,∞,∞,0]]`。k=s無變；k=a使D[s,t]=5、D[b,t]=3；k=b使D[s,a]=3、D[s,t]=4；k=t無變。終矩陣 `[[0,3,1,4],[∞,0,∞,1],[∞,2,0,3],[∞,∞,∞,0]]`，其source=s列與Dijkstra一致。

回到本節的負邊例（b→a權重−4），再加入a→b權重1後，a↔b環權−3；不是在剛才b→a權重2的非負例上加邊。BF額外輪找到可再改善邊後，從其受影響端點向外搜尋，才能把a,b,t標為−∞，s仍0。Floyd若D[k,k]<0，僅對可達i→k且k→j的pair標−∞。布林「有負環」不等於完整分類，distance陣列有限數不能直接當答案。

同步版的 copy(d) 不能藏在邊鬆弛成本內。對 $V \ge 2$ 且 $E=0$ 的合法圖，固定執行 $V-1$ 輪，單是複製便寫入 $V(V-1)$ 格，為 $\Theta(V^2)$；原地版只需 $\Theta(V)$ 初始化，空邊迴圈共 $O(V)$。一般同步版每輪 $\Theta(V+E)$，故 $\Theta(V(V+E))$；原地版為 $O(V+VE)$。只有另有 $E=\Omega(V)$ 的前提，兩者才都可簡寫 $O(VE)$。同步層數不變量較精確，卻必須支付維持兩層陣列的成本。若加「無改善提早停止」，空邊例只做一輪，必須另述這個版本。

逐層允許中介點的 Floyd 更新

Floyd–Warshall：允許中介點後如何更新？



初始：D[s,a]=4，D[s,b]=1，D[s,t]=∞

k=a：D[s,t]=min(∞,4+1)=5

D[b,t]=min(6,2+1)=3

k=b：D[s,a]=min(4,1+2)=3

D[s,t]=min(5,1+3)=4

中介點順序 s、a、b、t；k 在最外層。最終 s→b→a→t 長 4。

k=a 已建立 b→a→t，下一層才能用它更新 s→t。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最短路徑遞迴的歸納與負環判定

```
// Synchronous edge-count recurrence:
D[0,s]=0; D[0,v!=s]=infinity
D[k,v]=min(D[k-1,v], min_{(u,v) in E}(D[k-1,u]+w(u,v)))
// Allowed-intermediate recurrence, with k outermost:
F[k+1,i,j]=min(F[k,i,j],F[k,i,k]+F[k,k,j])
```

Bellman–Ford 的同步歸納：0邊僅能停在起點。至多k邊到v的走訪，若不到k邊已由舊值涵蓋，否則最後一邊是某u→v，其前綴至多k−1邊；枚舉所有最後一邊恰給遞迴式的兩個方向。有限最短值若存在，可移除非負環而不增長度，留下至多V−1邊的簡單路徑，所以V−1輪足夠。

額外輪仍有鬆弛，表示某條至多V邊走訪優於所有至多V−1邊走訪；該走訪必含重複頂點。若每個環都非負，移除環便得到不更長的短走訪而矛盾，故必有起點可達負環。反過來，若存在可達負環卻所有邊都滿足 d[v]≤d[u]+w(u,v)，沿負環把不等式加總會得0≤負數，矛盾。因此至少一邊仍能鬆弛。只有由這些受影響端點往外可達的頂點，其答案才是−∞；不相關分量不受影響。

Floyd 的歸納依「允許的中介點集合」而非邊數。無相關負環時，可選不重複頂點的最短路徑。加入中介點k後，路徑要麼不用k，要麼拆成i→k與k→j，兩段只用舊集合；合併最優兩段又是合法走訪，故取最小給等號。原地更新可行，是本輪 D[k,k]=0 時 D[i,k]、D[k,j]不被降低；若已有負環，此有限距離論證不再適用，需改用可達性分類−∞。不能因同樣是三層迴圈就任意改成i最外層。

最短路徑的條件比較

方法	必要條件	問題與成本
BFS	每邊相同正成本或只算邊數	單源 O(V+E)
DAG 拓樸鬆弛	有向無環，可含負邊	單源 O(V+E)
Dijkstra	非負邊權	單源，堆版本見〈Dijkstra 的優先佇列、舊項與前驅〉
Bellman–Ford	可含負邊	單源；原地 O(V+VE)，同步 O(V(V+E))；偵測可達負環
Floyd–Warshall	可含負邊	全點對 $\Theta(V^3)$

負權限制、圖是否無環、需要單源還是全點對，應先於資料結構偏好決定方法。

瓶頸路徑的目標、鬆弛與完整追蹤

```
bottleneck(s):
    B[*]=infinity; parent[*]=none; B[s]=-infinity
    H.push((-infinity,s)); settled={}
    while H not empty:
        (bu,u)=H.pop_min()
        if u in settled or bu!=B[u]: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            candidate=max(B[u],w)
            if candidate<B[v]:
                B[v]=candidate; parent[v]=u; H.push((candidate,v))
```

瓶頸值定義為路徑最大邊權，目標是最小化這個最大值，即 $\min_{P:s\rightsquigarrow t}\max_{e\in P} w(e)$，不是把邊權相加。空路徑用−∞作max的單位元，不可達用+∞。對非負邊可把起點改0，但若容許負邊，起點0會把全負路徑的瓶頸誤抬高到0。

無向例s–p=6、p–t=6、s–q=2、q–t=9、p–q=7。取出s後B(p,q,t)=(6,2,∞)；取q後(6,2,9)、t.parent=q；取p後(6,2,6)、t.parent=p；最後t確定6，路徑s,p,t。最短總長卻選s,q,t得11，另一路總長12。相同資料的兩個目標產生不同答案，不能只換方法名稱。

瓶頸路徑的目標、鬆弛與完整追蹤 （續）

正確性仍用最小標籤確定：任一更佳路徑的第一個未確定點，其已確定前驅已給它不大於該路徑前綴最大值的候選；而延長路徑取max永不降低標籤，所以它的標籤不大於整條更佳路，與取出最小u矛盾。這個單調性對負邊也成立，與加法Dijkstra的非負前提不同。二元堆含舊項的界同前述Dijkstra；有indexed heap為O((V+E)log(V+1))。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MST 路徑的瓶頸性質與門檻連通

```
threshold_bottleneck(s,t):
    require undirected graph and s!=t
    make_set(v) for all vertices
    for (u,v,w) in edges sorted by weight:
        union(u,v)
        if find(s)==find(t): return w
    return unreachable
```

無向圖取任一MST T，令其s到t唯一路徑上的最大邊權為β，選一條達β的邊e。移除e把T分成s、t兩側。若原圖有跨此割而更輕的邊f，用f換e會使生成樹總重下降，與T最小矛盾。因此任何s→t路徑跨割時至少用一條權重≥β的邊，瓶頸值不可能小於β；T的路徑本身達β，兩界相等。

這不表示MST路徑最小化總和。本章例子的MST為{s–q2,s–p6,p–t6}、總重14；s到t樹路徑總長12不是圖中最短11，但最大邊6確是最小瓶頸。MST也允許負邊，交換證明與max論證仍成立。此樹路徑性質不直接套到有向圖，因一般MST不是有向可達樹的定義。

MST 路徑的瓶頸性質與門檻連通 （續）

另一個等價觀點：答案是讓 s, t 首次在 $G_{\le \lambda}$ 連通的最小 λ。按權重由小到大union，第一次同集合時返回當前權重；對全部 V 個頂點 make_set 先花 Θ(V) 時間及 Θ(V) DSU 空間，故總時間 O(V+E log(E+1)+E α(V))。若另儲存待排序邊，邊緩衝為 Θ(E)，排序額外工作空間依具體排序法另計。E=0 而有 V 個孤立點時，演算法仍初始化全部頂點後回 unreachable，為 Θ(V)，不能把成本寫成零。這個方法只回答一對點而不計距離和；多次查詢可在MST上預處理路徑最大值，但還需另外設計資料結構與預處理成本。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

多棵 MST 的權重層與必要邊

```
classify_MST_edges(G):
    require G undirected and connected
    DSU=singleton components
    for all equal-weight groups in increasing weight:
        Q=multigraph whose vertices are current DSU components
        for edge (u,v) in this group:
            if find(u)==find(v): mark edge as never_in_MST
            else: add its original edge ID between these two vertices in Q
        mark Q bridges as in_every_MST; other Q edges as optional
        for (u,v) in this group: union(u,v)
```

固定門檻 λ，令 c_λ 是只保留權重至多 λ 的邊之原圖分量數。任一 MST 中權重至多 λ 的邊必有恰 $n - c_\lambda$ 條：若在某個 $G_{\le \lambda}$ 分量內還不連通，沿該分量路徑找一條跨樹內分量且權重至多 λ 的邊，加進 MST 造成的環必含權重大於 λ 的邊；替換即更輕，矛盾。因這些邊本來無環，連通各分量後恰有 $n - c_\lambda$ 條。

相鄰權重層的計數相減，得每棵MST對每個權值選的邊數都相同；不代表選的是同一組邊，也不代表所有可替代邊都同權。先收縮嚴格較輕邊的分量，在當前等權multigraph Q中選各分量的生成森林。Q的bridge必選；在cycle上的邊可選也可不選；收縮後self-loop永不選。必須整組分類完才union，逐邊先union會把同權替代邊誤判成永不選。

多棵 MST 的權重層與必要邊 （續）

Q可能有平行邊，雖原圖是簡單圖也一樣；兩個分量間的兩條平行邊彼此可替代。橋偵測需以edge ID略過父邊，而不是略過所有通向parent頂點的邊。排序加各Q的線性DFS，總O(E log(E+1))，另加DSU攤銷成本。這把「找到一棵」提升成可證明某邊是否對所有最優解都必要。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

橋的 low-link 完整程序與證明

```
bridges(Q):
    tin[*]=unset; clock=0; result=[]
    for v in Q.vertices:
        if tin[v]==unset: dfs(v,none)
    return result
// dfs shares tin, low, clock, result with the enclosing routine.
dfs(v,parent_edge):
    tin[v]=low[v]=clock; clock+=1
    for (u,edge_id) in Q.adj[v]:
        if edge_id==parent_edge: continue
        if tin[u]==unset:
            dfs(u,edge_id); low[v]=min(low[v],low[u])
            if low[u]>tin[v]: result.append(edge_id)
        else: low[v]=min(low[v],tin[u])
```

tin記第一次發現時間；low[v]是由v子樹先走零或多條DFS樹邊向下，再至多走一條非父回邊可達的最小tin。這是無向圖的DFS量，不是最短路徑距離。若樹邊v→u有low[u]>tin[v]，u子樹沒有邊回到v或更上祖先，刪掉(v,u)即斷開；若low[u]≤tin[v]，存在繞過父邊的回連路徑，故不是橋。

橋的 low-link 完整程序與證明 （續）

數例：Q頂點X,Y,Z，X–Y有e1、e2兩條平行邊，Y–Z只有e3。從X以e1進Y，tin(X)=0、tin(Y)=1；在Y略過e1，但e2回X使low(Y)=0。進Z得tin(Z)=low(Z)=2，2>1所以e3是橋；返回X時low(Y)=0不大於0，e1不是橋，e2是回邊也不是橋。若把所有通向X的邊都略過，就錯把e1判成橋。

每個頂點只首次進入一次，每條無向邊的兩筆鄰接項各處理一次，時間Θ(|V_Q|+|E_Q|)、額外空間O(|V_Q|)。分類MST時只需建立各權重組有實際入射邊的商圖頂點，所有組的頂點數總和O(E)，不應每組配置整個原圖V大小陣列而仍宣稱總線性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

切點：刪除一個頂點會發生什麼事？

無向圖的 articulation point（切點）是刪掉該頂點及其入射邊後，整張圖的連通分量數增加的頂點。這個定義也適用非連通圖；DFS 必須對每個未探索分量建立森林。

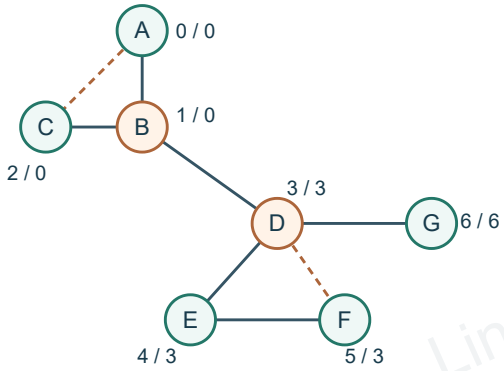
以 tin[v] 記發現時間，low[v] 記從 v 的 DFS 子樹經零或多條樹邊向下，再至多一條回邊，所能抵達的最小發現時間。沿樹邊 v→u 完成子樹後，以 low[u] 更新 low[v]。

非根 v 是切點，當且僅當有 DFS 子節點 u 滿足 low[u]≥tin[v]。DFS 根則必須至少有兩個 DFS 子節點；沒有父側分量，不能套用非根判斷。橋的條件是 low[u]>tin[v]，等號不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

切點與橋的數值追蹤

切點與橋：同一棵 DFS 樹



節點旁為 tin / low

實線：DFS 樹邊

虛線：回祖先的邊

B：low[D]=3 ≥ tin[B]=1

D：low[E]=3 ≥ tin[D]=3

橋為 BD、DG（嚴格大於）

根 A 只有一個 DFS 子節點 B，因此 A 不是切點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼切點是 ≥ 而橋是 > ?

非根 v 與子節點 u：若 low[u]≥tin[v]，u 子樹沒有回邊到 v 的嚴格祖先。刪去 v 後，子樹無法連到父側，所以 v 是切點；反之 low[u]<tin[v] 為每個子樹提供繞過 v 的回連，刪 v 不會把該子樹與父側分開。

等號 low[u]=tin[v] 時，子樹可經回邊回到 v，因此樹邊 (v,u) 本身有替代路徑而不是橋；但刪掉 v 時連同回到 v 的邊一起消失，v 仍可能是切點。根有至少兩個 DFS 子樹時，兩者之間不可能有跨邊，否則 DFS 會把它們合成同一個子樹；刪根才使分量數增加。

同圖中 low[E]=tin[D]=3，所以 DE 不是橋，但 D 是切點。low[D]=3>tin[B]=1，所以 BD 是橋；low[G]=6>tin[D]=3，所以 DG 也是橋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

切點的完整 DFS 程序

```
articulation_points(G):
    tin[*]=unset; clock=0; answer={}
    for v in V:
        if tin[v]==unset: dfs(v, parent_edge=none)
    return answer

dfs(v, parent_edge):
    tin[v]=low[v]=clock; clock+=1; children=0
    for (u, edge_id) in adj[v]:
        if edge_id==parent_edge: continue
        if tin[u]==unset:
            children+=1; dfs(u,edge_id)
            low[v]=min(low[v],low[u])
            if parent_edge!=none and low[u]>=tin[v]: answer.add(v)
        else: low[v]=min(low[v],tin[u])
    if parent_edge==none and children>=2: answer.add(v)
```

逐一處理森林可涵蓋孤立點。以 edge ID 排除進入該節點的那一條邊，能正確處理平行邊；不應排除所有通往父頂點的邊。鄰接串列時間 Θ(V+E)，額外空間 O(V)，遞迴深度最壞 Θ(V)。

AOV 網路與拓樸排序

- AOV 以頂點表示活動，有向邊表示先後依賴。合法排程要求圖為 DAG。
- Kahn 演算法把所有入度 0 的點入列。每次取一點輸出，移除其出邊，鄰點入度變 0 就入列。
- 不變量為剩餘入度只計尚未移除的邊。若還有點卻沒有入度 0，剩餘圖含有有向環。
- 輸出頂點數不足 V 即判定有環。時間 `Θ(V+E)`。多個入度 0 點會產生多種合法次序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AOE 網路與最早事件時間

- AOE 以邊表示有工期的活動，以點表示事件。活動需在前一事件完成後才能開始。
- 本教材工期 $d(u,v)\ge 0$，圖為 DAG，採共同起點及終點。多起終點可用 0 工期虛擬邊連接。
- 沿拓樸序計算 $E[v]=\max_{(u,v)}(E[u]+d(u,v))$，起點 $E=0$。
- 這是 DAG 最長路徑，而非 Dijkstra 最短路徑。最大值來自所有前置事件都必須完成的限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最晚事件、鬆弛與關鍵活動

- 終點 $L[t]=E[t]$，逆拓樸序計算 $L[u]=\min_{(u,v)}(L[v]-d(u,v))$。
- 活動 (u,v) 的最早開始為 $E[u]$，最晚開始為 $L[v]-d(u,v)$，總鬆弛為兩者差。
- 零鬆弛活動為關鍵活動。可有多條關鍵路徑，不能只回報任意一條後說其餘活動都可延遲。
- 此模型假設資源不限且工期固定。共用人員、機台衝突或隨機工期需要額外排程模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

關鍵路徑數據

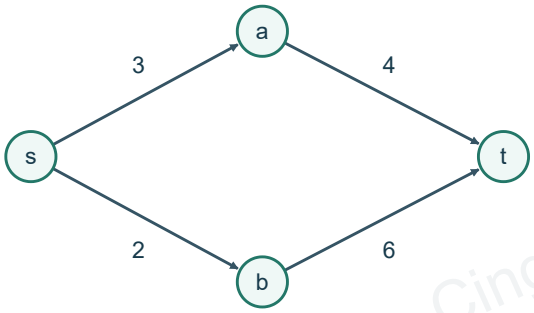
活動為 `s→a:3,s→b:2,a→t:4,b→t:6`。

- 最早事件為 `E[s]=0,E[a]=3,E[b]=2,E[t]=8`。
- 最晚事件為 `L[t]=8,L[a]=4,L[b]=2,L[s]=0`。
- `s→a` 與 `a→t` 的總鬆弛均為 1。`s→b` 與 `b→t` 的鬆弛為 0，關鍵路徑總工期 8。
- 兩個非關鍵活動各有 1 的總鬆弛，不代表可各自同時多耗 1 且仍不影響工期。共享同一路徑的鬆弛不能重複花用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AOE 的事件時間與活動鬆弛

AOE：事件時間與活動鬆弛



事件	最早 E	最晚 L
s	0	0
a	3	4
b	2	2
t	8	8

s→b→t 長 8；s→a→t 長 7，兩活動共享 1 單位鬆弛。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

拓樸排序與 AOE 的完整正反向計算

```
Kahn():
    indegree=counts of all incoming edges; Q=all zero-indegree vertices
    order=[]
    while Q not empty:
        u=Q.pop(); order.append(u)
        for (u,v) in outgoing: indegree[v]-=1; if indegree[v]==0: Q.push(v)
    require len(order)==V
AOE(order):
    E[*]=-infinity; E[start]=0
    for u in order:
        for (u,v,d): E[v]=max(E[v],E[u]+d)
    L[*]=infinity; L[finish]=E[finish]
    for u in reverse(order):
        for (u,v,d): L[u]=min(L[u],L[v]-d)
```

AOE同一s→a3,s→b2,a→t4,b→t6：初入度(s,a,b,t)=(0,1,1,2)，輸出s後(0,0,0,2)，輸出a後t降1，輸出b後t降0，最後t，得s,a,b,t。正向E初(0,−∞,−∞,−∞)，處理s得(0,3,2,−∞)，a後t=7，b後t=8。逆向L初t=8，處理b得L[b]=2，處理a得4，處理s得min(4−3,2−2)=0。

拓樸排序與 AOE 的完整正反向計算 (續)

四活動(最早開始,最晚開始,總鬆弛)為s→a:(0,1,1)，s→b:(0,0,0)，a→t:(3,4,1)，b→t:(2,2,0)。若活動工期固定且資源不限，零鬆弛路s,b,t決定總工期8；a路7共享一單位餘裕，兩活動各加1後變9，不能重複消耗相同餘裕。無共同start/finish時先加零工期虛邊，並確認每項欲分析活動位於起終可達範圍。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；MIT 6.006 圖搜尋與最短路徑講義。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

前置知識與目標

- 前置知識為陣列、堆積、遞迴、機率期望與第 1 章複雜度定義。
- 能說明每種排序的不變量、正確性與成本來源，區分穩定性、原地性及比較模型。
- 能根據已排序程度、鍵域、記憶體限制與最壞時間需求選擇方法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

排序契約與穩定性

- 輸出須為輸入的排列，並滿足 `A[i]≤A[i+1]`。只檢查有序不足以保證沒有遺失或複製元素。
- 穩定排序保留等鍵記錄的原相對順序。例如 `(2,A),(1,X),(2,B)` 排完仍須讓 A 在 B 前。
- 本教材稱「原地」為只需 `O(1)` 額外資料陣列空間，遞迴堆疊另列，不把它隱藏。
- 比較器必須符合一致的順序關係。C++ 排序要求 strict weak ordering，隨意使用 ≤ 作比較器會違反要求。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

插入排序的不變量

1. 在第 i 輪前，`A[0,i)` 已有序，且為原前 i 個元素的排列。
2. 保存 `key=A[i]`，將前綴中大於 key 的值向右移，直到插入位置空出。
3. 把 key 寫入該位置，得到有序前綴 `A[0,i+1)`。

只搬動「嚴格大於」key 的元素即可保留穩定性。原例 `[5,2,4,2]` 依序成 `[2,5,4,2]`、`[2,4,5,2]`、`[2,2,4,5]`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

插入排序與 inversions

- inversion（逆序對）為 `i<j` 且 `A[i]>A[j]` 的配對。每次右移消去目前 key 對應的一個逆序。
- 若總逆序數為 I，標準插入排序的時間可寫 `Θ(n+I)`。
- 已有序時 `I=0`，最佳 `Θ(n)`。完全反序且鍵互異時 `I=n(n-1)/2`，最壞 `Θ(n²)`。
- 這解釋插入排序適合小型或接近有序資料，並非只靠「平均很快」的模糊印象。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速排序的 partition 契約

- 本頁使用三向分割，選 pivot 值 p，將區間分成 `<p`、`=p`、`>p`，只遞迴左右兩區。
- 掃描期間維持 `[l,lt)<p`、`[lt,i)=p`、`[i,gt)` 未分類、`[gt,r)>p`。
- 小於 p 就交換至 lt 並前移 lt、i；等於 p 前移 i；大於 p 就減少 gt 並交換，i 留下重查。
- 終止量 `gt-i` 每輪下降，終止時沒有未分類元素。分割成本 `Θ(r-l)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三向分割逐輪索引與遞迴契約

```
partition(A,l,r,p):
    lt=l; i=l; gt=r
    while i<gt:
        if A[i]<p: swap(A[lt],A[i]); lt+=1; i+=1
        else if A[i]>p: gt-=1; swap(A[i],A[gt])
        else: i+=1
    return (lt,gt)
quicksort(A,l,r):
    if r-l<2: return
    choose p from A[l:r]
    (lt,gt)=partition(A,l,r,p)
    quicksort(A,l,lt); quicksort(A,gt,r)
```

對[4,2,4,7,1,4]，狀態(A,lt,i,gt)為：初始(原列,0,0,6)；讀4→(原列,0,1,6)；讀2→([2,4,4,7,1,4],1,2,6)；讀4→(同列,1,3,6)；讀7→([2,4,4,4,1,7],1,3,5)；重讀4→(同列,1,4,5)；讀1→([2,1,4,4,4,7],2,5,5)。只有[0,2)及[5,6)遞迴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三向分割逐輪索引與遞迴契約 （續）

等值區必非空，因pivot取自該區間，確保遞迴子區變小。若pivot不在資料且選到空等值區，某一側可能等於原區間而不終止。比較次數界須說明隨機選pivot；固定中點值不等於隨機選擇。較小側遞迴／較大側迴圈只保證堆疊$O(\log n)$，不改最壞$\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快排的最壞與期望

- 原例 `[4,2,4,7,1,4]` 選 p=4，可分成 `[2,1]` | `[4,4,4]` | `[7]`，區內順序不保證。
- 互異鍵若每次選到極端值，`T(n)=T(n-1)+Θ(n)`，最壞 `Θ(n²)`。
- 對固定互異輸入，每個子問題均勻隨機選 pivot，期望時間為 `Θ(n log n)`。若採經典「pivot 與每個其他鍵恰比較一次」的計數模型，期望比較數才是 `2(n+1)H_n−4n`。期望來自內部亂數。
- 三向分割可一次消去大量等鍵。此互異鍵的精確比較數公式不能直接套到所有三向實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機快排的比較期望與堆疊推導

```
quicksort_small_side(A,l,r):
    while r-l>=2:
        p=uniformly_random_value_from(A[l:r])
        (lt,gt)=partition(A,l,r,p)
        if lt-l<r-gt:
            quicksort_small_side(A,l,lt); l=gt
        else:
            quicksort_small_side(A,gt,r); r=lt
```

互異鍵排序後以排名 $1,\dots,n$ 表示，令 X_{ij} 指示排名 i,j 是否曾直接比較。經典二向版本每輪把 pivot 與其餘每鍵恰比較一次；兩鍵被比較若且唯若排名區間 $[i,j]$ 第一個被選作 pivot 者是兩端之一。區間內 $j-i+1$ 個鍵對稱，故 $\Pr(X_{ij}=1)=2/(j-i+1)$。線性期望給 $\mathbb{E}[C]=\sum_{i<j}2/(j-i+1)=2\sum_{d=1}^{n-1}(n-d)/(d+1)=2(n+1)H_n-4n=\Theta(n\log n)$，其中 $H_n=\sum_{k=1}^{n}1/k$。

隨機快排的比較期望與堆疊推導 （續）

三向分割本稿以兩次順序測試判別<、>、=，互異鍵時每輪仍對每個非pivot鍵做有界常數次比較，因此保留期望O(n log n)，但不能原封使用上述精確比較數。每個pivot對每個非pivot鍵至少要比較一次，相同的隨機排名分割也給上述指示變量和的下界，故為期望Θ(n log n)；這裡限互異鍵。全相同鍵三向分割一次結束，僅Θ(n)。

空間推導與時間期望分開：上方程序只真正遞迴較小非等值區，其長度≤(r−l−1)/2，活躍堆疊的子問題大小每次至少減半，故最壞O(log n)框架。較大區改由同一框架的while處理。即使亂數一直選到極端鍵，掃描總長n+(n−1)+⋯仍為Θ(n²)，只是堆疊不再線性。

遞迴空間與深度保護

- 直接遞迴兩側的快排，最壞堆疊深度 `Θ(n)`。資料分割原地不代表整體額外空間必為常數。
- 每次只遞迴較小的一側，較大的一側改迴圈，可把堆疊限制為 `O(log n)`，因遞迴子問題至多減半。
- 此改寫改善空間，不會消除 `Θ(n²)` 的最壞比較成本。
- introspective sorting 另在遞迴過深時切換到 heapsort。這是工程延伸，不宣稱原書必然包含。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

合併排序與穩定合併

- 左右兩段已排序時，比較各自尚未取出最小值，取較小者輸出。等鍵時先取左段即可保留穩定性。
- 不變量為輸出是已消耗輸入的有序合併，而且輸出末值不大於兩段未消耗元素。
- 原例 `[2_A,5]` 與 `[2_B,4]` 合併為 `[2_A,2_B,4,5]`。若等鍵先取右段，會破壞原順序。
- 陣列標準合併用 `Θ(n)` 緩衝區。遞迴 `T(n)=2T(n/2)+Θ(n)`，最壞 `Θ(n log n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

穩定合併與 merge-sort 的完整程序

```
merge(L,R):
    i=0; j=0; out=[]
    while i<len(L) and j<len(R):
        if L[i].key<=R[j].key: out.append(L[i]); i+=1
        else: out.append(R[j]); j+=1
    append remaining L[i:]; append remaining R[j:]
    return out
merge_sort(A):
    if len(A)<2: return A
    m=len(A)//2
    return merge(merge_sort(A[:m]),merge_sort(A[m:]))
```

合併L=[2_A,5]、R=[2_B,4]：先等鍵取A，游標(1,0)，輸出[A]；再取B，(1,1)，輸出[A,B]；再取4，(1,2)，輸出[A,B,4]；補5結束。每次輸出兩段尚未消耗最小值中較小者，證明排序；每個輸入只消耗一次，證明排列保持。

此 merge_sort 的切片版是語義偽碼；共享 buffer 的 C++ 實作避免每層保留不必要副本。

堆積排序的有序後綴

- 遞增排序先把陣列建成 max-heap，再交換根與末端，把末端排除在堆範圍外。
- 不變量為前段是 max-heap，後段已遞增，且前段所有鍵不大於後段所有鍵。
- 每次取出的根是尚未排序資料的最大值，放到後段最前位置。對剩餘根 sift-down 復原堆序。
- 建堆 `O(n)`，後續 n 次下沉總最壞 `O(n log n)`。可迭代實作為 `O(1)` 額外空間，通常不穩定。

堆積排序的完整驅動與有序後綴追蹤

```
heap_sort(A):
    build_max_heap(A)
    for end=len(A)-1 down to 1:
        swap(A[0],A[end])
        sift_down_max(A,0,end)    // heap is [0,end), sorted suffix excluded
```

堆積排序同一鍵列先成max-heap [5,2,4,2]。取5後[2,2,4|5]，下沉為[4,2,2|5]；取4後[2,2|4,5]；取2後[2|2,4,5]。豎線右邊是固定有序後綴，不能再參與sift。此非穩定算法即使值結果相同，也不能據此聲稱記錄身分順序保持。

sift_down_max 使用兩子中較大者，停止於 parent≥該子；索引與第 5 章 min 版相同，不等號相反。

計數排序的鍵域假設

- 鍵是 `[0,K)` 的整數時，以 K 格計數。前綴和給出各鍵在輸出中的結束位置。
- 由右向左掃原記錄，把 key=k 的記錄放入 `--end[k]`，可保留等鍵原次序。
- 時間 `Θ(n+K)`，輔助計數與輸出共 `Θ(n+K)`。K 遠大於 n 時可能不划算。
- 計數排序讀取鍵值並做直接索引，不只比較兩鍵，因此不受純比較排序下界直接限制。

穩定計數排序的配置與逐筆散佈

```
stable_count(records,K):
    out=array(len(records)); count=[0]*K
    for x in records: require 0<=x.key<K; count[x.key]+=1
    end=prefix_sum_inclusive(count)
    for x in reverse(records): end[x.key]-=1; out[end[x.key]]=x
    return out
```

對[(5,A),(2,B),(4,C),(2,D)]、K=6，count=[0,0,2,0,1,1]、end=[0,0,2,2,3,4]。由右掃D→out[1]，C→out[2]，B→out[0]，A→out[3]；得[B,D,C,A]，B仍在D前。LSD每輪套此穩定程序，不能把不穩定交換版當成等價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LSD 基數排序的穩定性需求

- 用 b 為基底，鍵有 d 位。由最低位到最高位，每輪對目前位做穩定排序。
- 歸納不變量為第 k 輪後，資料依最低 k 位的整數值排序。下一輪穩定性保留相同高位內的低位順序。
- 原例 `31,14,22,11`，個位排序為 `31,11,22,14`，十位穩定排序後為 `11,14,22,31`。
- 用穩定計數排序，每輪 `O(n+b)`，共 `O(d(n+b))`。有號數、變長字串與 Unicode 需要另定鍵編碼及順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LSD 的完整驅動與逐位不變量

```
LSD(A,b,d):
    for position=0..d-1:
        A=stable_count_by_digit(A,position,b)
    return A
```

LSD 固定寬 d 位、基底 b≥2，digit=(key//b^position) mod b；鍵須為非負整數或先作保序編碼。

31,14,22,11第一輪digit為1,4,2,1，穩定次序31,11,22,14；第二輪digit為3,1,2,1，穩定輸出11,14,22,31。第k輪後按低k位排序的不變量由等高位的穩定保留推出。若讀取一位需任意精度除法，Θ(d(n+b))的常數digit前提失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較排序下界的決策樹

- 假設 n 個互異鍵，演算法只能用兩鍵比較獲取順序資訊。確定性算法的比較歷程形成二元決策樹。
- n! 個輸入排列需要至少 n! 個可區別葉節點。高度 h 的二元樹至多 2^h 葉，因此 $h \ge \lceil \log_2(n!) \rceil$。
- 因 $n! \ge (n/2)^{n/2}$，有 $\log_2(n!) = \Omega(n \log n)$。所以最壞比較次數至少 $\Omega(n \log n)$。
- 限制在比較模型、互異鍵及一般排列。此證明沒有禁止已知鍵域的線性排序，也沒有聲稱每個輸入都需 n log n 次比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

方法比較必須保留限定

方法	最佳	最壞	主要額外空間	穩定
插入	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	是，僅移動嚴格較大值
隨機快排	視分割	$\Theta(n^2)$，期望 $\Theta(n \log n)$	堆疊可 $O(\log n)$	通常否
合併	$\Theta(n \log n)$，此標準版	$\Theta(n \log n)$	$\Theta(n)$	是，等鍵取左
堆積	視鍵重複與實作	$\Theta(n \log n)$	$O(1)$ 迭代版	否
LSD 基數	$\Theta(d(n+b))$，此版本	同左	$\Theta(n+b)$	每輪需穩定

表列成本以常數時間鍵操作為前提，不用不相容的模型比較「快慢」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

常見錯誤與驗證

- 測試排序必須同時檢查順序、元素多重集合與記錄穩定性。只有 `is_sorted` 可能漏掉資料遺失。
- 全相同鍵、已排序、反序、空陣列、單元素及大量重複鍵，會暴露不同分割邊界問題。
- `examples/sorting.cpp` 用固定種子資料與標準排序結果比對原創插入、合併及三向快排，並檢查負數與重複鍵。
- 程式案例不是正確性證明。分割不變量和遞迴終止論據仍需保留。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

來源與延伸

- 排序下界與非比較排序課程定位核對 MIT 6.006 2008 L10–L11。
- 比較器要求核對 C++ algorithms requirements。C++ 標準介面要求不等於標準規定某一個固定實作算法。
- 逆序數、分割追蹤、資料與練習均為原創教學編排。introsort 為延伸，不宣稱對應原書第二版節次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前置知識與目標

- 前置知識為字典 ADT、模算術、期望值、鏈結串列及攤銷分析。
- 能區分雜湊函數、碰撞解決與擴容政策，對每項效能結論明訂假設。
- 能追蹤 chaining、linear probing、刪除 tombstone 與重新雜湊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dictionary ADT 與碰撞

- 字典把每個唯一 key 映到一個 value，操作包含 find、insert-or-update、erase。
- 雜湊函數 $h:U\to\{0,\dots,m-1\}$ 把鍵域壓縮到 m 個桶。若 $|U|>m$，碰撞不可避免。
- key 相等時必須得到相同 hash。反向不成立，hash 相同的不同鍵仍需用 equality 分辨。
- 哈希值不是唯一識別碼。只比 hash 不比鍵，會使碰撞變成資料錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雜湊函數與機率空間

- 整數 division method 為 $h(k)=k \bmod m$。鍵若具有模 m 的規律，可能集中在同一桶。
- 隨機選函數與假設輸入均勻是不同模型。universal family 要求對任意不同 x、y，隨機 h 的碰撞機率至多 $1/m$。
- 若輸入在選 h 前固定，通用雜湊能以內部亂數界定碰撞期望；適應性攻擊者可能需要更強假設。
- 密碼學雜湊和一般雜湊表服務不同威脅模型。不能只看輸出位數便宣稱抗攻擊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate chaining 與負載率

- 每個桶保留同 hash 鍵的串列或其他集合，負載率 $\alpha=n/m$ 可以大於 1。
- 均勻雜湊下，獨立的不成功查詢之預期桶長為 α。若只採通用碰撞上界，且查詢鍵與儲存鍵集合在選函數前固定，桶長期望至多 α，不一定恰等於 α；兩者都給出搜尋期望 $O(1+\alpha)$，前提是鍵操作為常數時間。
- 成功搜尋的期望還與插入次序及查詢分布有關，但在常見均勻模型下也為 $O(1+\alpha)$。
- 最壞仍可有 n 個鍵同桶，搜尋為 $\Theta(n)$。隨機化給期望保證，沒有把最壞情況消掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chaining 例子

- 設 m=7、`h(k)=k mod 7`，依序插入 `10,17,24,8`。
- 桶 3 依插入尾端的政策保存 `[10,17,24]`，桶 1 保存 `[8]`，其餘空。
- 查 24 若用串列線性掃描，需檢查三個鍵。查 31 也會掃過桶 3，不能因 n/m 小便宣稱本次必為常數。
- 若需頻繁保留排序或處理攻擊性碰撞，可改用樹化桶。這會改變實作和成本，不屬於基本 chaining 自動具有的能力。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chaining 的完整字典操作

```
find(k):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): return reference(node.value)
    return absent
insert_or_update(k,v):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): node.value=v; return updated
    prepend newly constructed pair(k,v) to bucket[j]; n+=1
erase(k):
    j=hash(k) mod m; predecessor=dummy_before(bucket[j])
    while predecessor.next!=null:
        if equal(predecessor.next.key,k): unlink and release; n-=1; return true
        predecessor=predecessor.next
    return false
```

本程式採頭插，與先前示例的尾插政策不同：10,17,24都入桶3後，頭插順序為[24,17,10]；find24一次比較，find31三次。刪17後為[24,10]；再次insert24只更新value，不增加n。兩種插入政策都實作同Dictionary ADT，但成功搜尋的個別成本不同，不能拿一張trace當所有政策共同結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chaining 的完整字典操作 （續）

dummy_before是操作前驅，不必是常駐資料節點；刪首桶節點時要接回bucket head。平均鏈長α由鍵與hash模型決定；碰撞集中時最壞Θ(n)。插入若不查重而直接頭插，會變成允許重複的multimap而不是本章唯一key的dictionary。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

全表碰撞對與固定查詢的不同分母

```
// S is fixed before the random function h is selected.
tau=0
for each unordered pair {x,y} in S:
    tau += indicator(h(x)==h(y))
E[tau] = sum_{unordered {x,y}} Pr(h(x)==h(y))
```

一次不成功查詢的鏈長只把固定x與n個儲存鍵比較，期望上界n/m。全表碰撞對則有binom(n,2)個不同配對：若family為universal，即每對不同鍵碰撞機率≤1/m，便有E[τ]≤n(n−1)/(2m)。若每對恰1/m，則為等號。兩者都只需線性期望，不要求所有配對事件獨立；但不能把n/m誤當整張表的總碰撞數。

例如m=Θ(n^(4/3))時，全表E[τ]=O(n^(2/3))；固定查詢鏈長E[X]≤n/m=O(n^(−1/3))，還須加至少一次桶定位成本1。若m=Θ(n)，總碰撞期望可為Θ(n)，仍與單次查詢期望O(1)相容。碰撞對數與非空桶數也不同：單桶有b鍵就貢獻b(b−1)/2對，不是b−1對。

「最壞期望」在此指 $\max_{|S|=n} \mathbb{E}_h[\tau(S,h)]$。先固定任意 S，再對 h 取期望，通用界對每個 S 同時有效。它不是 $\mathbb{E}_h[\max_{|S|=n} \tau(S,h)]$：後者容許看過 h 才挑輸入，量詞次序改變。對非負 τ 及 $a>0$，還可用 Markov 得到 $\Pr(\tau \ge a) \le \mathbb{E}[\tau]/a$；小期望不表示每次選函數都沒有碰撞。

通用雜湊族的構造與偽隨機化反例

```
// p prime, keys are distinct vectors in F_p^2.
choose a,b independently and uniformly from 0..p-1
h(a,b,(x1,x2))=(a*x1+b*x2) mod p
// A random output translation of fixed g:
g_c(x)=(g(x)+c) mod m
```

對不同向量x,y，令δ=(x1−y1,x2−y2)≠(0,0)。若δ1≠0，固定任意b後，aδ1+bδ2=0在有限域中恰有一個a，因δ1有乘法反元素；故碰撞機率恰1/p。若δ1=0，則δ2≠0，同理由b解出唯一值。此證明包含a=b=0的函數，不能因它單次全碰撞就否定整個family的期望保證。

p=11，固定四鍵(0,0),(1,0),(0,1),(1,1)，共有6對；每對在121種(a,b)中恰11種碰撞，所以全表期望6/11。函數a=b=0使6對全碰撞；這是模型允許的低機率事件，期望仍按全部121函數計。

相反地，任意固定g再隨機選c平移輸出，g_c(x)=g_c(y)若且唯若g(x)=g(y)，原碰撞完全保留。例如g(x1,x2)=x1，鍵(3,1),(3,7)永遠碰撞，故不是通用族。把大整數先對p取模也可能先把不同鍵合成相同表示；若鍵域為0..p^2−1，應以兩位base-p向量作一對一編碼，而非只保留最低位。

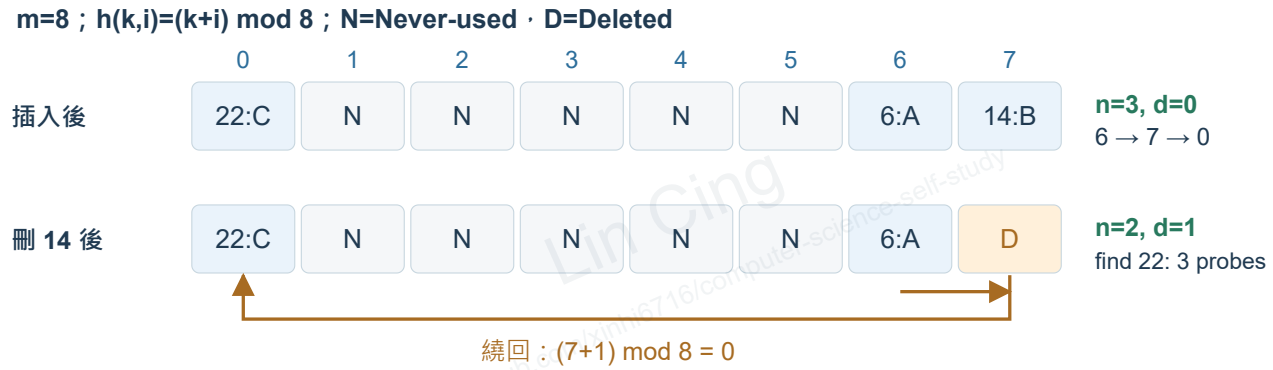
Open addressing 的槽位狀態

- 所有元素放在表內，每鍵依探測序列 `h(k,0),h(k,1),…` 尋找位置。通常要求 `n<m`。
- 槽位分為 Never-used、Occupied、Deleted，三種狀態語義不同。
- 搜尋遇到 Never-used 可停止，因插入時不會越過它卻把同鍵放後面。遇到 Deleted 必須繼續。
- 插入可記住首個 Deleted，但仍要繼續檢查後面是否已有同鍵，避免插入重複 key。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear probing：跨尾端的碰撞鏈

固定 m=8，$h(k,i) = (k+i) \bmod 8$。依序插入 (6,A)、(14,B)、(22,C)，三鍵的 home 都是槽 6。n 為活鍵數，d 為 Deleted 槽數；N 表示本次建表後未使用的 Never-used。



刪 14 只把槽 7 變 D。找 22 仍檢查槽 6、7、0，共 3 次；若誤把槽 7 變 N，就在第二次探測錯判不存在。D 不保留可讀的舊 pair，只保留「搜尋不能在此停下」的資訊。

Primary clustering：不同 home 也會延長同一段

另從刪除前的三鍵狀態開始，依序插入 8、1。下表只列鍵，N 表示尚未使用；兩次都不需擴容。

動作	0	1	2	3	4	5	6	7
初始	22	N	N	N	N	N	6	14
插 8：探 0→1	22	8	N	N	N	N	6	14
插 1：探 1→2	22	8	1	N	N	N	6	14

8 的 home 是 0，1 的 home 是 1，都不是原三鍵的 home=6，卻仍延長跨尾端的連續區 6→7→0→1→2。之後 home 落在這段內的新鍵，也必須掃到區段尾端以外。這就是 primary clustering：影響不只限於 home 完全相同的鍵。連續探測有局部性，但其機率行為不同於獨立的均勻隨機排列。

同一個 Deleted，更新與新增要做不同事

從刪 14 後的狀態開始，先 `insert_or_update(22,C*)`，再 `insert_or_update(30,Z)`。

更新 22 → C*

① 槽 6：key=6，不相等

② 槽 7：Deleted；記住候選 7

③ 槽 0：找到 key=22

更新槽 0；不占用候選槽 7

n=2，d=1；仍只有一筆 22

新增 30 → Z

① 槽 6：key=6，不相等

② 槽 7：Deleted；記住候選 7

③ 槽 0：key=22，不相等

④ 槽 1：Never-used，確認不存在

回寫槽 7；n=3，d=0

第一次在槽 0 更新 22，不使用候選槽 7。第二次探到槽 1 的 N 才確定 30 不存在，然後回寫最早的 D 槽 7。最後槽 0、6、7 分別為 22:C*、6:A、30:Z。

Never-used 的停止理由，Deleted 的繼續理由

不變量是：每個現存鍵從自己的 home 探到儲存位置之前，都不會先遇到 N。插入以第一個 N 作為不存在的證據；刪除 O→D 不製造新的 N；重用 D 也不改變其他鍵前方的 N。因此此不變量在這三種操作後仍成立。

錯誤改動	探測或寫入結果	破壞的契約
刪 14 後將槽 7 設 N	找 22 只查 6→7 就停止，漏掉槽 0	現存鍵的可達性
更新 22 時見 D 就立即寫槽 7	槽 7 與槽 0 都有 key=22	Dictionary 的鍵唯一性

`insert_new` 若要求呼叫端已證明鍵不存在，可省略查重；本章的 `insert_or_update` 沒有這項前提，必須找到同鍵、遇 N 或查滿 m 槽才作決定。演算法第 11 章把相同的停止狀態稱為 EMPTY；它與這裡的 Never-used 對應，兩科都不把 Deleted 當停止點。

Backshift deletion 是另一種刪除策略，必須按 home 與探測區間合法搬移，維持每個現存鍵的可達性；它不是把 tombstone 原地改成 N。

沒有 Never-used，仍要在一輪後結束

先單獨檢查固定容量的 `locate_or_slot`，暫停自動擴容與清理。依序插 6、14、22、1、2、3、4、5，填滿 8 槽；再刪除除 22 外的七鍵。此時 n=1、d=7，沒有 N。

查不存在的 38 時，home=6；依序檢查 6、7、0、1、2、3、4、5，記住第一個 D=6。八槽都沒有 38，便回傳 absent(6)，不能一直等待 N。若八槽全為 Occupied 且沒有同鍵，則回傳 full。

這個邊界狀態用來驗證探測程序。後文的自動維護政策會較早清理，不會讓表長期留在 d=7。**固定容量的合法狀態，不等於該維護政策容許保留的狀態。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同容量 rehash：重新定位，而非只清標記

沿用只剩 22 的固定容量狀態；新建同樣 8 槽的 N 表，只把 live pair 按新表的探測序列重插。

查 38：前 6→7→0→1→2→3→4→5（8 次）；後 6→7（2 次）

22 由槽 0 搬到槽 6，n 仍為 1，d 歸零。現在查 38 只探 6、7。若僅把舊 D 改 N 而保留 22 在槽 0，從 home=6 就會立刻錯停。清理減少了搜尋必須跨過的標記，但也花費掃表與重插入的成本；後文的清理門檻會把這些工作一起計帳。

Double hashing 與探測覆蓋

- $h(k,i)=(h_1(k)+i\cdot h_2(k)) \bmod m$。步長要與 m 互質，才會遍歷全部 m 個槽。
- 原例 m=8、步長 2，只能訪問同奇偶的 4 個槽，可能在表仍有空位時插入失敗。
- 選 m 為質數且令 $1\le h_2(k)<m$，可保證步長互質；也可用 2 的冪容量配奇數步長。
- 二次探測也需要指定係數、表大小與負載限制，不能只寫 $h+i^2$ 就保證找到所有空位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

查找或候選槽：先回傳判定，不立即寫入

```
locate_or_slot(key):
    first_deleted=none
    for i=0..m-1:
        j=probe(key,i)
        if state[j]==Occupied and equal(keys[j],key): return found(j)
        if state[j]==Deleted and first_deleted==none: first_deleted=j
        if state[j]==Never_used:
            return absent(first_deleted if first_deleted!=none else j)
    if first_deleted is set: return absent(first_deleted)
    return full
```

此程序最多 m 次探測，要求 probe 序列覆蓋全表。只有 Occupied 才讀取 keys[j]；Deleted 沒有 live pair，不得解參照已銷毀的鍵。回傳 found、absent(slot) 或 full，讓呼叫端分開處理更新、插入與容量不足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自動插入與刪除：重建後必須重找位置

```
insert_or_update(k,v):
    result=locate_or_slot(k)
    if result is found(j): values[j]=v; return
    if n+1>3*m/4: rehash(2*m); result=locate_or_slot(k)
    else if deleted>m/8: rehash(m); result=locate_or_slot(k)
    j=result.slot
    if state[j]==Deleted: deleted-=1
    construct pair(k,v) at j; state[j]=Occupied; n+=1

erase(k):
    if locate_or_slot(k) is found(j):
        destroy item; state[j]=Deleted; n-=1; deleted+=1
        if m>minimum_capacity and n<m/4:
            rehash(max(minimum_capacity,m//2))
```

先處理同鍵更新，再測擴容與清理門檻。rehash 可能改變位置，所以之後重新呼叫 locate_or_slot；不能把舊槽索引帶進新表。以上 8 槽的更新／新增例 d=1≤m/8、live 負載也未超門檻，不會額外重建。原 7 槽練習則明定固定容量、不套這層政策。

探測期望公式的適用模型

- 理想 uniform hashing 假設每鍵的探測排列均勻，且獨立於先前建表歷史。沒有 tombstone 且 $\alpha<1$ 時，不成功搜尋期望至多 $1/(1-\alpha)$。
- 同模型，鍵插入後未刪除，且成功查詢等機率選取現存鍵時，$\alpha>0$ 的期望可界為 $(1/\alpha)\ \ln(1/(1-\alpha))$。
- α=0.9 時，前一個界為 10，而非不隨負載改變的神奇常數。α 趨近 1，成本界發散。
- 此頁不是 linear probing 的專屬精確公式，也未包含 tombstone、hash 計算及長字串相等比較成本。

理想探測上界的推導

- 假設尚有 m−n 個 Never-used 槽、沒有 tombstone，且每鍵探測序列是獨立於已佔槽集合的均勻隨機排列。前 i 次都命中已占槽的機率為 $(n)_i/(m)_i \le \alpha^i$，其中 $(n)_i$ 是下降階乘。
- 以尾和公式，探測數 X 的期望 $E[X]=\Sigma_{i\ge 0}Pr(X>i)\le \Sigma \alpha^i=1/(1-\alpha)$。沒有 tombstone 與 α<1 都參與這個推導。
- 成功查詢等機率選 n 個插入且未刪的鍵，對插入時負載 j/m 取平均。因 $1/(1-x)$ 遞增，左矩形和小於積分，故 $E[X]\le(1/n)\Sigma_{j=0}^{n-1}1/(1-j/m)\le(1/\alpha)\ln(1/(1-\alpha))$。
- 此證明展示理想機率模型如何進入數學式。對線性探測的連續 cluster，均勻隨機排列前提不成立。

擴容與攤銷不能省略重算 hash

- 表大小改變時，hash 到槽位的映射也改變，須把現存鍵重新插入。不能只把原陣列複製到較大空間。
- 負載率達固定門檻時倍增，當 m=Θ(n)，掃描與配置為 Θ(n)；重新插入在適當隨機假設、負載遠離 1 時為期望 Θ(n)。任意碰撞下不能一概保證 rehash 最壞線性。這些條件下，連續插入才有期望攤銷 $O(1)$ 成本。
- 這同時使用機率假設與跨序列計帳，應明寫「期望攤銷」，不能只稱最壞常數。
- 若縮容門檻與擴容門檻相同，邊界附近交替插入刪除可能反覆重建。不同門檻能避免此震盪。

重新雜湊的鍵重定位與期望分析

```
rehash(new_m):
    allocate new_m Never_used slots
    choose compatible new hash parameters if required
    for each old Occupied pair (k,v):
        insert into new table using its new probe sequence
    commit new table; deleted=0; preserve n; release old table
```

以m=7的鍵10,17,24,8為例，h=k mod7時桶3=[10,17,24]、桶1=[8]。改成m=14、h=k mod14，新桶10=[10,24]、桶3=[17]、桶8=[8]；不能把原桶位置保留。開放定址同樣要按新probe重新插入，Deleted全數清除。

以固定不成功查詢鍵 $x \notin S$ 與固定儲存集合 S、$|S| = n$ 作通用雜湊分析：$X = \sum_{y \in S} \mathbf{1}[h(y) = h(x)]$，$\mathbb{E}[X] = \sum_{y \in S} \Pr(h(y) = h(x)) \le n/m$。這裡只需線性期望，不需各碰撞事件互相獨立；但 x, S 需在選函數前固定，適應性攻擊者不是同一模型。若 $x \in S$，自碰撞項必為 1，整個桶長期望的相應上界是 $1 + (n-1)/m$，不能沿用不成功查詢的界。成功查詢若分布集中在最長鏈，也不能把均勻查詢平均當成該分布保證。

重新雜湊的鍵重定位與期望分析 （續）

重建掃描Θ(m+n)；m=Θ(n)且新表負載有固定上限與適當隨機模型時，重插入期望Θ(n)。若hash全碰撞且insert必查重，重插入可能二次；「rehash最壞線性」不可無條件使用。維持擴容3/4、縮容1/4並清理tombstone，才能結合期望與攤銷而得更新序列成本。

清理門檻的反例與攤銷記帳

```
// m starts at a power-of-two minimum_capacity>=8.
// Before inserting a genuinely absent key:
if n+1>3*m/4: rehash(2*m)
else if deleted>m/8: rehash(m)
// After deleting a present key:
if m>minimum_capacity and n<m/4:
    rehash(max(minimum_capacity,m//2))
// Rehash resets deleted=0 and reinserts all n live entries.
```

擴縮容量與清 tombstone 是兩種不同的觸發原因。錯法「n+deleted+1>3m/4就同容量重建」會在 n=3m/4 時出問題：刪一鍵後 n=3m/4−1、deleted=1，再插回原鍵，雖可重用那個 Deleted 槽，錯門檻卻立即重建。m=128、96鍵反覆刪95再插95，共500對操作，舊規則重建500次、重插47,500筆，不能用常數個操作支付每次Θ(m)工作。

清理門檻的反例與攤銷記帳 （續）

修正後只在 deleted>m/8 時做同容量清理；每次至少累積Ω(m)次刪除，才能支付重建。插入時先查同鍵，更新既有value不觸發容量變化；真正新鍵先測live負載3/4，再測deleted門檻。每次插入完成，n+deleted≤7m/8：若不重建，n≤3m/4且deleted≤m/8；若重建，deleted歸零；重用Deleted不增加已使用槽數。刪除只把Occupied換Deleted，總使用數不變，故任意後續查詢仍至少有m/8個Never-used槽。

容量倍增後live負載約3/8，縮半後至多1/2；下一次相反方向的容量調整需要Ω(m)次live增減，不會一刪一插震盪。同容量清理另由其累積刪除數記帳。各重建在明示的均勻探測／相容亂數模型下為期望Θ(m+n)，可分別攤到觸發操作；因此總更新有期望攤銷O(1)探測成本，最壞碰撞仍可線性或使重插二次。雙重雜湊在2冪容量需奇數步長；此條件只保證遍歷，不等於理想隨機排列假設。

溢位處理與工程條件

- chaining 的 overflow 是桶內元素增加；open addressing 的滿表必須偵測，不能無限探測。
- 若 key 是長度 L 的字串，第一次計算 hash 至少可能需 $\Theta(L)$。只把槽位探測數當總時間會漏掉鍵成本。
- 若查詢者能控制大量碰撞鍵，均勻分布假設可能不成立。可用隨機 seed、合適的 keyed hashing 或不同容器。
- 若需求含排序、區間查詢或前驅查詢，平衡搜尋樹通常更直接，因雜湊表不保留鍵次序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考文獻與延伸閱讀

- 假設與期望成本核對 MIT 6.006 2008 L5–L7。
- C++ 無序容器的平均與最壞成本、hash/equality 要求，核對 unordered associative requirements。不能將標準平均保證說成每次最壞常數。
- MIT 6.006 2014 Recitation 10, § 1.1：Deleted 的搜尋語意及重複鍵更新。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前置知識與目標

- 前置知識為 binary heap、鏈結樹、歸納證明、對數與攤銷分析。
- 能由 meld（合併）需求推導不同堆積結構，追蹤 leftist、binomial、Fibonacci 及標準 two-pass pairing heap。
- 能區分真正執行步數、最壞界與攤銷界，並明訂 decrease-key 是否已有節點 handle。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mergeable priority queue 的契約

- 堆積狀態是含重複鍵的多重集合。meld 合併兩堆所有元素，保留各元素身分。
- 本章以 destructive meld 為主，合併後來源堆不再擁有節點。若要求保留兩份可獨立修改的來源，成本不同。
- decrease-key 先取得合法節點 handle，再降低鍵值。搜尋任意鍵的成本不能藏在 `O(1)` 或 `O(log n)` 之內。
- 多個相同最小鍵時可回傳其中任一個，若要穩定順序，另以插入序號組成複合鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Leftist heap：刻意讓合併走短路

將兩個 binary heap 的陣列串接後重新建堆，要花線性時間。Leftist heap 改用鏈結二元樹：**不要求整棵樹矮，只要求反覆往右走，很快就遇到空指標。** meld 每層只走其中一棵樹的右子樹，這正是要縮短的路。

令空樹的 rank 為 0，非空節點滿足

$$r(x) = 1 + \min\{r(x.left), r(x.right)\}.$$

葉的 rank 為 1；只有左子的節點仍為 1。rank 衡量通往空子位置的最短路徑，**不是高度，也不是子節點數。**

Leftist heap 同時要求父鍵不大於子鍵，以及 $r(left) \geq r(right)$。因此 $r(x) = 1 + r(x.right)$，rank 恰好等於右鏈上的非空節點數。只向左延伸的 n 點鏈是合法形狀，rank=1，高度卻為 n−1。

Meld：先留下小根，回程再調整左右

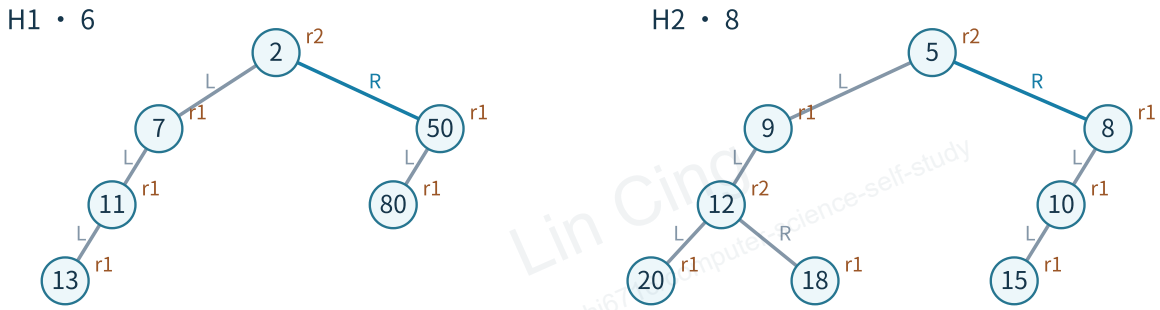
兩棵輸入須為合法、節點互不重疊的堆。下列採 destructive meld；相等鍵保留第一個根。節點記錄含 key、left、right、rank，不含 parent。

```
rank(null) = 0
meld(a, b):
    if a == null: return b
    if b == null: return a
    if a.key > b.key: swap(a, b)
    a.right = meld(a.right, b)
    if rank(a.left) < rank(a.right):
        swap(a.left, a.right)
    a.rank = 1 + rank(a.right)
    return a
```

下去時比較根；回來時比較 rank。 小根 a 的左子樹先保留，右子樹交給遞迴合併。回傳後才知道新右子樹的 rank，這時才決定是否交換子指標。swap 不交換 key 或 payload。

合併輸入：14 個節點，兩條待走的右鏈

節點內是 key，旁邊 r 是 rank；L、R 指明子指標。缺邊代表 null，rank=0。藍線標出原右鏈，其他邊不是本次遞迴向下走的路。



H1 有 6 點，右鏈 2→50；H2 有 8 點，右鏈 5→8，兩根 rank 都是 2。9 的左子樹可以很深，它的 rank 仍只有 1，因為右子指標為空。

遞迴向下：先不急著交換子樹

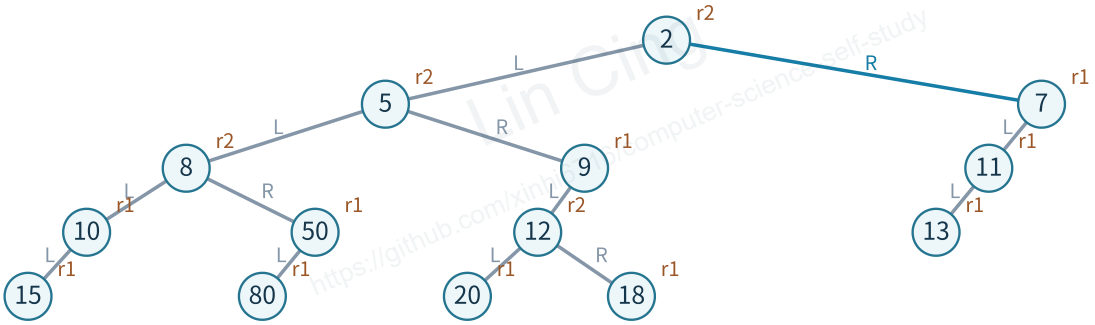
每次保留較小根，只把它的右子樹送入下一層。回傳前，尚未更新保留根的 rank。

呼叫	根比較與保留者	保留的左子樹	下一個呼叫
meld(2,5)	2<5，保留 2	根 7	meld(50,5)
meld(50,5)	5<50，交換局部根後保留 5	根 9	meld(8,50)
meld(8,50)	8<50，保留 8	根 10	meld(null,50)
meld(null,50)	空樹基例，不比較 key	—	回傳根 50

只做 **3 次根鍵比較**。2、5、8 的呼叫仍未結束；不能把暫時接上的 min-heap 當成已修復的 leftist heap。接下來依 8→5→2 的順序回傳。

回傳修復：為什麼要在 5 與 2 交換

回到的根	接回後的左、右根	左、右 rank	動作與新 rank
8	10、50	1、1	不交換，r(8)=2
5	9、8	1、2	交換成左 8、右 9，r(5)=2
2	7、5	1、2	交換成左 5、右 7，r(2)=2



為什麼 meld 的結果一定正確

對兩棵輸入的總節點數做歸納。基例有一棵為空，回傳另一棵，節點集合與兩項不變量均正確。皆非空時選較小根 a，遞迴只處理 a 的右子樹與另一堆，總節點數至少少一。

1. **元素不丟失、不重複。** 互不重疊的輸入分成 a、原左子樹與遞迴輸入。歸納假設保留遞迴中的每個節點；接回與交換子指標不增刪節點。
2. **Heap order 保持。** a 不大於原左右子樹內任何鍵，也不大於另一堆的根及其後代。因此 a 不大於新子根；子樹內依歸納假設合法。左右交換不改這些鍵比較。
3. **Leftist 性質恢復。** 兩棵子樹已合法，將 rank 較大者放左；設定 $r(a) = 1 + r(a.right)$ 就等於定義式。

總節點數嚴格減少也保證終止。meld(H,H) 違反節點互不重疊的前提，可能產生自環；必須在堆的所有權介面排除，不能把它當成複製堆。

最壞成本：只走兩棵輸入的原右鏈

先證 rank 界。若子樹 rank 至少 r，兩子樹 rank 都至少 r−1。令 N(r) 是 rank 至少 r 所需的最少節點數；N(0)=0，利用歸納假設得

$$N(r) \geq 1 + 2N(r-1) \geq 1 + 2(2^{r-1} - 1) = 2^r - 1.$$

所以含 n 點的堆滿足 $r \leq \lfloor \log_2(n+1) \rfloor$。基例 r=0 成立。

遞迴下降時還沒做回程交換。每一步只把其中一根換成它的**原右子樹**，兩個當前 rank 的和恰減 1。兩者皆非空時和至少為 2；輸入 rank 為 r1、r2 時，非空層至多 r1+r2−1，另加一次空樹基例。

每層 O(1) 工作，故時間與遞迴堆疊皆為 $O(1 + \log(n_1 + 1) + \log(n_2 + 1))$，是**最壞界**。14 點數例的 rank 和為 4，恰做 3 次根比較；一般也可能提早結束。

刪最小：刪根後，左右子堆重新 meld

延續 14 點結果，移除根 2，合併根 5 與根 7。空堆先拒絕刪除；單點堆移除後回傳 null。

呼叫／回程	決策
meld(5,7)	保留 5，左子樹 8 不動，遞迴 meld(9,7)
meld(9,7)	保留 7，左子樹 11 不動，遞迴 meld(null,9)
meld(null,9)	回傳根 9，連同所有後代
回到 7	左 11、右 9 的 rank 皆 1，不交換，r(7)=2
回到 5	左 8、右 7 的 rank 皆 2，不交換，r(5)=3

結果為 `5:(8,7)`、`8:(10,50)`、`7:(11,9)`，其他邊保留。新右鏈 5→7→9，min=5，共 13 點。**刪除後 rank 反而增加並不矛盾**：rank 衡量形狀，並非隨元素個數單調變化。

插入與記號：別把不同的 rank 混在一起

insert 先建 `left=right=null, rank=1` 的單點，再 meld。向上一頁的 13 點堆插入新鍵 1：保留 1，meld(null,5) 回傳原堆；新根左 rank=0、右 rank=3，交換後原堆接在左側，新根 rank=1。結果 14 點，兩次呼叫、一次鍵比較，不需逐點搜尋原堆。

insert、delete-min 最壞皆為 $O(1+\log(n+1))$，find-min 看非空根為 O(1)。此處不計任意精度鍵比較與記憶體配置成本。

本章採 **null rank=0、leaf rank=1**。若另採 null npl=−1、leaf npl=0，對同一位置有 $npl = r - 1$；右鏈非空節點數為 npl+1，子樹至少有 $2^{npl+1} - 1$ 點。公式與基例須整套轉換。

這不同於 binomial／Fibonacci 的 degree（直接子節點數），或 splay 分析的 $\log_2 size$。它們衡量不同的量，不能一律加減 1 換算，也不能把右鏈界套成整樹高度界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二項樹 B_k 的結構

- B_0 為單點。B_k 由兩棵 B_{k-1}，把其中一根連成另一根的子節點形成。
- 歸納得節點數 2^k、高度 k、根度數 k，根的子樹分別為 $B_0,\dots,B_{k-1}$。
- binomial heap 保留一組符合 heap order 的二項樹，每個 degree 最多一棵。
- n 的二進位表示決定有哪些 degree。例如 n=13=1101_2，包含 B_0,B_2,B_3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial meld 是二進位進位

- 先依 degree 合併兩個根清單，遇到相同 degree 的兩樹就比較根，把較大根連到較小根底下。
- 兩棵 B_k 形成 B_{k+1}，相當於同位元相加產生進位。遇到三棵同 degree 時保留一棵，再把另外兩棵 link。
- 根清單最多 $\lfloor \log_2 n \rfloor + 1$ 棵，標準 eager consolidation 的 meld 最壞 $O(\log n)$。
- 例如 $5 = 101_2$ 與 $3 = 011_2$ 相加，degree 0、1、2 連續進位，最後只剩 B_3，共 8 點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial 的 link 與專用插入

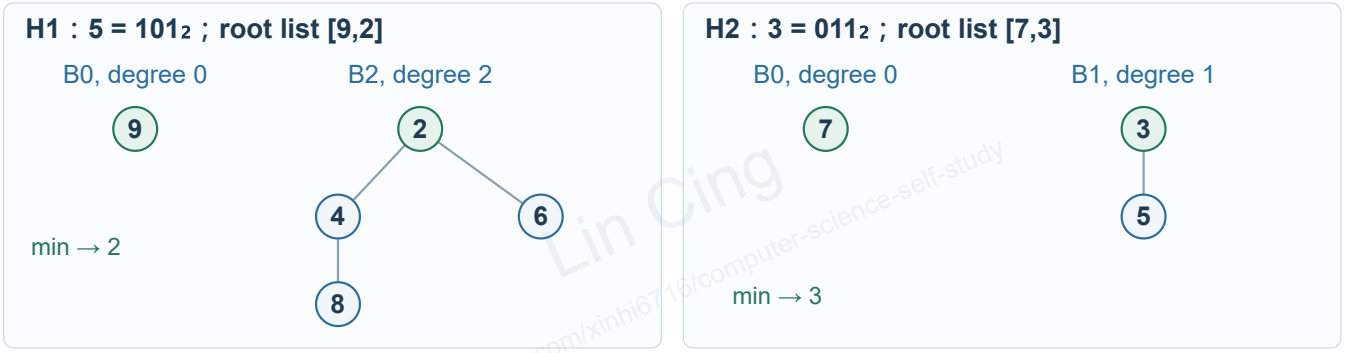
```
binomial_link(a,b):
    require degree(a)==degree(b)
    if a.key>b.key: swap(a,b)
    prepend b to a.child list; b.parent=a
    a.degree+=1; return a
binomial_insert(x):
    carry=singleton(x); d=0
    while tree[d] exists: carry=link(carry,tree[d]); clear tree[d]; d+=1
    tree[d]=carry; update min pointer
```

`swap(a,b)` 只交換局部根指標，不交換節點的鍵與 payload。link 的兩棵輸入都為 Bk，勝根原子 degree 為 k−1,⋯,0；把敗根 Bk 前插，便得到 k,k−1,⋯,0，正是 B(k+1) 的子樹組成。較小根留下，也維持新邊的 heap order。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial meld：輸入森林與串列順序

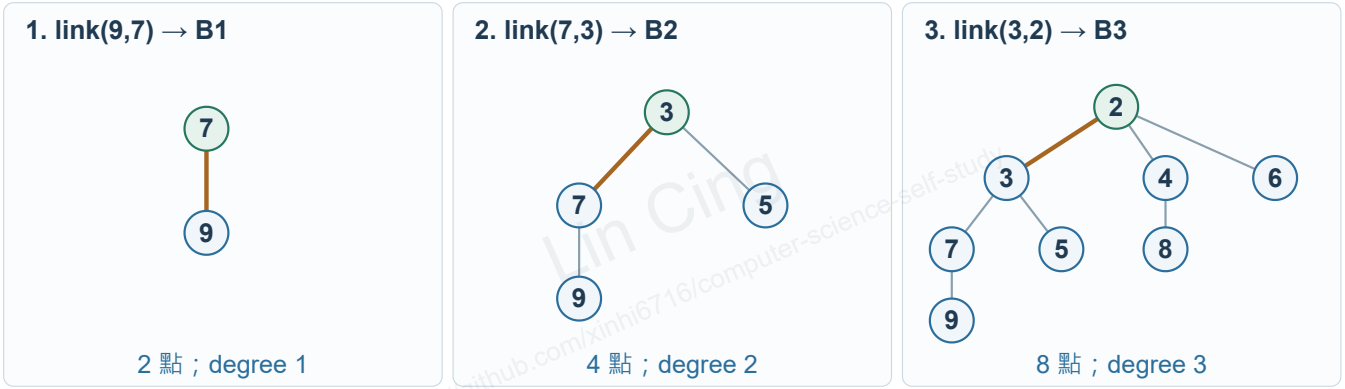
本例所有鍵互異。記法 `x:[子根…]` 按子串列順序列出，未列者為葉。H1 為 `9:[]`、`2:[4,6]`、`4:[8]`；H2 為 `7:[]`、`3:[5]`。根串列依 degree 遞增，子串列依 degree 遞減；link 把敗根連成勝根的**第一個子節點**。



H1 有 1+4=5 點，H2 有 1+2=3 點。根串列第一個根未必最小；H1 的最小鍵為 2，而非 9。

Binomial meld：三次比較與子樹合併

先 link 兩棵 B0 的根 9、7，再與根 3 的 B1 合併，最後與根 2 的 B2 合併。每次只比較兩個根；較大根原有的全部後代一併移動。



棕色邊是該次新增的 parent→child；其他連線保留。最後 2 的子根依序是 3、4、6，degree 分別為 2、1、0。八個鍵各出現一次，沒有搬換 payload，也沒有遺失敗根的子樹。

Binomial meld：每一個 degree 留下什麼？

步驟	本次 link	尚存根：key(degree)	根數
輸入	無	H1: 9(0),2(2)；H2: 7(0),3(1)	4
degree 0	9 與 7 → 7	7(1),3(1),2(2)	3
degree 1	7 與 3 → 3	3(2),2(2)	2
degree 2	3 與 2 → 2	2(3)	1

一次 link 少一棵樹、不減節點；根數從 4 降到 1，恰好做 3 次 link。它同時保持兩個不變量：相同 degree 的 B_k 合成 B_{k+1}，節點數為 $2^k + 2^k = 2^{k+1}$；較小根留下，所以所有新舊邊都符合 min-heap order。最後只剩 degree 3，也吻合 5+3=8=1000_2。

這是本例的操作次數，不能推出一般 meld 為 O(1)。一般演算法還要走訪來源根串列，合法 binomial heaps 的根數為 O(log n)；Fibonacci heap 的未整併根數可能是 Θ(n)，不能直接套用。

三棵同 degree，為什麼不能全部 link 成一棵？

以 7=111_2 加 3=011_2 為例；每欄最多有兩個來源根和一個低位進位。

degree	H1 的根數	H2 的根數	低位進位	此位保留	送往下一位
0	1	1	0	0	1
1	1	1	1	1	1
2	1	0	1	0	1
3	0	0	1	1	0

degree 1 的三棵 B1 共 6 點，必須保留一棵 B1，再把兩棵 link 成 B2；6 不是 2 的冪，不能把三棵當成一棵二項樹。結果保留 B1 與 B3，節點數 2+8=10，對應 1010_2。保留哪一棵可能改變最終樹形；合法結果不必唯一，但 degree、節點集合與 heap order 都必須正確。

Binomial 其餘操作的條件

- 專用 insert 把單點 B_0 加入並沿低 degree 進位，進位停止即結束，最壞 $O(\log n)$。以進位次數計帳，可得攤銷 $O(1)$；若每次都呼叫完整掃根清單的 meld，不能直接沿用此插入攤銷界。
- delete-min 先找最小根，移除該根，把其子樹清單按 degree 反轉後 meld，最壞 $O(\log n)$。
- find-min 若每次掃根，為 $O(\log n)$；額外維護最小根指標後可為 $O(1)$。
- decrease-key 可沿父鏈交換內容，最壞 $O(\log n)$。若 handle 指向節點身分，需同步處理內容交換對 handle 語義的影響。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial meld：以 degree 表處理進位

```
meld_roots(R1,R2):
    by_degree=empty
    for x in concatenation of detached root snapshots R1,R2:
        while by_degree[x.degree] contains y:
            clear by_degree[x.degree]
            x=binomial_link(x,y)
        by_degree[x.degree]=x
    return roots in degree order; recompute minimum pointer
```

每個 snapshot 中的根只作為輸入一次。碰撞時先清空舊 degree 格，再 link 並以新 degree 重試；每次少一個根，所以總 link 數不超過輸入根數減一。合法來源森林只有 O(log(n1+n2)) 個根，故走訪與進位共 O(log(n1+n2))。這個根數界不能套到未整併的 Fibonacci heap。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

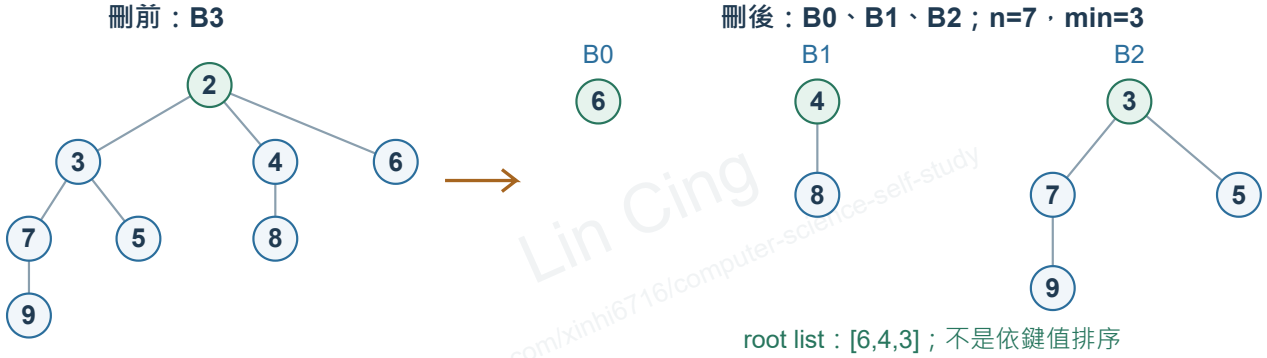
Binomial 刪根與降鍵：恢復 parent 和 min

```
delete_min():
    require roots not empty
    z=minimum root
    remove z from roots
    children=detach z.children; clear their parent pointers
    release z
    roots=meld_roots(roots,children)
decrease_key(x,new):
    require new<=x.key; x.key=new
    while x.parent!=null and x.key<x.parent.key:
        swap complete key-value contents and update handle mapping
        x=x.parent
    update minimum pointer
```

刪根先摘出子樹、清其 parent，再釋放舊根；meld 最後重算 min。降鍵可交換完整 key-value 內容，但若 handle 指向邏輯元素，handle→node 對應也要隨內容更新，不能讓下次操作找錯元素。

刪最小值：從同一棵 B3 拆回森林

刪除上一個合併結果的根 2，保留其三棵完整子樹。原子串列依 degree 為 2→1→0；改成根串列時反轉為 0→1→2，並把三個新根的 parent 設為 null。



新的根串列為 [6,4,3]，其樹分別有 1、2、4 點，總數 7=111₂。此例沒有其他根，degree 已互異，不需再 link；掃描三個根重設 min=3。反轉的理由是**degree 順序**，不是鍵值排序；degree-table 版可不反轉輸入，但輸出根串列仍依本章契約排列。

Fibonacci heap 延後整併

- Fibonacci heap 保留 heap-ordered forest、根循環雙向串列、最小根指標、每點 degree 及 mark。
- insert 只把單點加進根串列；meld 拼接根串列並更新最小根。這兩項不立即把相同 degree 整併。
- delete-min 將其子節點提升成根，再以 degree 表連結同 degree 根，最後重建最小根指標。
- 節點可失去部分子節點，所以其樹不是固定形狀的二項樹。degree 上界需要 cascading cuts 維持。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decrease-key 與 cascading cuts

1. 降低 x.key。若仍不小於父鍵，不需切斷。
2. 若破壞 heap order，把 x 從父 p 切下，加到根串列，清除 x.mark。
3. 若 p 不是根，第一次失子把 p 標記；若它已被標記，再把 p 切下並向上處理。

非根節點在成為別人的子節點後，至多失去一個子節點而仍留在原父底下。根不需要因失子而作同樣標記。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heap 的整併與標記操作

```
link_equal_degree_roots(a,b):
    require a.degree==b.degree
    if a.key>b.key: swap(a,b)
    roots=ring_remove(roots,b)
    a.child=ring_prepend(a.child,b)
    b.parent=a; b.mark=false; a.degree+=1
    return a
cut(x,p):
    p.child=ring_remove(p.child,x); p.degree-=1
    x.parent=null; x.mark=false; roots=ring_prepend(roots,x)
cascade(p):
    g=p.parent
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heap 的整併與標記操作 （續）

```
    if g==null: return
    if not p.mark: p.mark=true
    else: cut(p,g); cascade(g)
decrease_key(x,new):
    require new<=x.key; x.key=new
    p=x.parent
    if p!=null and x.key<p.key: cut(x,p); cascade(p)
    if x.key<min.key: min=x
delete_min():
    require min!=null
    return extract_root(min)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整併前若四個singleton根鍵為[7,3,8,5]，依此順序處理：7占degree0；3遇7，link成3→7、degree1；8占degree0；5遇8，成5→8、degree1，再遇3，成根3，子7及5→8，degree2。此表內最多一根每degree；必須以roots快照或安全迭代避免link時修改正在走訪的串列。

Fibonacci heap 的整併與標記操作 （續）

連鎖切斷用合法狀態中已標記的非根p、g示範：根r=2，其下g=3，其下p=5，其下x=7，g.mark=p.mark=true。降x到1，依序cut x、p、g，三者成根，r保留。t由1變4、m由2變0，Φ=t+2m由5降4，ΔΦ=−1；實際三次cut的固定成本以縮放勢能支付，端點常數留在攤銷O(1)。此例只展示標記記帳；degree大小的一般證明仍用後續連入順序與最多失一子的歸納，不能由四點圖取代。

insert只加未標記singleton根並更新min；meld拼接兩個根雙向環且來源失去所有權。空delete-min必拒絕，最後一節點被移除後min=null。任意節點刪除採後續的erase(handle)與extract_root完整程序，不修改鍵值、不指定哨兵最小鍵。即使目標大於父鍵，直接刪除造成的失子仍要觸發cascade，才能保留degree證明的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向環與共用整併的完整修復

```
ring_remove(head,x):
    if x.right==x: head=null
    else:
        x.left.right=x.right; x.right.left=x.left
        if head==x: head=x.right
    x.left=x.right=x; return head
ring_prepend(head,x):
    require x.left==x and x.right==x
    if head!=null:
        x.left=head.left; x.right=head
        head.left.right=x; head.left=x
    return x
snapshot_of_root_ring():
    S=[]
    if roots==null: return S
    x=roots
    repeat: S.append(x); x=x.right
    until x==roots
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向環與共用整併的完整修復 （續）

```
    return S
consolidate():
    A=[]                        // missing degree entries mean null
    for x in snapshot_of_root_ring():
        if x.parent!=null: continue
        while len(A)<=x.degree: A.append(null)
        while A[x.degree]!=null:
            y=A[x.degree]; A[x.degree]=null
            x=link_equal_degree_roots(x,y)
            while len(A)<=x.degree: A.append(null)
        A[x.degree]=x
    roots=null; min=null
    for x in nonnull entries of A:
        x.left=x.right=x; x.parent=null; x.mark=false
        roots=ring_prepend(roots,x)
        if min==null or x.key<min.key: min=x
```

ring_remove的前提是x確實在head指定的環內；單點環回傳null，多點環接回左右鄰居，若摘掉代表點便改用其右鄰。摘下後x自成單點環，才能交給ring_prepend。這兩個操作只改固定數目指標，不搜尋前驅。它們同步用於根環與子環，因此cut、提升與link都有可重建的連結步驟。

雙向環與共用整併的完整修復 （續）

整併先保存根節點指標快照，避免link改環時破壞目前走訪。相同degree的根比較鍵，較大根自根環摘下、接到較小根子環、清mark並增加父degree；這就是前頁的link_equal_degree_roots。先清A舊格，再按新degree反覆進位，直到該格為空。A中每個非空格最後恰保留一棵合法堆序樹。

最後不再沿舊根環走訪，而依A逐根重建環及min；不能釋放z後讀舊min.key，也不能因z不是舊min便省略新根的比較。degree表按需補空格，總長O(1+log(n+1))；以t′表示cut/cascade後根數、d表示目標degree，本快照版另用O(t′+d)暫存指標，最壞O(n)，沒有宣稱額外空間只有degree表。單次實際工作同樣可能Θ(n)，攤銷時間與當下空間是不同指標。

Fibonacci degree 上界的理由

- 將 x 仍保留的 k 個子節點按連入時間排序。第 i 個連入時，x 至少已有 i−1 個子，因此該子當時 degree 至少 i−1。
- 它留作 x 的子時至多失去一子，所以目前 degree 至少 $\max(0,i-2)$。
- 用最小子樹大小遞迴可得 degree k 的節點至少有 F_{k+2} 點，故 $k=O(\log n)$。
- 這個下界連結「允許失子數」與「degree 不會太大」。省略 cascading cuts 就不能沿用此證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從子節點限制到 Fibonacci 下界

- 令 S(k) 為 degree 至少 k 節點的最小子樹點數，包含自身。這個定義使 S(k) 隨 k 不減，才能將子節點 degree 的下界代入。S(0)=1，S(1)≥2。
- degree 至少 k 的節點，取前 k 個保留子節點，按連入次序第 i 子的 degree 至少 $\max(0,i-2)$，所以對 $k\geq 2$ 有 $S(k)\geq 1+2S(0)+\Sigma_{j=1}^{k-2}S(j)$。
- 以歸納假設 $S(j)\geq F_{j+2}$ 代入，得到 $S(k)\geq 3+\Sigma_{j=1}^{k-2}F_{j+2}=F_{k+2}$，最後等式來自 Fibonacci 前綴和。
- 因 $F_{k+2}\geq \phi^k$、$\phi=(1+\sqrt{5})/2>1$，n 點堆中的任何 degree 皆滿足 $k\leq \log_\phi n$。這才是整併 degree 表只需 O(log n) 格的依據。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

勢能法把延後工作入帳

- 設 `Φ=t+2m`，t 為根數，m 為被標記的非根節點數。攤銷成本為 `ĉ=c+Φ_after−Φ_before`。
- insert 增一個根，只增加常數勢能。meld 的勢能可加，因此維持常數攤銷成本。
- decrease-key 若切 c 個已標記祖先，每次切斷增一根但清一標記，勢能約下降 1，支付連續切斷的實際成本。
- 實作中每次 link/cut 的固定指標成本，可把 Φ 整體乘足夠常數支付。若從空堆開始且 `Φ≥0`，總實際成本不超過總攤銷成本加上初始勢能。這是序列最壞保證，不是隨機平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci 的共同勢能與完整攤銷界

Fibonacci decrease-key若共切k≥1點，根數增加k。除了首切以外，每個繼續被切的祖先原來都已標記，至多最後停止祖先新增一個mark，所以 Δm≤1−(k−1)=2−k，Δ(t+2m)≤4−k。若實際成本≤ak+b，取縮放勢能 A(t+2m)、A≥a，攤銷成本≤ak+b+A(4−k)≤b+4A，是與k無關的常數。完全不切時只比較／更新鍵與最小指標，也為常數。

刪最小根前t根、該根d子，degree上限D；實際成本≤a(t+d+D)+b。提升子、link及重設根mark不增加標記數，整併後根數≤D+1，故 ΔΦ≤A(D+1−t)。取A≥a後，攤銷成本
≤a(d+D)+A(D+1)+b=O(D+1)=O(1+log(n+1))，使用d≤D。這補足「勢能支付掃根」的代數，不把單次Θ(n)工作說成最壞對數。

- Fibonacci heap 使 Dijkstra 的抽象成本達 `O(E+V log V)`，前提為正確使用 handle 與 decrease-key，且圖權重符合 Dijkstra 條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不修改鍵值的 erase 與任意根移除

```
erase(h):
    z=resolve_live_owned_handle(h)   // reject stale/foreign handle first
    if z.parent!=null:
        p=z.parent; cut(z,p); cascade(p)
    return extract_root(z)
extract_root(z):
    require z is a live root owned by this heap
    result=copy(z.item)             // fixed-size nonthrowing model
    while z.child!=null:
        c=z.child
        z.child=ring_remove(z.child,c); z.degree-=1
        c.parent=null; c.mark=false; roots=ring_prepend(roots,c)
    roots=ring_remove(roots,z); min=null
    invalidate_handle(z); release(z); n-=1
    if n==0: roots=null; return result
    consolidate()
    return result
```


不修改鍵值的 erase 與任意根移除 （續）

每個節點保存key、item、parent、child、degree、mark與雙向環的left/right；roots、child分別是根環與子環的代表指標，空環為null。handle指向特定活節點，不是「目前等於某鍵的任一節點」。erase只使目標handle失效；即使兩節點鍵相同，也不搬換其餘節點的item或身分。外部不透明handle必先經所屬容器的活節點登錄表驗證，再取得節點；不能先解參照已釋放raw pointer檢查alive。本節成本以已持有合法handle為前提，額外的查找／驗證成本另計。

非根目標z先切成根，並把原父p失去一子的事件交給既有cascade：p非根且未標記則標記；已標記則切p並繼續到祖父；到根即停止。這次cut不是因鍵變小，而是因即將刪除節點；省略cascade會允許非根連失兩子，破壞Fibonacci degree下界。原本已是根的z直接進extract_root，不因刪掉根而任意標記其子。

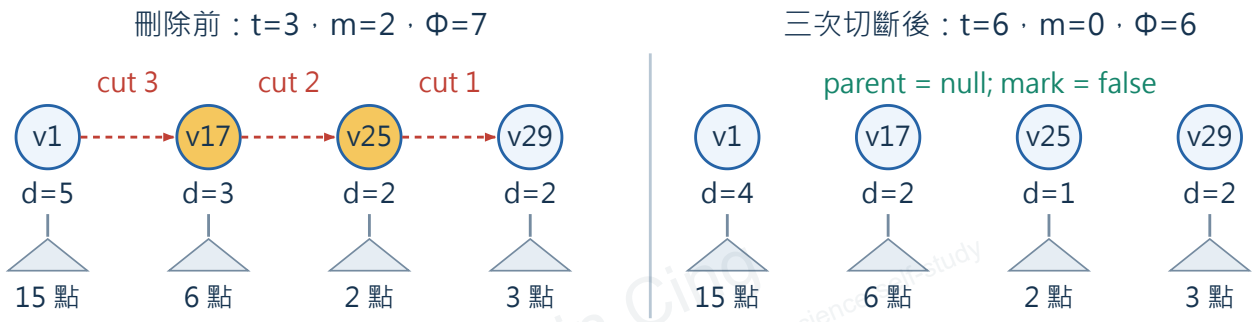
extract_root不要求z=min。逐一先從z子環摘下c、減degree、清parent/mark，再接根環；整棵c子樹保留。移除z後先清min以避免保留懸空最小指標，再使其handle失效及釋放，僅減n一次。仍有節點就整併全部根並重算真正min，不把目標假裝成最小鍵。空堆delete_min先拒絕；空堆沒有合法erase handle。刪最後一點時子環必空，n=0、roots=min=null；再次使用舊handle須拒絕。meld後可接受哪些handle由容器的所有權契約決定，不能繼續對失去所有權的來源堆刪除。

Erase 的合法起始狀態與三級切斷

```
for i=0..32: h[i]=insert(i)     // v_i is permanent identity, initially key i
delete_min()                    // removes v_0; consolidates to root v_1
decrease_key(h[18],-10)
decrease_key(h[26],-11)
erase(h[29])                    // key stays 29 until release
```

根與子環都以前插建立，快照沿right走訪。前33次插入後刪v0，得到degree5的根v1。其子v17原有子v25,v21,v19,v18；v25原有子v29,v27,v26；v29有v31,v30，v31有v32。將v18降至−10使它切離並標記v17；將v26降至−11使它切離並標記v25。刪除前根為v26(key−11,d0)、v18(key−10,d0)、v1(key1,d5)，標記者v17、v25。目標v29不是最小值，且仍有兩棵子樹。

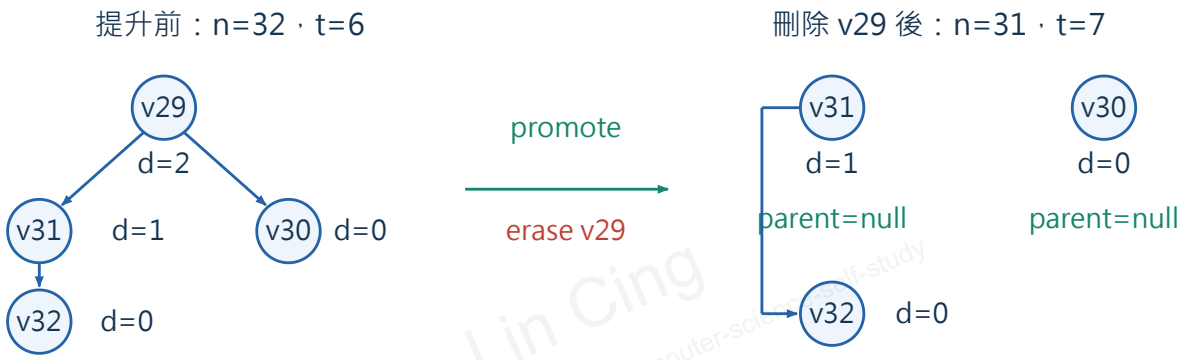
Erase 的合法起始狀態與三級切斷 （續）



另有根 v18(key −10)、v26(key −11)；金色表示 mark=true。
三角形代表保留的子樹群；點數不含上方節點。兩態皆有 n=32。

四個顯示節點、15／6／2／3 點的子樹群及另兩個根合計 32 點。切斷只改 parent 連結，保留所有節點身分；已標記的祖先切成根後清除 mark，到根 v1 時停止，不把 v1 標記。

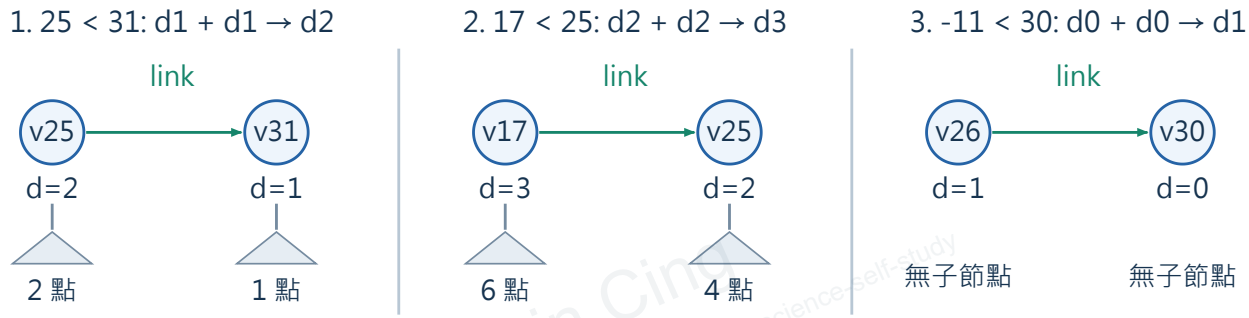
提升目標的子樹與移除唯一節點



其餘五根 v1、v17、v25、v26、v18 共 28 點不變；只有 h29 失效。

移動 v31 時保留其子 v32，因此它仍是兩點樹；加上單點 v30，提升的兩棵樹共有三點。唯一移除的是 v29，key=−11 的 v26 仍是最小節點，其餘 handle 仍指向原節點。

依 degree 進位的三次 link



A[0]=v18, A[1]=v26, A[3]=v17, A[4]=v1; min=v26 (key −11)
最終四棵樹的點數：1 + 2 + 12 + 16 = 31；t=4，m=0，Φ=4。

第一個 link 使 v25 成為 degree=2 的五點樹；再接到 v17 下方，得到 degree=3 的十二點樹。v30 接到 v26 下方後形成 degree=1 的兩點樹。未變動的根 v18、v1 分別保留一點與十六點。

Erase 的狀態表與身分守恆

已完成步驟	根數t	標記非根數m	Φ=t+2m	必要狀態變化
刪除前	3	2	7	v29.parent=v25，v25.parent=v17
cut v29	4	2	8	v25.degree由2降1
cascade切v25	5	1	7	v25清mark，v17.degree由3降2
cascade切v17	6	0	6	v17清mark，v1.degree由5降4
提升v31、v30	8	0	8	兩者parent=null，v29.degree=0
摘除並釋放v29	7	0	7	n由32降31，h29失效
三次link後	4	0	4	degree各異，重選min=v26

整併快照依序是v30(d0),v31(d1),v17(d2),v25(d1),v26(d0),v18(d0),v1(d4)。v25與v31同degree，v25贏得v31而升d2；再遇v17，v17贏v25而升d3；v26(key−11)贏v30而升d1。degree表最後0→v18、1→v26、3→v17、4→v1，共31節點、min仍為key−11。v31與v30只在中間狀態被提升為根，之後又合法成為子，handle仍指原節點；不能把「曾提升」誤解成最終一定是根。

任意刪除的正確性與攤銷推導

```
// Potential Phi_A=A*(t+2*m), with a constant A large enough for pointer work.
// Cut/cascade phase: k cuts, actual <= a*k+b, Delta(t+2m)<=4-k.
// Root extraction: degree d, initial t' roots, final at most D(n)+1 roots.
// Actual <= a*(t'+d+D(n)+1)+b; marks never increase in this phase.
```

正確性分三段。第一，切出z只刪一條父子邊，原有子樹內堆序不變；cascade嚴格執行「非根首次失子標記、再次失子即切斷」，維持degree證明所需的歷史限制。第二，提升z每個子樹保留其完整元素集合，唯一移除的資料就是z.item；移除前保存結果，移除後只有z的handle失效。第三，link只以較小根為父、等degree進位，不丟節點且保留堆序；從最後所有根取min即全域最小。即使z不是舊min，三段論證都不改變。

切斷階段若有k≥1次cut，根數增加k；除了首次切z之外，被繼續切的祖先原先都已標記，最後至多新標記一個祖先，故Δm≤2−k、Δ(t+2m)≤4−k。實際成本≤ak+b，取縮放係數A≥a，攤銷≤ak+b+A(4−k)≤b+4A=O(1)。z原本是根則k=0，這一段只做常數檢查。

任意刪除的正確性與攤銷推導 (續)

令切斷後有t′根，被刪根degree為d，整併所需最大degree上界為D(n)=O(log(n+1))。extract_root實際成本≤a(t′+d+D(n)+1)+b；提升與link都只清mark，最後根數≤D(n)+1，所以該段ΔΦ_A≤A(D(n)+1−t′)。加總並用A≥a及d≤D(n)，得到攤銷≤a(d+D(n)+1)+A(D(n)+1)+b=O(1+log(n+1))。兩段勢能差相接望遠鏡消去，整個erase即O(1+log(n+1))攤銷；不是對每段分別挑無法相容的勢能。

此操作不保證單次對數：長級聯、大量根快照／掃描都可花Θ(n)。從空堆的合法操作序列以同一非負Φ_A計帳；任意初始狀態另保留Φ初值。結果／鍵比較與節點釋放按固定成本模型，合法handle的查找成本不隱藏在其中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing heap 的簡單 link

- 標準 pairing heap 是多子 min-heap。meld 比較兩根，把較大根作為較小根的新子節點，只改固定數目指標。
- insert 是與單點 meld。decrease-key 通常在降低後把該節點與其子樹切下，再與根 meld。
- 找到並從兄弟串列切下需要額外 sibling/parent 或前驅資訊。只有單向 next 而沒有定位資訊，切斷不一定常數時間。
- 操作前須確認 handle 存活且屬於此堆；固定指標步數不代表 decrease-key 有 Fibonacci heap 的 O(1) 攤銷界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing 的 sibling 反向連結與切斷

```
// Pairing roots passed to link have parent=prev=next=null.
detach(x):
    p=x.parent
    if x.prev==null: p.first_child=x.next
    else: x.prev.next=x.next
    if x.next!=null: x.next.prev=x.prev
    x.parent=x.prev=x.next=null
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    b.parent=a; b.prev=null; b.next=a.first_child
    if b.next!=null: b.next.prev=b
    a.first_child=b; return a
```

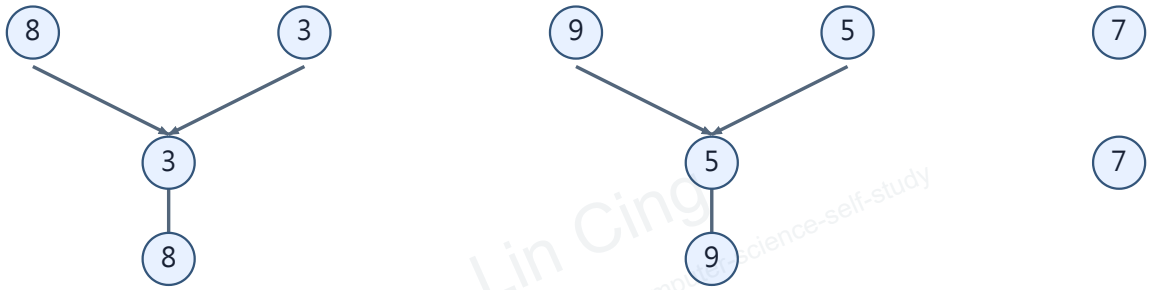
每個節點明存 parent、前兄弟 prev、後兄弟 next 與 first_child。detach 只用於非根x；first_child 的 prev=null，其餘兄弟的 prev 指前一兄弟。解除第一子必改父的first_child，解除其他子必改前兄弟的 next，兩者都要改後兄弟的prev。x的整個子樹原封保留，故降低鍵後可與根link。根本身降鍵不需 detach。刪根的第一輪配對前，逐子保存next並清parent/prev/next，才能把各子當作獨立根交給link。

Pairing 的完整兩輪程序與延後成本

```
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    prepend b to a.children; return a
delete_min():
    require root!=null
    children=detach_all_children(root); release(root)
    paired=[]
    for adjacent (a,b) left-to-right: paired.append(link(a,b))
    if odd last child remains: paired.append(last)
    root=null
    for heap in reverse(paired): root=link(heap,root)
decrease_key(x,new):
    require new<=x.key; x.key=new
    if x!=root: detach x in O(1) using backlinks; root=link(root,x)
```

Pairing 的完整兩輪程序與延後成本 （續）

Pairing 第一輪：相鄰兩兩比較，保留未配對7



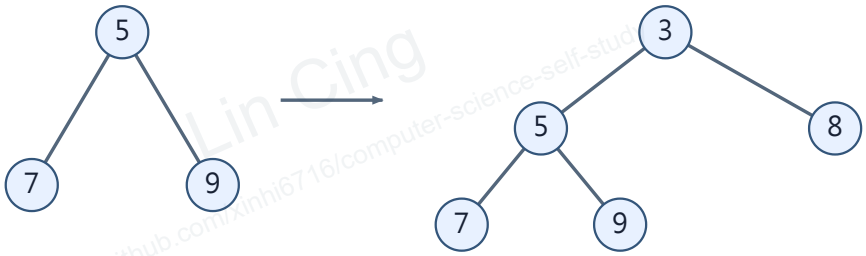
子根8,3,9,5,7先配(8,3)得3含子8、(9,5)得5含子9，7獨留。右至左先link(7,空)=7，再link(5,7)得5含子7,9，最後link(3,5)得3含子5,8。故最終3的子5仍保留7,9，沒有把被配對子樹拆散。每次delete-min的當下實際link數與舊根degree同階，可為Θ(n)。

Pairing 第二輪保留子樹的實際形狀

```
paired=[root3(child8),root5(child9),root7]
acc=link(root7,null); acc=link(root5,acc)
acc=link(root3,acc)       // 兩輪共 4 次非空 link
```

第二輪由右至左，整棵子樹一起合併

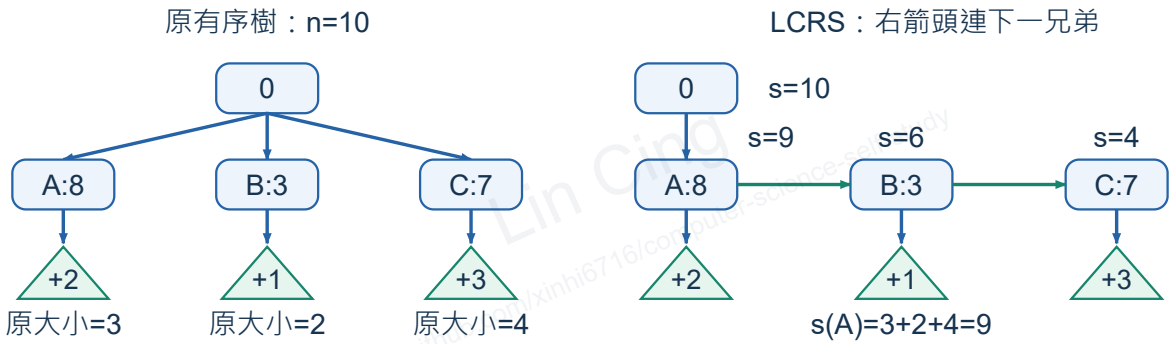
link(5,7) → 5



link(3,5) → 3

LCRS 表示與非負勢能

把有序多叉樹改看成 left-child/right-sibling（LCRS，左子／右兄弟）二元樹。令 s(v) 是從 v 沿這兩種指標可達的節點數，ρ(v)=log₂s(v)，Φ=Σ_vρ(v)。s(v) 包含右側兄弟的子樹，**不是原多叉樹中 v 的子樹大小**。所有 s(v)≥1，所以 Φ≥0；空堆 Φ=0。這些量僅供分析，不在操作時重新計算。



△ +j：省略的後代共 j 點，不含上方根

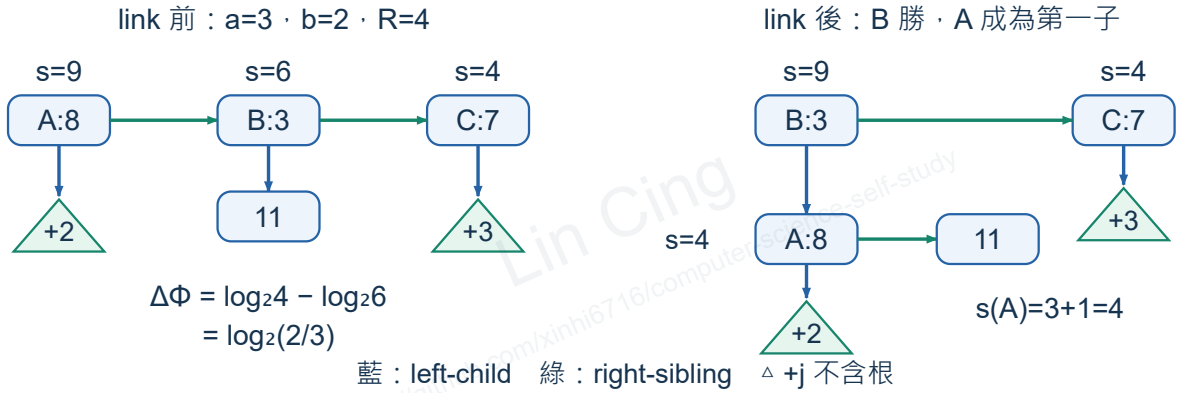
例中原根0有三棵子樹，大小3、2、4；二元表示中 s(A)=9、s(B)=6、s(C)=4，根的 s=10。刪根後，子樹按原順序形成 forest（森林），只有根的 log₂10 被移除，其餘 s 不變。

LCRS 表示與非負勢能 （續）

數個獨立堆的總勢能是各堆勢能之和，根之間不接 right-sibling。delete-min 內部則按剩餘分量順序定義暫時 forest 的虛擬兄弟鏈。即使用陣列暫存已清 next 的根，也用分量順序解讀此虛擬鏈；不把逐次清指標當作另一種堆勢能。每次宏觀 link 包含常數次解除／重接，掃描、暫存及最後釋放根共 O(k+1)，兩端勢能均與實際堆一致。

相鄰 link 只改兩個根的秩

考慮同一 forest 中相鄰的 A、B，其原多叉子樹大小為 a、b≥1，右方其餘分量共有 R≥0 點。link 前 s(A)=a+b+R，s(B)=b+R。較大鍵的根成為勝根的**第一個子節點**；相等鍵固定採任一一致規則。



link 後勝根的 s=a+b+R，敗根的 s=a+b−1。其他節點的 s 不變：左側根仍能到達同一批節點，舊子鏈內部也未改變。因此無論誰贏，

$\Delta\Phi_link=\log_2(a+b-1)-\log_2(b+R)$。

第一輪的正後綴不等式

令 A=a+b、T=A+R。當 R>0，由相鄰 link 公式得到

$1+\Delta\Phi_link = \log_2[2(A-1)/(b+R)] \le \log_2(2A/R) \le 2\log_2(T/R)$。

第一個不等式用 $A-1\le A$、$b+R\ge R$；第二個等價於 $2AR\le(A+R)^2$，因右減左為 $A^2+R^2\ge 0$。此推導只用大小，不依賴根鍵比較的勝負。

第一輪由左至右處理第 j 對，記其剩餘總量 $T_j=a_j+b_j+R_j$。尚未處理的右側完整保留，故相鄰兩對滿足 $R_j=T_{j+1}$。把各次不等式相加時，$\log T_{j+1}$ 與 $-\log R_j$ 抵消，留下首尾項。R=0 的最後一對不能代入除以 R 的式子，下一頁分開處理。

奇數個子根的第一輪望遠鏡和

刪除前 n 點，刪根後 N=n−1 點。原根有 k=2q+1 個子節點，第一輪做 q 次 link，最右一棵大小 r≥1 的樹不配對。每一對右方都至少有這棵樹，因此每次 R_j>0。

$q+\Delta\Phi_1 \le 2\Sigma_{(j=1)}^{q} \log_2(T_j/R_j) = 2\log_2(N/r) \le 2\log_2 N$。

所以 $\Delta\Phi_1 \le -q+2\log_2 N$。負的 −q 項將用來支付線性數目的 link；只寫 O(log n) 的勢能增加會漏掉這項必要抵銷。

既有五子例8,3,9,5,7皆為單點：N=5、q=2、r=1。兩次 link 的精確勢能差為 $\log_2(1/4)$、$\log_2(1/2)$，合計 −3。若 k=1，q=0、r=N，兩邊空和均為0，第一輪沒有 link；不需引入 log0。

偶數個子根的最後一對

k=2q≥2 時，最右一對沒有右後綴，R_q=0。令該對總大小 A_q=a_q+b_q≥2。前 q−1 對仍可望遠鏡相加：

$(q-1)+\Delta\Phi_{前} \le 2\log_2(N/A_q)$。

最後一對直接使用精確公式：$\Delta\Phi_{末}=\log_2(A_q-1)-\log_2 b_q \le \log_2 A_q$。相加得到

$\Delta\Phi_1 \le -q+1+2\log_2(N/A_q)+\log_2 A_q \le -q+1+2\log_2 N$。

q=1 時前段沒有 link，N=A_q，空和上界為0，公式仍成立。奇偶兩種情況可統一為 $\Delta\Phi_1 \le -q+2\log_2 N+1$，其中 $q=\lfloor k/2 \rfloor$；k=0 另作單點基例。

第二輪與完整 delete-min 攤銷界

第一輪留下 m=⌈k/2⌉ 棵樹。第二輪由右至左，令左邊待接樹大小 a、已累積右樹大小 b，故 R=0。每次 $\Delta\Phi_link=\log_2(a+b-1)-\log_2 b \le \log_2((a+b)/b)$。

下一次的累積量就是本次 a+b；因此 $\Delta\Phi_2 \le \log_2(N/b_0) \le \log_2 N$，其中 $b_0 \ge 1$ 是最右分量大小，$m \le 1$ 時沒有 link。五子例的兩次精確比值為2、4/3；連同第一輪及移除原根的 1/6，整次勢能比值為 (1/6)(1/4)(1/2)(2)(4/3)=1/18。

$k \ge 2$ 時，移根再加兩輪：$\Delta\Phi \le -\log_2 n - q + 3\log_2 N + 1 \le -q + 3\log_2 n + 1$。實際工作 $\le ck+d$，且 $k \le 2q+1$。固定選同一縮放勢能 $\Psi=C\Phi$、$C \ge 2c$，則實際+$\Delta\Psi \le (2c-C)q + 3C\log_2 n + c + d + C = O(1+\log n)$。這同時支付掃描及兩輪共 k−1 次 link，並非只支付第一輪。

k=0 時 n=1，刪後空堆，ΔΦ=0、實際 O(1)；k=1 時只把唯一子樹當根，$\Delta\Phi=-\log_2 n$、實際 O(1)。空堆 delete-min 拒絕。陣列式程序暫存 O(k) 根，可能 O(n) 空間；此證明不宣稱 O(log n) 暫存空間。

同一勢能下的 meld、insert 與 decrease

meld 兩個獨立非空堆，大小 a、b，n=a+b。link 前兩根各為獨立根，秩為 $\log_2 a$、$\log_2 b$；之後勝根 s=n、敗根 s=n−1，其餘不變。故

$\Delta\Phi_meld=\log_2[n(n-1)/(ab)] \le \log_2 n$，因 $ab \ge a+b-1=n-1$。任一堆為空則 ΔΦ=0。insert 是與單點 meld：非空插入的 $\Delta\Phi=\log_2 n$，空堆第一點 ΔΦ=0。雖兩操作當下均 O(1)，**此勢能分析對它們收 O(1+log(n+1)) 攤銷**。

decrease-key 若目標已為根，改鍵不改形狀、ΔΦ=0。非根 x 的原多叉子樹大小為 a。先切為兩個獨立堆：x 的 s 失去右兄弟後綴，原來能到達該子樹的二元祖先也只會減少可達點，其餘 s 不變，所以切斷的 $\Delta\Phi \le 0$。再 meld 回主堆，勢能增加 $\le \log_2 n$；使用正文 parent/prev/next 的合法 handle 才有 O(1) 實際 cut。故同一 Ψ 下 decrease-key 收 O(1+log(n+1))，不在此宣稱更強的 O(1) 攤銷。

序列總成本與分析的適用範圍

操作	當下實際成本	本節同一 Ψ 的攤銷收費
find-min	O(1)	O(1)
insert	O(1)	O(1+log(n+1))
meld	O(1)	O(1+log(n+1))，n 為合併大小
decrease-key	O(1)，須合法 handle/backlink	O(1+log(n+1))
delete-min	O(k+1)，最壞 O(n)	O(1+log(n+1))

從空堆集合開始，$\Psi_0=0$、最終 $\Psi \geq 0$，Σ實際=Σ攤銷+$\Psi_0-\Psi_末 \leq \Sigma$攤銷。令 N 是最多存活點數，I、M、D、K、F 分別為插入、合併、刪根、降鍵、查最小的次數，得到 $O((I+M+D+K)(1+\log_2(N+1))+F)$ 總成本；另建空堆每次 O(1)。任意非空初態另加 Ψ_0。

本節完整證明的是標準 two-pass 的上述一組共同界。不能把其他更精細分析的 O(1) 攤銷 insert/meld 直接移進此總式，改稱 O(I+M+(D+K)log N+F)；也不能從 O(1) 當下 cut 推成 O(1) 攤銷 decrease-key。操作方向與 prepend-child 規則若改變，必須重新核對不等式。

複雜度比較與選型

結構	find-min	meld	delete-min	decrease-key
Leftist	O(1) 最壞	O(log n) 最壞	O(log n) 最壞	本章不提供快界
Eager binomial	O(1)，需 min 指標	O(log n) 最壞	O(log n) 最壞	O(log n) 最壞
Fibonacci	O(1) 最壞	O(1) 最壞	O(log n) 攤銷	O(1) 攤銷
Two-pass pairing	O(1) 最壞	O(1) 指標工作	O(log n) 攤銷	本節同一勢能 O(log n) 攤銷

表中 decrease-key 均以已持有合法 handle 為前提。不同操作的攤銷界須屬於同一套分析，不能各取最樂觀結論任意拼接。

所有對數成本在空／單點情況都保留常數項：例如兩空堆meld也要判斷指標，為O(1)；leftist精確上界寫O(1+log(n1+1)+log(n2+1))。delete-min只對非空堆定義；空輸入應回報錯誤，不套用log0。

參考文獻與進階閱讀

- Fibonacci heap：Fredman–Tarjan 論文。
- Pairing heap：原始論文與作者說明、Pettie 的進階分析。
- Binomial heap：Stanford CS166 課程。
- 標準 two-pass 變體的 prepend-child、配對方向與 backlink：Sinnamon–Tarjan, § 2。本章採用的 LCRS 勢能與各操作共同收費，已在前文完整推導。
- Chicago CS223：二項樹表示、first-child link 與根串列順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前置知識與目標

- 前置知識為 BST、旋轉概念、遞迴、機率加權期望、動態規劃與攤銷分析。
- 能分辨 optimal BST 的靜態加權目標、AVL 與紅黑樹的最壞高度保證，以及 splay tree 的序列保證。
- 能以中序序列及結構不變量檢查旋轉，而不是只背圖形。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最佳化什麼成本

- 高度最小化降低最壞搜尋深度，未必最小化平均搜尋成本。
- 若鍵 `k_i` 以已知機率 `p_i` 被查詢，成功搜尋期望比較數為 `Σp_i(depth(k_i)+1)`。
- 若失敗查詢也計成本，要給 n+1 個鍵間空隙機率 `q_0,…,q_n`，並定義 dummy leaf 的成本慣例。
- Optimal BST 適合分布相對固定的靜態索引，頻繁插刪時重新最佳化成本必須另外計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功查詢版 optimal BST 遞迴

- 定義 `C[i,j]` 為有序鍵區間 `[i,j]` 的最小加權比較成本，空區間成本為 0。
- 設 `W[i,j]=Σ_{k=i}^j p_k`。若選 r 為根，兩子樹每個鍵都多一次比較。
- 因此 `C[i,j]=W[i,j]+min_{r=i..j}(C[i,r-1]+C[r+1,j])`。
- 依區間長度由短到長計算，前綴和使 W 查詢 O(1)，時間 `O(n³)`、空間 `O(n²)`，並記錄 root 重建樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

加權例子

- 三鍵機率 `p=[0.1,0.2,0.7]`。若選第二鍵為根，成本為 `1+0.1+0.7=1.8`。
- 若選第三鍵為根，左子樹以第二鍵為根，成本為 `0.7·1+0.2·2+0.1·3=1.4`。
- 最低高度的根二不會得到最低期望成本。高頻鍵靠近根可補償低頻鍵更深的成本。
- 「每次選最高機率鍵」仍不保證全域最優。反例 `p=[0.4,0.35,0.25]` 選第一鍵為根的最小成本是 1.85，選第二鍵為根則是 1.65。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

含失敗查詢的標準 DP 慣例

- 令 `e[i,i-1]=q_{i-1}`，空子樹視為一次 dummy 比較。令 `w[i,j]=Σp_i..p_j+Σq_{i-1}..q_j`。
- 遞迴仍為 `e[i,j]=w[i,j]+min_r(e[i,r-1]+e[r+1,j])`，但此時不可以把空子樹成本錯設為 0。
- 此成本是 `Σp_i(depth(key_i)+1)+Σq_j(depth(dummy_j)+1)`。若只計真鍵比較，失敗部分各減一次，總差為常數 `Σq_j`，最優樹不變。
- Knuth 的 `O(n²)` 最佳化需要適用的非負權重與單調根性質。一般任意區間 DP 不能只加相同名稱就沿用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimal BST 的完整區間表與失敗成本數例

```
for i=1..n+1: C[i,i-1]=0
for length=1..n:
    for i=1..n-length+1:
        j=i+length-1; C[i,j]=infinity
        for r=i..j:
            candidate=W(i,j)+C[i,r-1]+C[r+1,j]
            if candidate<C[i,j]: C[i,j]=candidate; root[i,j]=r
build(i,j):
    if i>j: return null
    r=root[i,j]; return Node(k[r],build(i,r-1),build(r+1,j))
```

成功版p=[.1,.2,.7]：C11=.1、C22=.2、C33=.7；C12候選根1=.5、根2=.4；C23候選根2=1.6、根3=1.1；C13候選根1=2.1、根2=1.8、根3=1.4。重建為3的左子2、2的左子1，三鍵深度2,1,0，直接加權=.1×3+.2×2+.7×1=1.4。

若p1=.6,q0=.1,q1=.3，只有一真鍵。dummy也計一次的初始化e10=.1,e21=.3,w11=1，得e11=1.4；只數真鍵比較則每次成功或失敗都比較該真鍵一次，期望1，兩者差q0+q1=.4。不能把e空區初始化0卻仍報dummy成本1.4。

Optimal BST 的完整區間表與失敗成本數例 （續）

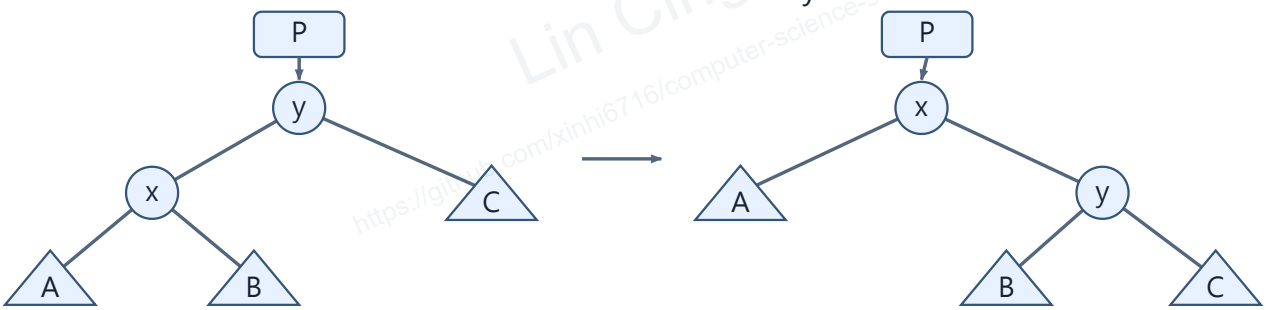
最優子結構證明：固定根r後，左右鍵範圍被順序唯一切開，任一非最優子樹都可替換成同鍵範圍的更小加權成本樹，而整體根比較成本W不變，與最優矛盾。依區間長度歸納便證DP，不需貪婪取最高機率鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

旋轉保留鍵的中序次序

- 右旋前根 y、左子 x，子樹順序為 `A<x<B<y<C`。右旋後根 x、右子 y，中序順序仍相同。
- 指標更新依序保留 B，令 `y.left=B`、`x.right=y`，再接回原父。若維護 parent，B 與兩根的 parent 也要更新。
- 高度、子樹大小或其他 augmentation 先更新降到下面的 y，再更新升上來的 x。
- 旋轉只保證 BST 次序不變，不會自動維持任意額外不變量。紅黑色彩或 AVL 高度要各自修復。

右旋：外部父與 A < x < B < y < C



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 的局部高度約束

- `h(null)=-1`，`h(leaf)=0`。balance factor `bf(x)=h(left)-h(right)` 必須屬於 `{-1,0,1}`。
- 對 h≥1，高度 h 的最小點數 `N(h)=1+N(h-1)+N(h-2)`，基底 N(−1)=0、N(0)=1；因最高子樹高 h−1，另一側至少 h−2。
- 此遞迴以 Fibonacci 數增長，故 h=`O(log n)`。搜尋、插入與刪除都有最壞 `O(log n)` 路徑長。
- 只檢查根平衡不夠，約束必須對所有節點成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 插入的四種局部形狀

- LL 為 x 左高且新點在左子的左側，對 x 右旋。RR 對稱左旋。
- LR 先對左子左旋，再對 x 右旋。RL 對稱處理。
- 原例插入 `30,10,20`，形成 LR。先讓 20 升到 10 上方，再讓 20 升到 30 上方，結果根 20、左右 10、30。
- 插入修復在第一個失衡祖先旋轉後，該子樹高度回到插入前，通常不需再向上旋轉，但仍需正確更新 metadata。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 刪除可能連續修復

- 先依 BST 刪除，從實際減少高度的位置往根更新。刪除後的局部修復不保證高度回到刪除前。
- 若 `bf(x)=2` 且 `bf(left)≥0`，右旋；若左子 bf<0，做 LR。右高情況對稱。
- 刪除特有的子 bf=0 情況不能沿用只看插入方向的判定。
- 修復最多沿根路徑進行 `O(log n)` 次局部更新，總最壞 `O(log n)`。只修第一個失衡點可能留下更高祖先失衡。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 重算、插入與刪除的共同程序

```
fix(x):
    if x==null: return null
    recompute_height(x)
    if bf(x)==2:
        if bf(x.left)<0: x.left=rotate_left(x.left)
        return rotate_right(x)
    if bf(x)==-2:
        if bf(x.right)>0: x.right=rotate_right(x.right)
        return rotate_left(x)
    return x
insert(x,k):
    if x==null: return leaf(k)
    if k<x.key: x.left=insert(x.left,k)
    else if k>x.key: x.right=insert(x.right,k)
```

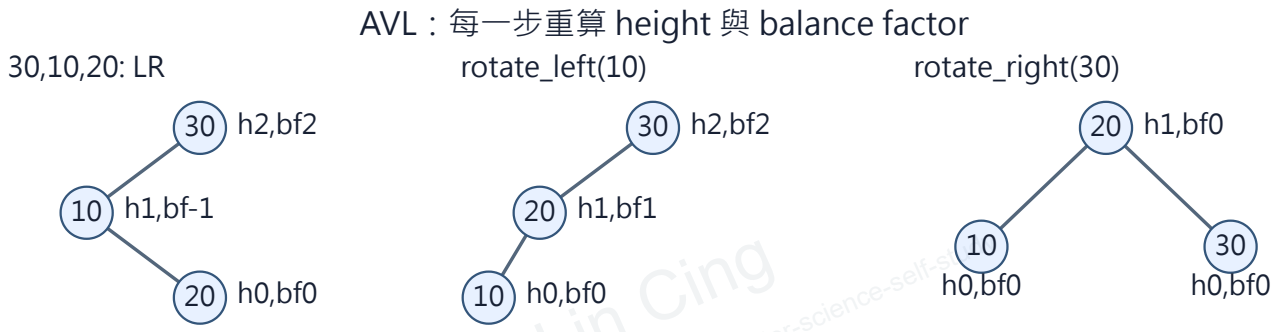
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 重算、插入與刪除的共同程序 （續）

```
    else: update/reject duplicate
    return fix(x)
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x has at most one child:
            child=the nonnull child or null; release(x); return child
        s=minimum(x.right); x.contents=copy(s.contents)
        x.right=erase(x.right,s.key)
    return fix(x)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 重算、插入與刪除的共同程序 （續）



每個旋轉先重算下降節點，再重算新根；不能只更換root指標而沿用舊height。30,10,20的LR過程：旋前30(h2,bf2)、10(h1,bf−1)、20(h0,bf0)；先旋10後30仍(h2,bf2)，其左20(h1,bf1)；再旋30後20(h1,bf0)、兩葉高度0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 重算、插入與刪除的共同程序 （續）

刪除例4的左2含1,3、右5為葉：刪5後4的bf2、左2的bf0，fix採單右旋而非LR，結果2左1右4、4左3。新局部高度仍2，所以此例上層無高度減少；其他刪除旋轉可使高度下降，需回溯繼續fix至根。遞迴每層常數旋轉與metadata工作，總O(log n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般紅黑樹不變量

- 每個節點為紅或黑，根黑，所有 NIL 葉為黑。
- 紅節點的子節點必為黑，且從任一節點到各後代 NIL 路徑的黑節點數相同。
- 本章講一般 node-colored 紅黑樹，不要求所有紅連結左傾。Princeton 的 LLRB 是較強的另一套表示慣例。
- 檢驗圖時必須把隱含 NIL 算入黑高約定，不能只目測有色實節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑樹高度上界

- 定義 bh(x) 為 x 以下、不含 x、含 NIL 的路徑黑點數。對 bh=b 的子樹，實節點數至少 `2^b-1`。
- 節點數下界可歸納證明：兩個子樹的黑高都至少 $b-1$，各至少有 $2^{b-1}-1$ 個實節點，加上根即至少 2^b-1；NIL 為零節點基底。紅點不能連續，因此任何根到 NIL 路徑至多有約兩倍黑高的邊數。
- 由 `n≥2^{bh(root)}-1` 得高度不超過 $2\log_2(n+1)$ 的標準界，端點常數依高度約定處理。
- 這是最壞高度保證，不需要鍵隨機或查詢分布均勻。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑插入修復的情況

- 新葉設紅，可保留原路徑黑高。若父黑即可停止，若父紅則產生 red-red。
- 叔叔紅時，把父與叔染黑、祖父染紅，將問題上推到祖父。
- 叔叔黑時，若是折線先旋父變直線，再旋祖父並交換適當顏色。鏡像情況對稱。
- 最後把根染黑。原創插入 `10,5,1` 會對 10 右旋，得到黑根 5、紅子 1 與 10。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑插入修復的完整祖先迴圈

```
insert_fix(z):
    while color(z.parent)==red:
        p=z.parent; g=p.parent
        if p==g.left:
            u=g.right
            if color(u)==red:
                p.black; u.black; g.red; z=g
            else:
                if z==p.right:
                    z=p; rotate_left(z); p=z.parent; g=p.parent
                p.black; g.red; rotate_right(g)
        else: apply exact left/right mirror
    root.black
```

新資料葉為紅、兩子NIL黑；根parent使用黑色哨兵，使空parent不被解參照。10先成黑根；插5紅不修；插1紅造成父5紅叔NIL黑，直線旋10右，得5黑、1與10紅。插7到10左，父10紅叔1紅，兩者黑、祖父5紅再根黑。最後每條根到NIL三黑，中序1,5,7,10。

紅黑插入修復的完整祖先迴圈 （續）

叔紅情況只上推z，沒有保證一次結束；叔黑折線先旋父再重取p/g，否則第二次旋轉用到舊parent會接錯。每輪問題上升或常數旋轉結束，總O(log n)時間。色彩由一般紅黑不變量推導，不要混用LLRB強制左傾的程式分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑刪除的黑高缺額

- 若刪掉紅節點，不改變黑高。若黑節點被紅子取代，把取代子染黑即可。
- 其餘情況在取代位置 x 追蹤「少一個黑」的缺額。這是分析狀態，不必真的新增第三種節點顏色。
- 設 x 是左子、兄弟為 w。若 w 紅，旋轉並換色，使兄弟轉為黑再處理。
- 若 w 黑且兩子黑，把 w 染紅並將缺額推到父。若父原紅，把父染黑即可消除缺額。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑刪除的近侄與遠侄

- 仍假設 x 在左側。w 的左子是近侄、右子是遠侄。
- 若 w 黑、遠侄黑、近侄紅，先右旋 w 並換色，化為遠侄紅的情況。
- 若遠侄紅，令 w 取得父的原顏色，父與遠侄染黑，對父左旋，消除 x 側黑高缺額。
- x 在右側時完全鏡像。每次修復只做常數局部工作或將問題上推，總最壞 `O(log n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

旋轉與紅黑刪除的移植入口

```
right_rotate(y):
    x=y.left; require x!=NIL
    b=x.right; p=y.parent
    y.left=b
    if b!=NIL: b.parent=y
    x.parent=p
    if p==NIL: root=x
    else if p.left==y: p.left=x
    else: p.right=x
    x.right=y; y.parent=x
    pull(y); pull(x)
transplant(u,v):
    if u.parent==NIL: root=v
    else if u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent       // including NIL: deletion needs its deficit parent
RB_erase(z):
```


旋轉與紅黑刪除的移植入口 （續）

```
    y=z; removed_color=y.color
    if z.left==NIL:
        x=z.right; p=z.parent; transplant(z,x); pull_up(p)
    else if z.right==NIL:
        x=z.left; p=z.parent; transplant(z,x); pull_up(p)
    else:
        y=minimum(z.right); removed_color=y.color; x=y.right
        if y.parent==z: x.parent=y
        else:
            p_old=y.parent; transplant(y,x)
            pull_up(p_old)          // repair old path before y changes parent
            y.right=z.right; y.right.parent=y
        transplant(z,y); y.left=z.left; y.left.parent=y; y.color=z.color
        pull(y); pull_up(y.parent)  // repair y at its new position
    release(z)
    if removed_color==black: delete_fix(x)
```

旋轉與紅黑刪除的移植入口 （續）

此頁採一般紅黑樹的黑色NIL哨兵，非資料節點。root.parent=NIL，NIL的color永遠黑；delete_fix採〈紅黑刪除四情況的完整控制流程〉。左旋是左右完全互換的程序。本節的完整增廣為size：NIL.size=0、pull(x)=1+size(left)+size(right)；pull_up由下向上重算至NIL，純紅黑色彩不需因旋轉自動改色。旋轉只在真子存在時執行，且先保留b與外部父p，才更改連結。

兩子刪除移動後繼節點y到z的位置，不複製鍵後直接釋放y。決定是否少一黑的是y原位置的removed_color，不是移到新位置後取得的z.color。x是y原位置的唯一右子或NIL。y緊鄰z時，x.parent必直接設y；否則先把y的右子移植到舊位置，再把整個z右子樹接y。最後左子樹與原z顏色接回，釋放的是z，其他元素身分保留。

NIL可由所有空子共用，但修復當前缺額時必須保留該x的parent。旋轉不替任意空中間子b重寫NIL.parent，以免覆蓋缺額位置；只有明確的transplant／緊鄰後繼分支設定它。刪唯一黑根使root=NIL，delete_fix迴圈因x==root跳過，仍是合法空樹。

旋轉與紅黑刪除的移植入口 （續）

正確性分工：BST移植保留中序與所有未刪元素，複製z顏色使y新位置的黑數不變，唯一可能缺額在y舊位置。四情況迴圈只處理這個缺額，維持其他子樹合法，直到在紅父、遠紅侄或根處消除。搜尋、後繼與修復皆走O(log n)路徑，移植／旋轉每次常數指標工作；另外兩條pull_up路徑各長O(log n)，不能省略其metadata更新。

子樹大小、選秩與兩條修復路徑

```
pull(x):
    require x!=NIL
    x.size=1+x.left.size+x.right.size
pull_up(x):
    while x!=NIL: pull(x); x=x.parent
select(k):
    require 1<=k<=root.size          // one-based rank
    x=root
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
    unreachable                     // sizes and rank invariant forbid this
```

每個資料節點的不變量為 size(x)=1+size(x.left)+size(x.right)，NIL.size=0。新資料葉size=1；插入先完成BST連接、從新葉的父pull_up至根，再做insert_fix。每次旋轉都先pull下降節點，再pull新局部根，才能在修復前後維持精確size。

子樹大小、選秩與兩條修復路徑 （續）

以合法插入4,2,6,1,3,5,7後刪紅葉1為例，刪除不需顏色修復或旋轉，但size仍須改變：

欄位	刪除前	只transplant、漏pull	正確刪除後
size(2)	3	3	2
size(4)	7	7	6
select(3)	節點3	走到NIL	節點4

漏更新時，根4讀到左size=3，誤認自己的秩為4，令第3秩往左。節點2再把它導向3的右NIL。正確左size=2時，根本身就是第3秩，直接回4。顏色與黑高正確不會自動證明增廣欄位正確。

至多一子分支只改z原父以上的子集合，故transplant後從保存的父p向上pull。兩子且後繼y不緊鄰z時，先在y原位置接回x，立刻從p_old向上pull；此時原祖先鏈仍完整。再把y搬到z位置，接齊左右子後pull(y)，並由其新父向上pull。緊鄰後繼省略原位置鏈，仍要做新位置鏈。兩鏈可能相交，但只多常數倍對數工作。

子樹大小、選秩與兩條修復路徑 （續）

一般正確性由下向上歸納：鏈外子樹未變，最底變動節點的兩子size已正確，pull便算對該點，逐層到根。後續紅黑修復只改色或旋轉；改色不影響size，旋轉僅改兩個局部根的子集合。依下降點→上升點pull後，局部總元素集合不變，其外部祖先的size不需再改。select的迴圈保留「k是目前子樹內所求的有效秩」，因左size正確而只排除不可能區間，有限高度保證終止。

size版本的兩條修復路徑與常數次旋轉，連同搜尋／後繼均為O(log n)最壞時間；select亦為O(log n)。若改維護height，旋轉後的局部總高度可能改變，必須另向祖先傳播，不能沿用size的「局部元素數不變」理由。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑刪除四情況的完整控制流程

```
while x!=root and color(x)==black:
    if x==x.parent.left:
        p=x.parent; w=p.right
        if color(w)==red:
            w.black; p.red; rotate_left(p); w=p.right
        if color(w.left)==black and color(w.right)==black:
            w.red; x=p
        else:
            if color(w.right)==black:
                w.left.black; w.red; rotate_right(w); w=p.right
            w.color=p.color; p.black; w.right.black
            rotate_left(p); x=root
    else: apply exact left/right mirror
x.black
```

前提是BST刪除／節點transplant已完成，x代表少一黑的位置，包含NIL哨兵且可取得其parent；原被實際移除節點若是紅色不需此迴圈。`.black/.red` 表示設色，不是新建節點。每次旋轉須更新外部parent及root。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

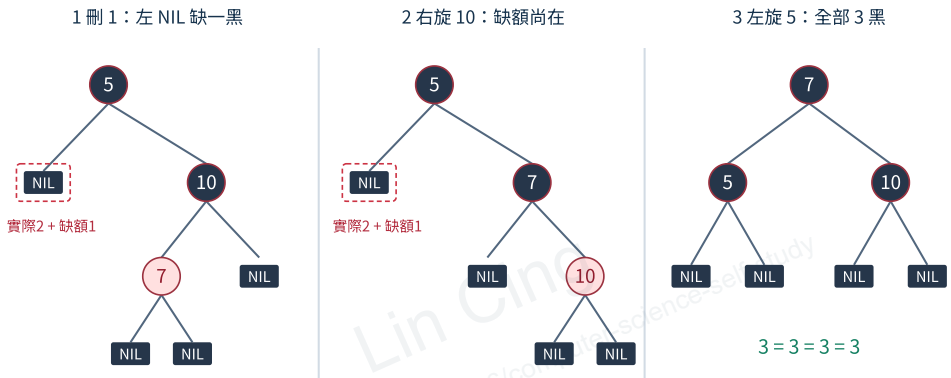
紅黑刪除四情況的完整控制流程 （續）

具體合法樹：根5黑，左1黑，右10黑，10左7紅，其餘NIL黑。刪1後左NIL缺一黑，兄弟10黑、近侄7紅、遠侄NIL黑。先把7黑、10紅，右旋10，得兄弟7黑且遠侄10紅。再令7取父5原黑、5黑、10黑，左旋5，得黑根7、黑子5與10。每條根到NIL共3黑，恢復刪前對外黑高；中序從[1,5,7,10]變[5,7,10]。

若兄弟紅，第一次轉換不立即消除缺額，只把兄弟轉黑後再進入其他情況；若兄弟兩子都黑，缺額向上，可能一直到根。不能把四個if任意改成互斥早退，也不能只修單層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑刪除的缺額與兩次旋轉對照



B：黑；R：紅；所有 NIL 皆黑。中序保持 5,7,10。

同一合法起始樹 5B(1B,10B(7R,NIL)) 刪除 1。圖的 NIL 都是黑；紅框虛線只表示左側分析上的一黑缺額。第一態左路實際含 5B、NIL 共 2 黑，右路為 3 黑；把缺額記入才暫時相等。第二態先把近侄 7 染黑、10 染紅並右旋 10，缺額仍在 5 的左 NIL，尚未修完。第三態 7 取 5 的黑色、5 與遠侄 10 染黑，再左旋 5，全部四條根到 NIL 路徑各 3 黑，缺額消失。

紅黑刪除的缺額與兩次旋轉對照 （續）

旋轉前後中序從 [5,7,10] 保持不變。每態的外部根連結與 parent 都要同步更新；圖中的黑數是含局部根及 NIL 的路徑計數，與前文 bh(x) 排除 x 的定義分開使用。若只做第二態的右旋便停止，左路仍少黑，不能把「近侄轉成遠侄」當成修復完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay 的 zig、zig-zig、zig-zag

- 查到 x 後把 x 旋到根。若只剩父而無祖父，做一次 zig。
- 若 x 與父同為左子或同為右子，做 zig-zig，先旋父相對祖父，再旋 x 相對父。
- 若方向相反，做 zig-zag，先旋 x 相對父，再旋 x 相對祖父。
- 這些步驟保留 BST 中序，但不維持固定平衡因子，也不保證每次操作後高度為 `O(log n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

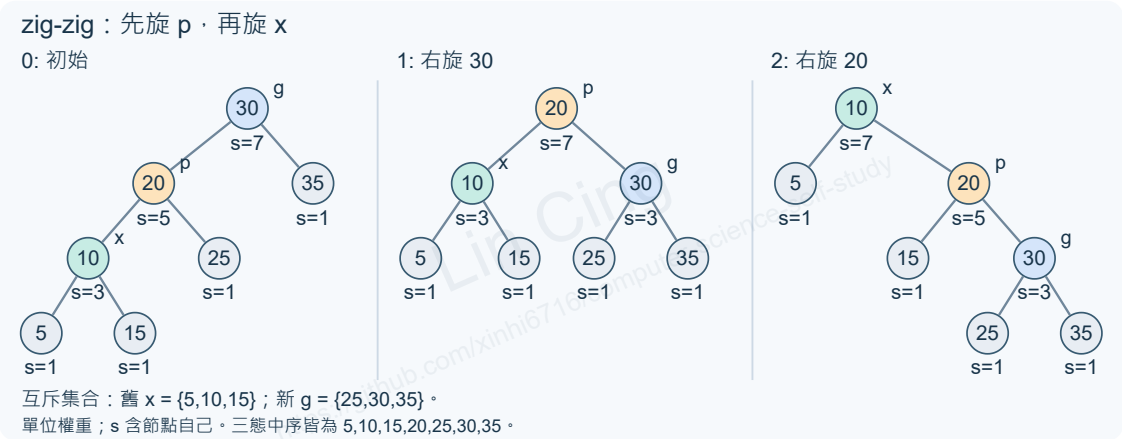
Splay 的存取引理與攤銷

- 對每點設定權重，令子樹權重和為 s(x)，rank $r(x)=\log_2 s(x)$，勢能 $\Phi=\Sigma r(x)$。
- access lemma 將一次 splay 的攤銷成本界為 $O(1+\log(s(root)/s(x)))$。
- 等權時得到每次操作攤銷 $O(\log\ n)$。對任意起始樹要保留初始勢能項，不能把任意單次操作說成最壞 log n。
- 原創鏈 $1\to2\to3\to4$ 搜尋 4，實際可走 4 點；反覆存取已在根的 4 卻只需常數工作，反映序列自調整。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zig：先讓父節點上升

固定 x=10、p=20、g=30。橘色 p 先越過 g，再讓綠色 x 越過 p；三個字母代表同一批節點，旋轉後不重新命名。



中間態根是 20，最後根才是 10。若先讓 x 越過 p，再越過 g，會得到不同樹形；「把目標轉到根」不足以指定 splay 的操作順序。

Zig-zig 的勢能帳，可以逐項核對

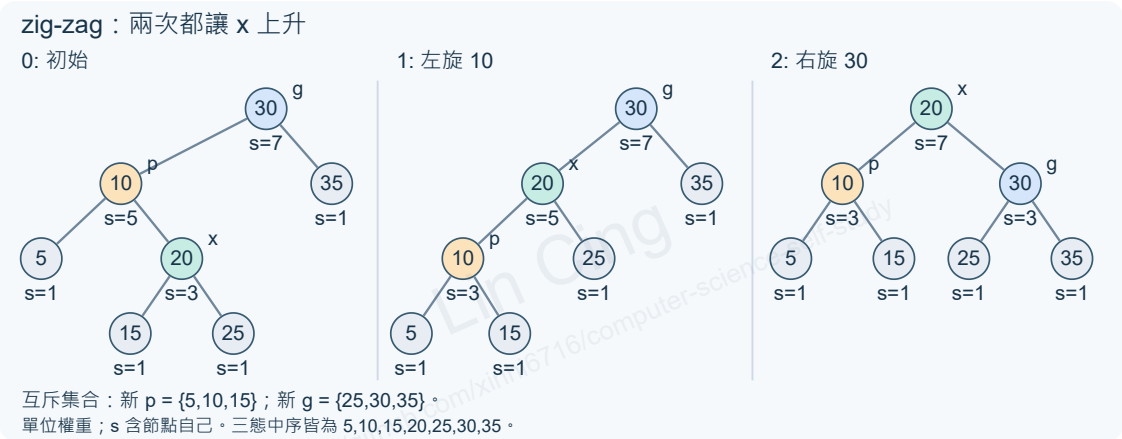
每個節點權重為 1，$s(v)$ 是含自己的子樹點數，$\Phi = \sum_v \log_2 s(v)$。四個葉節點的 rank 都是 0，只列會改變的三點。

節點	旋轉前 s	右旋 30 後 s	再右旋 20 後 s
x=10	3	3	7
p=20	5	7	5
g=30	7	3	3

前後勢能都為 $\log_2(3 \cdot 5 \cdot 7) = \log_2 105$，故 $\Delta\Phi = 0$，兩次旋轉的攤銷成本 $\hat{c} = 2$。引理給的上界為 $3\log_2(7/3) \approx 3.667$，確實大於 2。**勢能不必每次下降；它要支付的是整段操作的總成本。**

Zig-zag：兩次都讓目標上升

固定 x=20、p=10、g=30。先左旋 10，再右旋 30；四個葉節點的連結也必須隨旋轉接回。



最後 x 的左右子樹分別以 p、g 為根，兩者互不重疊。這正是下一頁使用凹性不等式時，要找出的兩個集合。

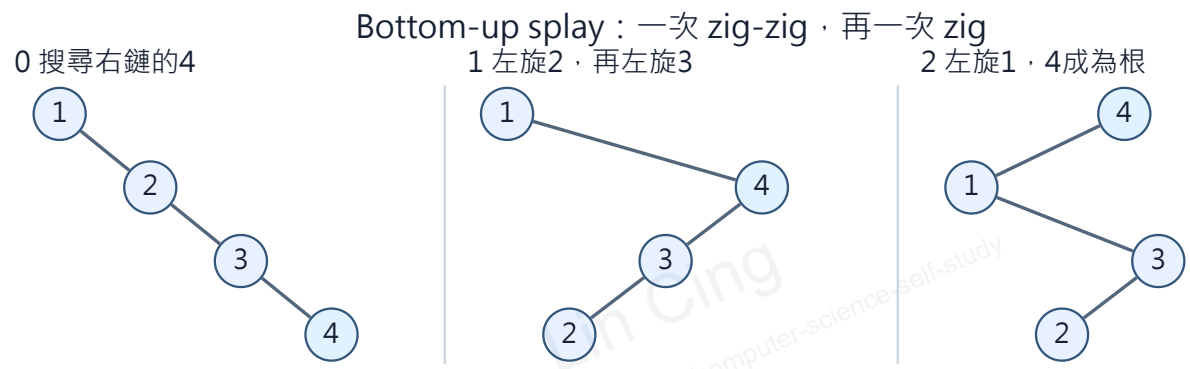
Zig-zag：數值如何對回凹性不等式？

節點	旋轉前 s	左旋 10 後 s	再右旋 30 後 s
x=20	3	5	7
p=10	5	3	3
g=30	7	7	3

$\Delta\Phi = \log_2(7\cdot 3\cdot 3) - \log_2(3\cdot 5\cdot 7) = \log_2(3/5) \approx -0.737$。所以 $\hat{c} = 2 + \Delta\Phi \approx 1.263$，小於 $2\log_2(7/3) \approx 2.445$。

圖中的兩個新子樹大小 a=b=3，且 a+b=6≤S=7。由 $(a-b)^2 \geq 0$ 可得 $ab \leq ((a+b)/2)^2 \leq (S/2)^2$，取 log₂ 得 $\log_2 a + \log_2 b \leq 2\log_2 S - 2$。正權重確保可取對數；**本例檢查數值，一般證明使用的是任意正 a、b。**

從右鏈搜尋到完整 splay



搜尋依序訪問 1、2、3、4；伸展先左旋 2、左旋 3，再左旋 1。最後 4.left=1、1.right=3、3.left=2，中序仍為 [1,2,3,4]。再次搜尋 4 只訪問根、不旋轉。這組操作前後的勢能同為 $\log_2 24$，此例仍符合存取引理；攤銷保證不要求每次勢能下降。

Access lemma 的局部記帳

固定每個節點的正權重，令 $s(v)$ 為子樹權重和、$r(v)=\log_2 s(v)$，勢能 $\Phi=\sum_v r(v)$。每次旋轉收取 1 單位成本；沒有旋轉的存取仍有常數搜尋成本。

一次雙旋只改變 x、p、g 的子樹權重。撇號表示雙旋完成後，由 $r'(x)=r(g)$ 消去一項，可得

$\hat{c}=2+\Delta\Phi=2+r'(p)+r'(g)-r(x)-r(p)$。

若兩個互不重疊的子樹權重為 a、b，且 a+b≤S，前頁已證明 $\log_2 a+\log_2 b\le 2\log_2 S-2$。接下來依雙旋形狀，選出這兩棵子樹。此處計算旋轉數；搜尋深度與旋轉數相差常數，指標操作的固定倍數可透過整體縮放勢能支付。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zig 的攤銷不等式

舊 x 子樹與新 g 子樹互不重疊，且都包含在新 x 子樹。以 a=s(x)、b=s′(g)、S=s′(x) 代入，可得

$r(x)+r'(g)\le 2r'(x)-2$。

又因 $r(p)\ge r(x)$、$r'(p)\le r'(x)$，所以

$\hat{c}\le 2+r'(x)+r'(g)-2r(x)$。

再用 $r'(g)\le 2r'(x)-2-r(x)$ 代入上式，整理得到 $\hat{c}\le 3(r'(x)-r(x))$。能相加的是 x 的 rank 增量，不是每一步的樹高下降量。兩次旋轉本身並不代表常數攤銷成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zag、zig 與望遠鏡和

Zig-zag。 新 p 與新 g 子樹互不重疊，因此 $r'(p)+r'(g)\le 2r'(x)-2$。配合 $r(p)\ge r(x)$，得 $\hat{c}\le 2(r'(x)-r(x))$，亦不大於三倍差量。

最後一個 zig。 單旋的攤銷成本為 $1+r'(p)-r(x)\le 1+r'(x)-r(x)$，至多 $1+3(r'(x)-r(x))$。每次完整 splay 至多出現一次 zig。

整次存取。 各步的 x rank 差量相消，最後 x 成為根。令整樹權重為 W、存取前 x 子樹權重為 $s_0(x)$，則攤銷旋轉數至多

$3(\log_2 W-\log_2 s_0(x))+1$。

單位權重時 W=n、$s_0(x)\ge 1$，得到 O(log n) 攤銷界。總實際成本仍須加上初始勢能、減去最終勢能，故不排除某次存取花 Θ(n) 時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay 插入、刪除與適用限制

- 插入可先 BST 插入新葉，再把新葉 splay 至根。
- 刪除先把目標 splay 到根，移除後得到 L、R。若 L 非空，把 L 的最大鍵 splay 成根，其右子為空，可接上 R。
- 不成功搜尋可 splay 最後訪問的節點，維持對查詢序列的調整效果。
- splay 會在讀取時修改結構，並發唯讀存取與快照需求須額外設計。單次低延遲需求較適合有最壞界的平衡樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay 的完整旋轉與 split/join 更新

```
splay(x):
    while x.parent!=null:
        p=x.parent; g=p.parent
        if g==null: rotate x over p
        else if (x is p.left)==(p is g.left):
            rotate p over g; rotate x over p
        else: rotate x over p; rotate x over g
    root=x
insert(k):
    if root==null: root=Node(k,parent=null); return
    search k; splay(found or last visited node)
    if root.key==k: update/reject duplicate; return
    if k<root.key:
        z=Node(k,left=root.left,right=root,parent=null); root.left=null
        if z.left!=null: z.left.parent=z
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay 的完整旋轉與 split/join 更新 （續）

```
    else:
        z=Node(k,left=root,right=root.right,parent=null); root.right=null
        if z.right!=null: z.right.parent=z
    root.parent=z; root=z
delete(k):
    search and splay target or last visited node; if absent: return
    (L,R)=(root.left,root.right)
    if L!=null: L.parent=null
    if R!=null: R.parent=null
    release root
    if L==null: root=R
    else:
        root=L; splay(maximum(L)); root.right=R
        if R!=null: R.parent=root
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

空樹insert直接建葉；split/join每次同時接回child與parent。刪根後先把L、R的parent清空，再釋放舊根，避免splay沿已釋放祖先走訪。鏈1→2→3→4搜尋4，先在祖父2與父3做兩個左旋，局部4左3左2；再對根1做zig，成4左1、1右3、3左2，並非單純變成反向鏈。

Splay 的完整旋轉與 split/join 更新 （續）

查詢的access lemma已逐旋轉證明；更新另有權重及勢能變化。單位權重插入split時，新根rank至多log(n+1)，被截斷子樹只會降低既有節點rank，故額外勢能增加O(log(n+1))。刪除後split不增rank，對L最大值splay用lemma，再接R只有新根rank最多增加log n，故更新也有O(log n)攤銷界。這不提供單次最壞高度或唯讀操作不修改的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較與課程延伸

結構	最佳化對象	搜尋保證	更新代價特徵
Optimal BST	已知靜態加權成本	依分布最優	重建另計
AVL	局部高度差	O(log n) 最壞	維護高度，刪除可連續旋轉
紅黑樹	黑高與紅節點限制	O(log n) 最壞	色彩與旋轉修復
Splay	整段操作序列	O(log n) 攤銷	查詢本身會旋轉

研究所延伸可從 access lemma 改用非均勻固定權重推導存取頻率的代價，或驗證 optimal BST 的 root 單調性。前者需說明權重與查詢次數的關係，後者完整最佳化證明不在本稿範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考用書與延伸閱讀

- Sleator、Tarjan，〈Self-Adjusting Binary Search Trees〉，JACM 32(3)，1985，§2：bottom-up splay 與 access lemma。圖中數值例採本章自訂鍵值與單位權重。
- AVL 的課程定位核對 MIT 6.006 2020 L7。紅黑樹延伸對照 Princeton balanced search trees，該頁採 LLRB，不能混用插刪程式。
- 本章採用的比較成本、黑高及旋轉慣例均在相關段落定義；對照其他教材時，先確認端點與成本定義一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前置知識與目標

- 前置知識為 BST、二分搜尋、有序序列、樹高與基本磁碟頁面概念。
- 能由 m-way search tree 推導 B-tree 佔用率及高度限制，追蹤分裂、借位與合併。
- 能區分 B-tree 與 B+ tree 的資料位置、separator 語義與範圍掃描成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多元搜尋樹的區間不變量

- 一個內部節點有遞增鍵 $k_1<\dots<k_r$，最多 m−1 鍵，並有 r+1 個子樹。
- 子樹區間依序是 $(-\infty,k_1)$、(k_1,k_2)、⋯、$(k_r,+\infty)$，並受祖先區間再限制。
- 單一節點可二分搜尋 r 個鍵，CPU 比較成本 $O(\log m)$。但一般 m-way tree 不保證平衡或最低佔用。
- 若節點可長期只有一個子或葉深度不同，不能僅因「每點最多 m 子」就宣稱高度 $O(\log_m n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

外部記憶體模型與 fanout

- 資料切成 block（區塊），一次 I/O 在主記憶體與外部儲存間搬一個 block。本章用 B 表示每塊可容納的項目量。
- 多個鍵與子指標放同頁，可在一次頁讀後決定下一層路徑。大 fanout（分支數）用來減少層數。
- 在分支及葉容量皆為Θ(B)的頁索引中，$O(1+\log_B n)$ 描述頁面 I/O，不代表每頁內只做 O(1) 次 CPU 比較。
- 實際頁大小、鍵長、指標、頁頭與填充率共同決定 fanout。SSD 也有頁與快取成本，但延遲不能由理論界直接推定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 採 minimum degree t 慣例

- 本教材 $t \geq 2$。非根節點含 $t-1$ 至 $2t-1$ 鍵。非葉內部節點有鍵數加一個子節點。
- 非葉根含至少 1 鍵、2 子。根若是葉，可為空。所有葉具有相同深度。
- 每個節點的鍵與子樹區間遵守搜尋次序。資料記錄可放在內部節點與葉，不限於葉。
- 若別的教材以「order m」表示最大子數，m 不一定與此 t 同義。比較公式前必須換算定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 高度上界推導

- 對高度 $h \ge 1$ 的非空樹，根至少有 2 子，此後每個非根內點至少 t 子。
- 深度 $d \ge 1$ 至少 $2t^{d-1}$ 個節點，各至少 $t-1$ 鍵。
- 因此 $n \ge 1+2(t-1)\Sigma_{d=1}^{h} t^{d-1}=2t^h-1$，得到 $h \le \log_t((n+1)/2)$。
- t 與 block 容量同階且一點占一塊時，搜尋只走一條根葉路徑，I/O 為 $O(1+\log_B n)$，根快取可再少一次讀取。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋同時檢查節點鍵與子區間

1. 在節點內找第一個不小於 x 的鍵位置 i。
2. 若恰等於 x，回傳該記錄。若未找到且是葉，回報不存在。
3. 否則進入第 i 個子樹，重複上述步驟。

根鍵 $[20,50]$ 將查詢 37 導向中間子樹。B-tree 在內部鍵相等時可以成功，這點與本教材 B+ tree 的葉端查詢不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 插入先分裂滿子節點

- 採 top-down 插入。若根滿，先建立新根並分裂舊根，再下降。
- 滿節點有 `2t−1` 鍵，中位第 t 鍵提升到父。左右各保留 t−1 鍵及相應子指標。
- 每次下降前先分裂滿子，保證抵達葉時有空位。父節點此時也必須有空位，這由下降不變量保證。
- 分裂保持鍵總集合與葉深度一致。根分裂是唯一使整體高度增加一層的事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

t=2 插入 trace

- 依序插入 `10,20,5`，根葉為 `[5,10,20]`，已滿。
- 插入 6 前分裂根，提升 10，左右葉為 `[5]`、`[20]`。再把 6 插入左葉得 `[5,6]`。
- 繼續插入 `12,30`，右葉成 `[12,20,30]`。插入 25 前分裂該葉，把 20 提升到根。
- 最終根 `[10,20]`，三葉 `[5,6]`、`[12]`、`[25,30]`。鍵 10 與 20 在內部，不重複留在葉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 分裂與插入的完整陣列操作

```
split_child(parent,i):
    y=parent.children[i]; require len(y.keys)==2*t-1
    z=Node(leaf=y.leaf,keys=[],children=[])
    median=y.keys[t-1]
    z.keys=y.keys[t:]; y.keys=y.keys[:t-1]
    if not y.leaf:
        z.children=y.children[t:]; y.children=y.children[:t]
    insert median at parent.keys[i]
    insert z at parent.children[i+1]
insert_nonfull(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k: return already_present
    if x.leaf: insert k at i; return
    if x.children[i] is full:
        split_child(x,i)
        if k>x.keys[i]: i+=1
        else if k==x.keys[i]: return already_present
    insert_nonfull(x.children[i],k)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 分裂與插入的完整陣列操作 (續)

根滿時先以新根含舊根一子，split_child(newroot,0)，才呼叫insert_nonfull。內部節點需同時分配children，不能只分鍵；左t子和右t子各對應t−1鍵，葉深度因新根提升才整體加1。

t=2的本章序列最後根[10,20]、葉[5,6],[12],[25,30]。再插7只改左葉為[5,6,7]；再插8前，分裂該葉提升6，根成[6,10,20]，葉[5],[7,8],[12],[25,30]。父此時滿但本次已完成；下一次插入前才分裂根。這展示top-down滿子分裂與「溢位後才分裂」是不同成套程序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除前確保下降子節點有至少 t 鍵

- 若欲下降的子只有 t−1 鍵，先透過借位或合併補強，避免刪除後低於最低佔用。
- 若相鄰兄弟有至少 t 鍵，從兄弟經父 separator 旋轉一鍵，必要時一併移動子指標。
- 若兩相鄰子都只有 t−1 鍵，連同父的一個 separator 合併，得到 `2t−1` 鍵。
- 借位不改變層高。合併可能使父變少；top-down 的下降條件使非根父仍合法，根有特例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除內部鍵與縮根

- 若鍵在內部節點，左子至少 t 鍵時，可用其前驅取代，再從左子刪前驅。
- 否則若右子至少 t 鍵，使用後繼；若兩子都只有 t−1 鍵，與目標鍵合併，再往合併子刪除。
- 若根刪到 0 鍵而仍有一子，讓該子成為新根，高度減 1。若空葉根無資料，整樹為空。
- 每層至多常數次節點重組，I/O `O(1+log_t n)`；若節點內以陣列搬動 t 個鍵，CPU 時間可為 `O(t(1+log_t n))`。

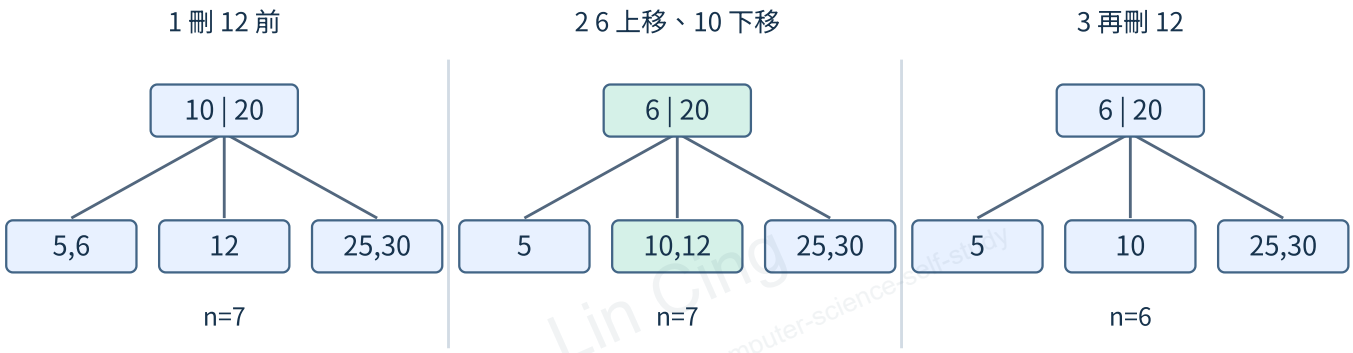
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除借位 trace

- 從根 `[10,20]`、葉 `[5,6]`、`[12]`、`[25,30]` 刪除 12，取 t=2。
- 中間葉只有 1 鍵，下降前向左兄弟借位。父的 10 下移到中葉，左兄弟最大鍵 6 上升成父鍵。
- 狀態變成根 `[6,20]`，葉 `[5]`、`[10,12]`、`[25,30]`。再刪 12 得中葉 `[10]`。
- 所有非根節點至少 1 鍵，葉深度一致，中序鍵集合正好移除 12。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 借位時資料鍵的雙向移動



t=2，每個非根葉需 1 至 3 鍵。下降到 [12] 前，先把父資料鍵 10 放進中葉，同時把左兄弟最大資料鍵 6 放進父的原槽位；這是兩筆不同資料的移動，不是把 6 複製到父後仍留在左葉。第二態中葉已含兩鍵 [10,12]，才可刪除 12，得到第三態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 借位時資料鍵的雙向移動 （續）

三態的中序依次為 [5,6,10,12,20,25,30]、同一序列、[5,6,10,20,25,30]。借位不遺失或重複記錄，刪除才恰少一筆；葉深度都為 1。若兄弟是內部節點，連同邊界子指標移動，不能只照葉圖搬鍵。B+ 的父 separator 只是路由副本，不能把此 B-tree 資料移動規則套給 B+ 葉借位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 刪除借位、合併與縮根的控制流程

```
prepare_child(x,i):
    child=x.children[i]
    if len(child.keys)>=t: return i
    if i>0 and len(x.children[i-1].keys)>=t:
        left=x.children[i-1]
        child.keys.insert(0,x.keys[i-1]); x.keys[i-1]=left.keys.pop()
        if not child.leaf: child.children.insert(0,left.children.pop())
    else if i+1<len(x.children) and len(x.children[i+1].keys)>=t:
        right=x.children[i+1]
        child.keys.append(x.keys[i]); x.keys[i]=right.keys.pop(0)
        if not child.leaf: child.children.append(right.children.pop(0))
    else:
        if i>0: merge_children(x,i-1); return i-1
        merge_children(x,i); return i
    return i
```

若目標在葉直接移除；在內部時，左子至少t鍵便以前驅替換並遞迴刪前驅，否則右子至少t鍵便用後繼，兩子都t−1則合併目標與兩子再向下。若目標不在當前節點，先prepare_child再用回傳的索引下降；向左合併後原索引會減1，不能沿用舊i。結束後根空且有一子便縮根。

B-tree 刪除借位、合併與縮根的控制流程 （續）

本章刪12的左借例為根[10,20]、中葉[12]，左[5,6]借6上父、10下中，刪後根[6,20]、葉[5],[10],[25,30]。再刪5：左葉只有1鍵，右鄰中葉也1鍵，合併[5]+父6+[10]得[5,6,10]，根剩[20]，再刪5得[6,10]。若接著把右葉[25,30]刪25成[30]、左刪6成[10]，再刪10前需合併[10]+20+[30]，根空而縮為新葉[10,20,30]，刪後[20,30]。

每次借位保持排序與同葉深；合併容量2t−1恰合法。節點一頁時每層常數I/O，總O(1+log_t n)；頁內陣列位移可O(t)每層，不能把頁I/O界當CPU步數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 根入口與完整刪除遞迴

```
insert(k):
    if root is full:
        old=root; root=Node(leaf=false,keys=[],children=[old])
        split_child(root,0)
    insert_nonfull(root,k)
erase_from(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if x.leaf: remove x.keys[i]; return
        if len(x.children[i].keys)>=t:
            p=maximum_key(x.children[i]); x.keys[i]=p
            erase_from(x.children[i],p)
        else if len(x.children[i+1].keys)>=t:
            s=minimum_key(x.children[i+1]); x.keys[i]=s
            erase_from(x.children[i+1],s)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree 根入口與完整刪除遞迴 （續）

```
        else:
            merge_children(x,i); erase_from(x.children[i],k)
    else:
        if x.leaf: return absent
        i=prepare_child(x,i); erase_from(x.children[i],k)
erase(k):
    erase_from(root,k)
    if len(root.keys)==0 and not root.leaf:
        old=root; root=old.children[0]; release(old)
merge_children(x,i):
    left=x.children[i]; right=x.children[i+1]
    left.keys+= [x.keys[i]] + right.keys
    if not left.leaf: left.children+=right.children
    remove x.keys[i] and x.children[i+1]; release(right)
```

空B-tree以一個0鍵葉根表示，因此空樹erase會在葉回報absent。此頁簡寫key；key-value記錄實作需搬動整筆配對，不能只改鍵。merge_children只用於兩子各t−1鍵的狀態，合併後(t−1)+1+(t−1)=2t−1鍵，恰未超容量。父的中間鍵真的是資料記錄，與B+內部純separator不同。

B-tree 根入口與完整刪除遞迴 （續）

下降不變量是非根遞迴入口至少t鍵；prepare_child先借位或合併維持此條件。命中內部鍵的前驅／後繼分支只選至少t鍵的子，刪後仍至少t−1；兩子都不能借時，把目標一起合併才繼續刪。prepare_child若往左合併必回傳i−1，因舊child[i]已被釋放。即使鍵不存在，也可能先重組路徑；抽象鍵集合不變，不承諾不存在時實體樹形不變。

根[20]、葉[10],[30]，t=2，刪10前兩子都只有一鍵，merge_children把根20夾在中間，得葉[10,20,30]；刪10後[20,30]，空內部根縮成此葉。若錯把separator20直接丟掉，會遺失尚未刪的資料。葉深一致性由同層兄弟重組保留，只有根合併改變全樹共同高度。

B+ tree 把資料記錄留在葉

- 本章 B+ tree 的資料 key-value 全在葉，內部節點只放 separator 與子指標。
- 內部 separator 定義為右子樹最小鍵，查詢相等時往右側下降。此慣例與「取左最大」不可混用。
- 葉依鍵序連結，讓範圍查詢在定位首葉後沿葉鏈掃描。
- 內部 separator 是路由資訊，可能與葉內鍵重複，不能把內部與葉鍵數相加當成資料筆數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ 容量必須分別定義

- 本教材取整數 $M \geq 3$、$L \geq 2$。非根內部節點子數範圍 $[a, M]$，其中 $a = \lceil M/2 \rceil \geq 2$；內部根有 2 至 M 子，內部鍵數為子數減一。最低分支至少 2 才能支援後續對數高度論據。
- 非根葉容量範圍 $[\lceil L/2 \rceil, L]$ 筆記錄。根為葉時允許 0 到 L 筆。
- 內部 M 與葉 L 可不同，因內部儲存子指標，葉可能存 record pointer 或完整資料。
- 葉分裂後把右葉最小鍵複製到父。內部分裂則提升分隔邊界並重新分配子節點，不能直接照搬葉分裂公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ 葉分裂與 separator

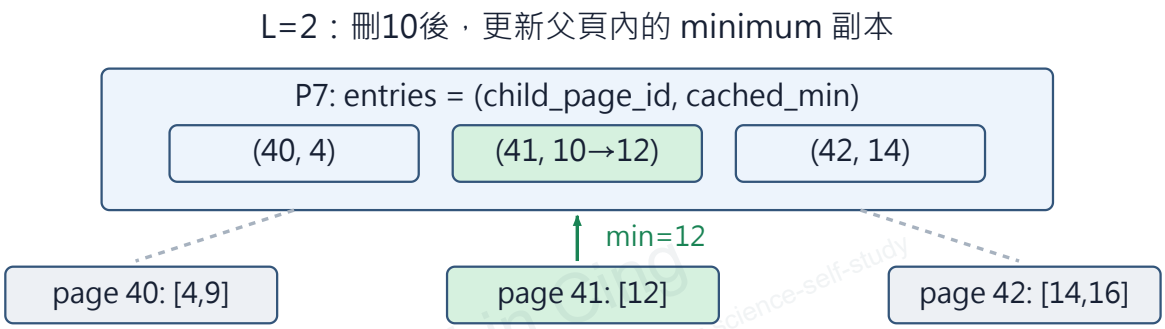
- 設 L=3，原葉 `[4,9,12]`，插入 10 形成暫時序列 `[4,9,10,12]`。
- 分成左葉 `[4,9]`、右葉 `[10,12]`，把 separator 10 複製到父，並接好前後葉鏈。
- 若父只有兩個子，根為 `[10]`。查 10 時必須向右，直到右葉讀到真正記錄。
- 與 B-tree 不同，10 不因提升到父而從葉移除。

磁碟頁布局與局部 minimum 更新

```
// Internal page: entries[j]=(child_page_id,cached_min), all in this page.
// Separators are entries[1:].cached_min; no persistent parent pointers.
refresh_local(p):
    if p.leaf: p.minimum=p.records[0].key if nonempty else absent
    else: p.minimum=p.entries[0].cached_min
entry(p): return (p.id,p.minimum)
find_leaf(k):
    path=[]; p=read(root_id)
    while not p.leaf:
        i=upper_bound(p.entries[1:].cached_min,k)  // equality goes right
        path.push((p.id,i)); p=read(p.entries[i].child_page_id)
    return path,p
```

磁碟模型中，每個節點恰占一頁。內頁將每個子頁的ID與其精確minimum副本放在本頁；separator只是第1筆以後的cached_min，不需再讀子頁。葉頁存有序記錄與next/prev頁ID。minimum副本由被修改子頁回傳後，沿祖先stack增量維護；未變動entry保持原值。

磁碟頁布局與局部 minimum 更新 (續)



圖採M=3、L=2，父P7的三筆entry原為(40,4),(41,10),(42,14)，葉41為[10,12]。刪10後葉41剩[12]仍達最低1筆，只需將P7的entry1改為(41,12)，separator由[10,14]變[12,14]。頁40、42的資料沒有改，cached_min也不改；不必讀它們。

磁碟頁布局與局部 minimum 更新 (續)

若refresh逐一讀M個child.minimum，即使只改一筆也可能多M次冷頁讀取；若internal split搬Θ(M)個child後逐頁改parent，也會多Θ(M)次I/O。把欄位稱作「快取」並不讓它自動在父頁。本布局明定這些副本的實際位置，並完全取消持久parent；祖先stack只存(page_id,index)，向上重讀父頁即可。

這與Open Data Structures § 14.2.2避免B-tree子頁parent改寫的設計理由一致。本章的B+ entry/minimum與葉鏈布局為此處明訂的教學模型。

B+ 插入、精確separator與向上分裂

```
insert(k,v):
    path,node=find_leaf(k)            // stack holds (parent_page_id,index)
    insert/update (k,v) in sorted node.records
    while true:
        if node exceeds L records or M entries:
            parts=split_into_floor_ceil_halves(node)
            if node.leaf: splice both leaf links and repair adjacent leaf
        else: parts=[node]
        for p in parts: refresh_local(p); write(p)
        if path empty:
            if len(parts)==1: root_id=parts[0].id
            else:
                r=new_internal(entries=[entry(p) for p in parts])
                refresh_local(r); write(r); root_id=r.id
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        parent.entries[i:i+1]=[entry(p) for p in parts]
        node=parent
// entry(p)=(p.id,p.minimum); internal splits move complete cached entries.
```


B+ 插入、精確separator與向上分裂 （續）

L=3的[4,9,12]插10→[4,9,10,12]→葉[4,9],[10,12]，根[10]。再插14使右葉[10,12,14]；插16使右葉分成[10,12],[14,16]，父根變[10,14]。10與14在父是路由副本，各資料記錄仍在葉，根分隔區間是(-∞,10)、[10,14)、[14,+∞)。

B+ 插入、精確separator與向上分裂 （續）

此處採前節定義的磁碟頁布局：內頁 entries 同頁保存 (child_page_id,cached_min)，不在子頁存 parent。左半沿用原page-id，右半配置新頁；分裂只搬完整 entry，既有 cached_min 隨它一起移動；左右新頁各由本頁資料 refresh_local，父頁只接收已變動子頁回報的新 entry。不讀其他子頁來重算 minimum，也不逐一改 moved child 的磁碟 parent。

find_leaf 保存的是祖先頁ID與下降索引，不保留全部祖先頁。向上時重讀當前父頁，以一或兩筆新 entry 取代原 entry；父頁若溢位再分裂，根溢位才建新根。沒有溢位也繼續傳播精確 minimum，故首鍵變更不會在上層留下舊 separator。非根內頁至少a=ceil(M/2)子，葉至少ell=ceil(L/2)筆，M≥3,L≥2確保兩半合法。

每層只讀寫常數個頁；葉的雙向鏈至多另更新相鄰葉一頁。因此本布局插入為O(h+1)頁I/O，頁內陣列搬移則最壞O(L+Mh) RAM工作。祖先stack為O(h+1)個詞，另外需常數個大小O(M+L)的頁緩衝。完整記錄大小、entry大小與頁容量須先固定，不能把RAM與磁碟欄位任意混用。

B+ 刪除與邊界更新

- 葉刪除後低於最低記錄數，先嘗試從相鄰葉借記錄，否則合併並移除父的 separator。
- 即使沒有 underflow，只要右子樹最小鍵改變，採精確「右最小」separator 的本教材就需更新相應祖先邊界。
- 原例從右葉 `[10,12]` 刪除 10，若 L=2、最少 1 筆，葉不缺額，但父 separator 應由 10 改成 12。
- 某些系統允許保留舊但仍合法的 fence key，規則不同。不能一邊採精確右最小定義，一邊說父鍵永不需要更新。

B+ 刪除的借位、合併與範圍讀取

```
erase(k):
    path,node=find_leaf(k)
    if k absent: return
    remove record k from node
    while true:
        refresh_local(node)
        if path empty:
            if node internal and len(node.entries)==1:
                root_id=node.entries[0].child_page_id; free(node)
            else: write(node)
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        if node meets its nonroot minimum occupancy:
            write(node); parent.entries[i]=entry(node)
        else:
```


B+ 刪除的借位、合併與範圍讀取 (續)

```
            read adjacent sibling(s) of the same parent
            if a sibling can lend:
                move a boundary record or complete cached entry
                refresh_local both; write both; update their parent entries
            else:
                merge adjacent records or cached-entry arrays
                repair both leaf links if leaves
                refresh_local/write merged page; free the other page
                replace their two parent entries by entry(merged)
        node=parent
range(lo,hi):
    locate first record>=lo
    scan leaf links until key>hi or leaf chain ends
```

取L=3，根[10,14]，葉[4,9],[10,12],[14,16]，最低2筆。刪10使中葉[12]缺額；兩兄弟各2不能借，與左葉合併為[4,9,12]，根只剩[14]，葉鏈接[4,9,12]↔[14,16]。再刪14使右葉[16]缺額；左葉有3可借，移12到右葉前，得到[4,9]↔[12,16]，根separator更新12。若只刪右葉最小但未缺額，精確separator仍要更新。

B+ 刪除的借位、合併與範圍讀取 （續）

借位／合併都移動完整本頁entry；不讀被移動的子頁，也不改子頁parent。借位後仍沿保存stack傳播minimum，不能因結構缺額消失而漏掉祖先separator更新。內頁合併最多2a−1≤M個entry，葉合併最多2ell−1≤L筆；同層合併保留葉深。根若只剩一子，直接採其page-id為root。每層常數個父／兄弟／相鄰葉頁I/O，總O(h+1)，RAM搬移最壞O(L+Mh)。

區間[9,14]在最後狀態先定位左葉9，再沿next讀12，遇16>14停止，輸出[9,12]。以一般高度h與非根葉最低ell筆計，索引I/O為O(h+1+k/ell)；只有M、L都與block容量B同階時，才化成O(1+log_B n+k/B)。若葉只存record pointer且資料分散，再加D個資料頁，最壞D=k。Θ(k)輸出工作另計。

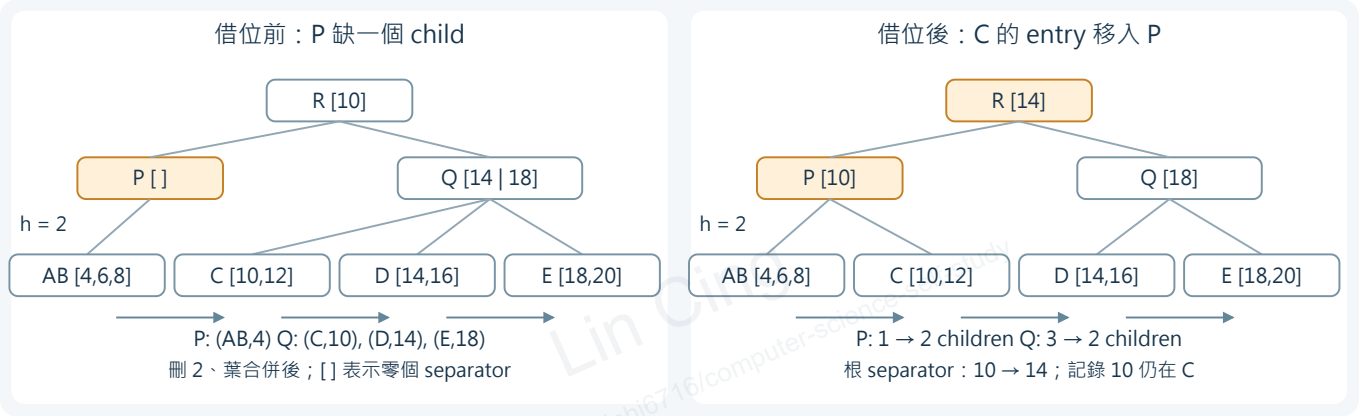
內部缺額：先看 child 數，不只看 separator

取 M=4、L=3：非根內頁至少 2 個 child，非根葉至少 2 筆記錄。根 R 有子頁 P、Q；P 的葉為 A=[2,4]、B=[6,8]，Q 的葉為 C=[10,12]、D=[14,16]、E=[18,20]。所有葉深度為 2；R 的 separator 是 10。

步驟	葉與內頁的實際變化	必須繼續處理的原因
刪除 2	A 只剩 [4]	1 筆低於葉最低 2 筆
檢查 B	B=[6,8]，不能出借	出借後 B 也只剩 1 筆
合併 A、B	保留 A 的頁 ID，改稱 AB=[4,6,8]；釋放 B	3 筆恰好不超過 L
更新 P	P.entries=[(AB,4)]；P.minimum=4	**只有 1 個 child，低於內頁最低 2 個**

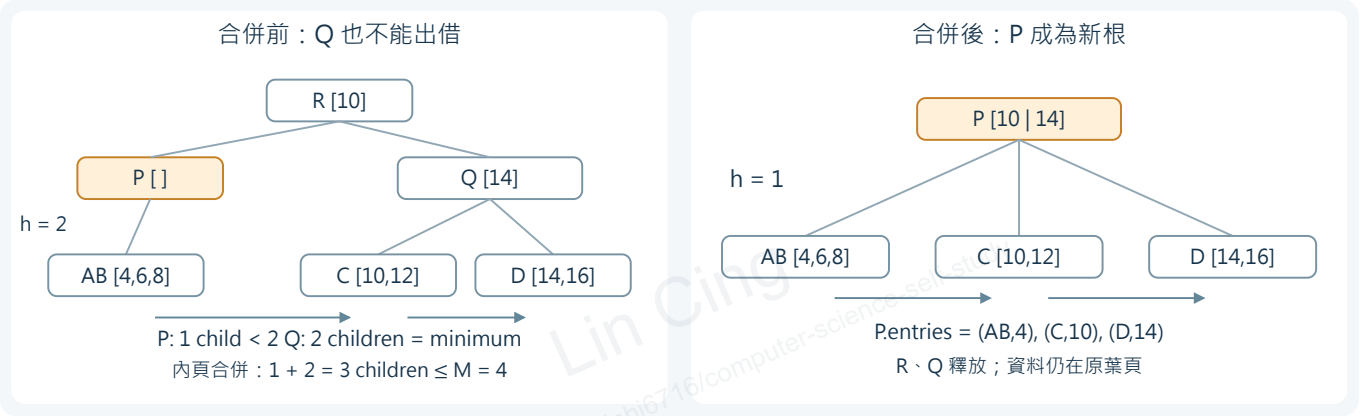
P 沒有 separator，卻仍是一個指向葉 AB 的內頁，不是空葉。葉鏈由 A↔B↔C 改成 AB↔C；C.prev 改指向 AB。根暫時仍含 (P,4)、(Q,10)。這是一次操作中的暫態，修好 P 後才交付合法索引。

內頁借位：移動整筆 entry，葉資料留在原頁



Q 原有 3 個 child，可以把最左的 (C,10) 移到 P 尾端。P 變成 [(AB,4),(C,10)]，Q 變成 [(D,14),(E,18)]；兩者各有 2 個 child，達到最低佔用。P 的 separator 為 10，Q 的 separator 為 18，根由 [(P,4),(Q,10)] 改成 [(P,4),(Q,14)]。

兄弟不能借：合併內頁，再收縮根



另取一棵沒有 E 的初始樹，其他容量與 A、B、C、D 不變。刪 2、合併 A/B 後，P 只有 (AB,4)，Q 只有 (C,10)、(D,14)，不能出借。串接兩頁的 entry 得 [(AB,4),(C,10),(D,14)]，保留 P、釋放 Q；P 的 separators 由本頁副本重建為 [10,14]。

R 只剩 (P,4) 一個 child，故令 root_id=P、釋放 R，高度由 2 降為 1。

用搜尋與葉鏈核對重組結果

兩個分支各自只刪除記錄 2。內頁移動的是頁 ID 與 minimum 副本；C 的實體資料仍是 [10,12]。父 separator 10 不是另一筆待插入葉的資料。

檢查	內頁借位後	內頁合併、縮根後
內頁與高度	R=[14]；P=[10]、Q=[18]；h=2	新根 P=[10,14]；h=1
查 10	R 向左到 P，相等向右到 C	P 在 10 處相等向右到 C
葉鏈	AB↔C↔D↔E，共 9 筆記錄	AB↔C↔D，共 7 筆記錄
葉記錄 10	只在 C 出現一次	只在 C 出現一次

在縮根分支查閉區間 [8,15]：先在 AB=[4,6,8] 找到 8，沿 next 到 C=[10,12]，再到 D=[14,16]。依序輸出 8、10、12、14；檢查到 16>15 時停止。圖中箭頭只畫 next，prev 指向同一條鏈的前一頁。內頁重組沒有改動這條葉鏈。

為什麼內頁修復不遺失資料，也不增加樹高？

內頁 entry 按子樹最小鍵排序。向右兄弟借最左 entry，等於把有序子樹序列的分界右移一格；向左兄弟借最右 entry 則相反。子樹內部不變，所有 child 深度相同，因此排序與葉深度都保留。精確 separator 直接由新 entry 序列重建，祖先 minimum 副本則繼續向上更新。

令 a=[M/2]。缺額內頁有 a−1 個 child，不能出借的兄弟有 a 個；合併後為 2a−1≤M 個 child。M 偶數時為 M−1，奇數時為 M。本例 1+2=3≤4。**B+ 合併串接 child entries，不額外插入父 separator 作為資料**；新內頁的 separator 數自然是 child 數減一。

每次向上只處理當前頁、父頁與至多兩個相鄰兄弟；葉合併另修常數個葉鏈欄位。本章沒有持久 child.parent，搬 entry 不必改寫被搬動的子頁。故每層常數頁 I/O，最壞 O(h+1)；這是漸近界，精確讀寫數仍取決於哪些頁已在 buffer（頁面緩衝）中。只有收縮根會使所有葉共同減少一層。

B+ 高度的分支與葉容量前提

```
a=ceil(M/2) >= 2; ell=ceil(L/2) >= 1
h=number of edges on a root-to-leaf path
if h>=1:
    leaf_count >= 2*a^(h-1)
    n >= 2*ell*a^(h-1)
    h <= 1+log_a(n/(2*ell))
// Root-only trees h=0 cost one page, including the empty tree.
```

根為內頁時至少2子，之後每個非根內頁至少a子。因此深度h至少2a^(h−1)個葉；每葉至少ell筆，得到上式。根是葉的h=0情形單獨處理。一般定位成本為h+1頁，故可統一寫O(1+log_a(1+n/ell))，不能只由M≥3、L≥2便把底數換成B。

反例取任意整數q≥3，M=3、L=B=2^q，所有內頁恰2子，所有2^q個葉各滿載B筆。樹高h=q、記錄n=B^2，合法冷頁點查需q+1頁；log_B n卻恆等於2。當q增長，O(log_B n)常數界不成立。失敗原因是內頁最低分支固定2，沒有隨B增長。

B+ 高度的分支與葉容量前提 （續）

只有另假設固定大小的key／entry／record、一節點一頁，且M=Θ(B)、L=Θ(B)，才有a、ell=Θ(B)，高度化為O(1+log_B n)。一般range回傳k筆時，除首尾可能只取部分頁，其餘掃過的葉至少含ell筆輸出，因此索引I/O為O(h+1+k/ell)。加上上述兩個容量條件，才成O(1+log_B n+k/B)。

點查與範圍查詢 I/O

根葉定位為O(h+1)頁，一般h界見上一節，不預設M、L隨B增長。範圍查詢的索引頁成本為O(h+1+k/ell)。只有M=Θ(B)、L=Θ(B)、每點一頁及固定項目大小，才可寫O(1+log_B n+k/B)。

葉若僅存record pointer，完整資料另讀D個資料頁，最壞D=k；聚集且每頁Θ(B)筆時，資料頁可再受O(1+k/B)控制。輸出k筆仍需Ω(k)基本輸出工作，頁I/O與CPU工作是不同分母。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

理論 B+ 與資料庫實作

- PostgreSQL 18 官方文件描述多層 B-tree 索引，各層頁面有雙向鏈，葉項目指向資料列，內部項目指向下一層頁。
- 這支持「多路頁面索引」的實務動機，但不是本教材簡化刪除演算法的逐步實作保證。
- 真正資料庫另需並發控制、分裂中的讀取一致性、WAL 與崩潰恢復。單執行緒指標圖沒有處理這些問題。
- 本章不提供可直接部署的磁碟 B+ 引擎；練習以明確容量和單一更新序列驗證不變量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

來源與定義對照

- 外部記憶體與最低佔用的高度論據核對 Pat Morin 的 Open Data Structures § 14.2。該來源以 B 至 2B 子數定義，本章採 t 至 2t 子數，符號已分離。
- 資料庫實例核對 PostgreSQL 18 B-tree implementation。本章只引用頁面和項目配置事實，不複製其內部實作文件。
- B+ 的 M、L、separator 與分裂例子為本教材獨立慣例，尚未核對指定參考書第二版採用哪種 order 定義。
- 練習題 與 推導解答 各自獨立。授課前須保留本章容量約定，不能省略後再要求唯一答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study